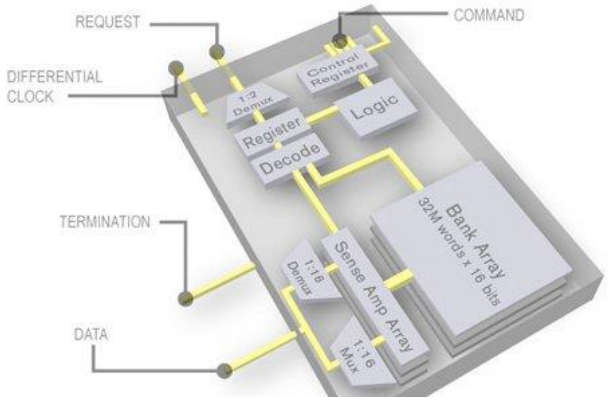
Logic and Computer Design Fundamentals

Chapter 7 – Memory Basics



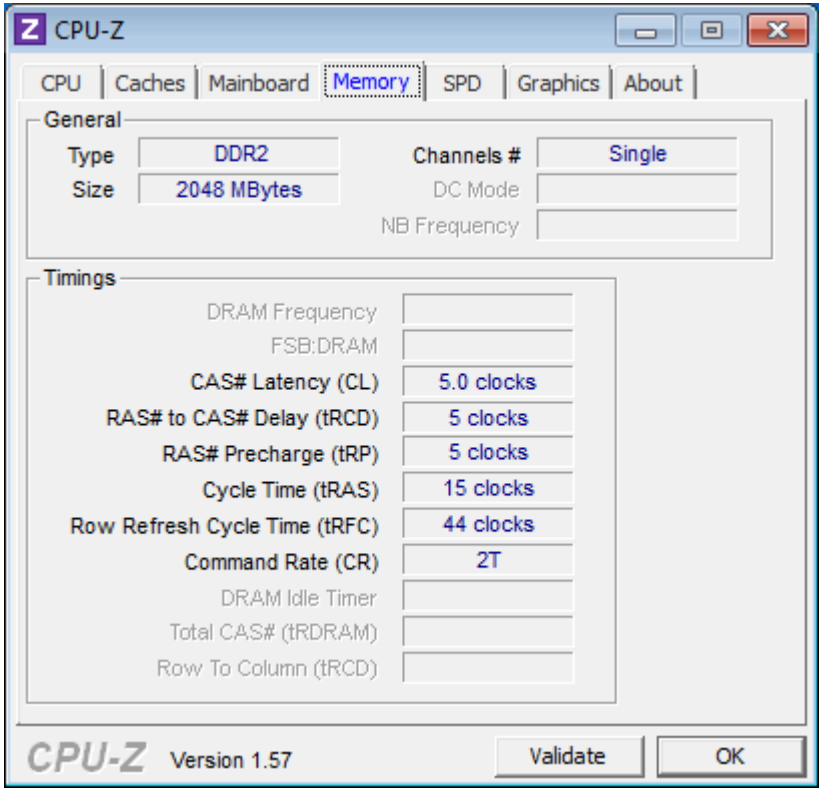
Ming Cai
cm@zju.edu.cn
College of Computer Science and Technology
Zhejiang University

Overview

- Memory definitions
- Random Access Memory (RAM)
- Static RAM (SRAM) integrated circuits
  - Cells and slices
  - Cell arrays and coincident selection
- Arrays of SRAM integrated circuits
- Dynamic RAM (DRAM) integrated circuits
- DRAM Types
  - Synchronous (SDRAM)
  - Double-Data Rate (DDR SRAM)
  - RAMBUS DRAM (RDRAM)
- Arrays of DRAM integrated circuits

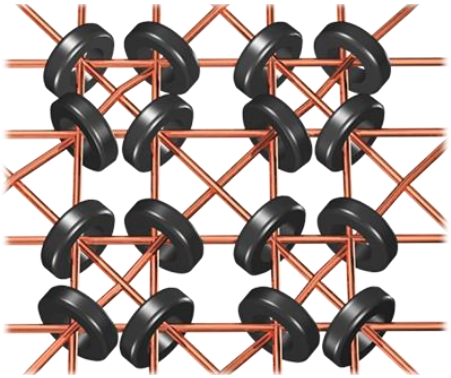
Memory Specification

- **Are you familiar with the technical specifications reported for memory by the profiling tool CPU-Z?**



History of RAM (Random Access Memory)

- **Early Memory Technologies (1940s–1950s)**
  - **Delay Line Memory (1940s) – Used sound waves in mercury-filled tubes to store data (e.g., UNIVAC).**
  - **Williams-Kilburn Tube (1940s) – Used cathode-ray tubes (CRTs) to store bits as dots on a screen.**
- **Magnetic Core Memory (1950s–1970s)**
  - **used tiny magnetic rings (cores) with wires**
  - **non-volatile, expensive and relatively slow**
  - **dominant memory before semiconductor RAM**



An Wang (王安)
1920-1990

History of RAM (Random Access Memory) (continued)

- **Semiconductor RAM (1960s–1970s)**
  - **SRAM (Static RAM, 1964) – Developed by Robert Dennard at IBM. It used flip-flop circuits to store data and did not require refreshing.**
  - **DRAM (Dynamic RAM, 1966) – Also invented by Robert Dennard at IBM. DRAM stored data in capacitors, requiring periodic refreshing but was cheaper and denser than SRAM.**

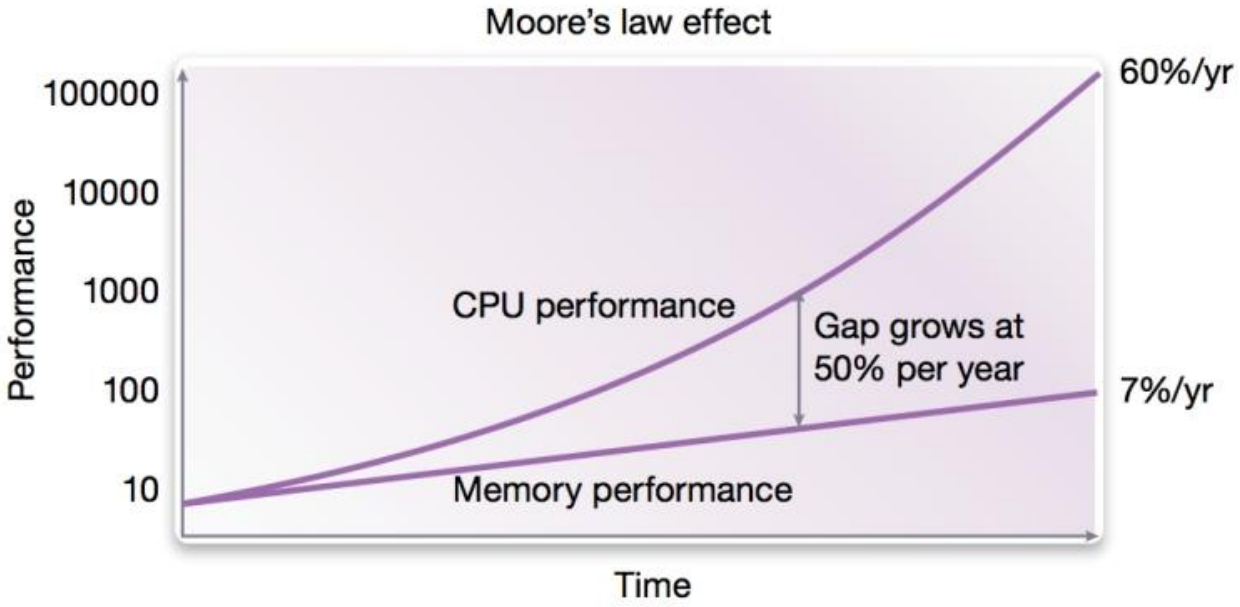
  - **First Commercial DRAM Chip (1970):**
    - **Intel 1103 (1Kbit DRAM) – The first commercially successful DRAM chip, marking the shift from magnetic core memory to semiconductor memory.**

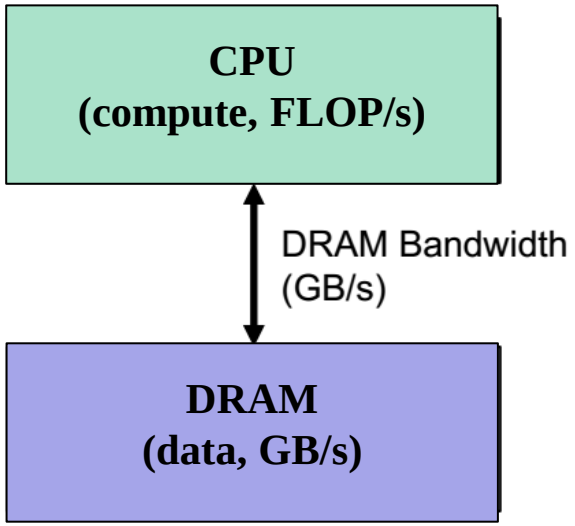
History of RAM (Random Access Memory) (continued)

- **DRAM Evolution (1980s–1990s)**
  - **FPM DRAM (Fast Page Mode): Improved access times.**
  - **EDO DRAM (Extended Data Out): Faster than FPM.**
  - **SDRAM (Synchronous DRAM): Introduced by Samsung in 1993, synchronized with the CPU clock for better efficiency.**
- **DDR SDRAM (2000s–Present)**
  - **DDR (2000): First DDR standard, doubling bandwidth over SDRAM.**
  - **DDR2 (2003): Higher speeds, lower power consumption.**
  - **DDR3 (2007): Even faster, more efficient.**
  - **DDR4 (2014): Higher capacity, lower voltage, mainstream until the early 2020s.**
  - **DDR5 (2020–Present)**

Memory Wall

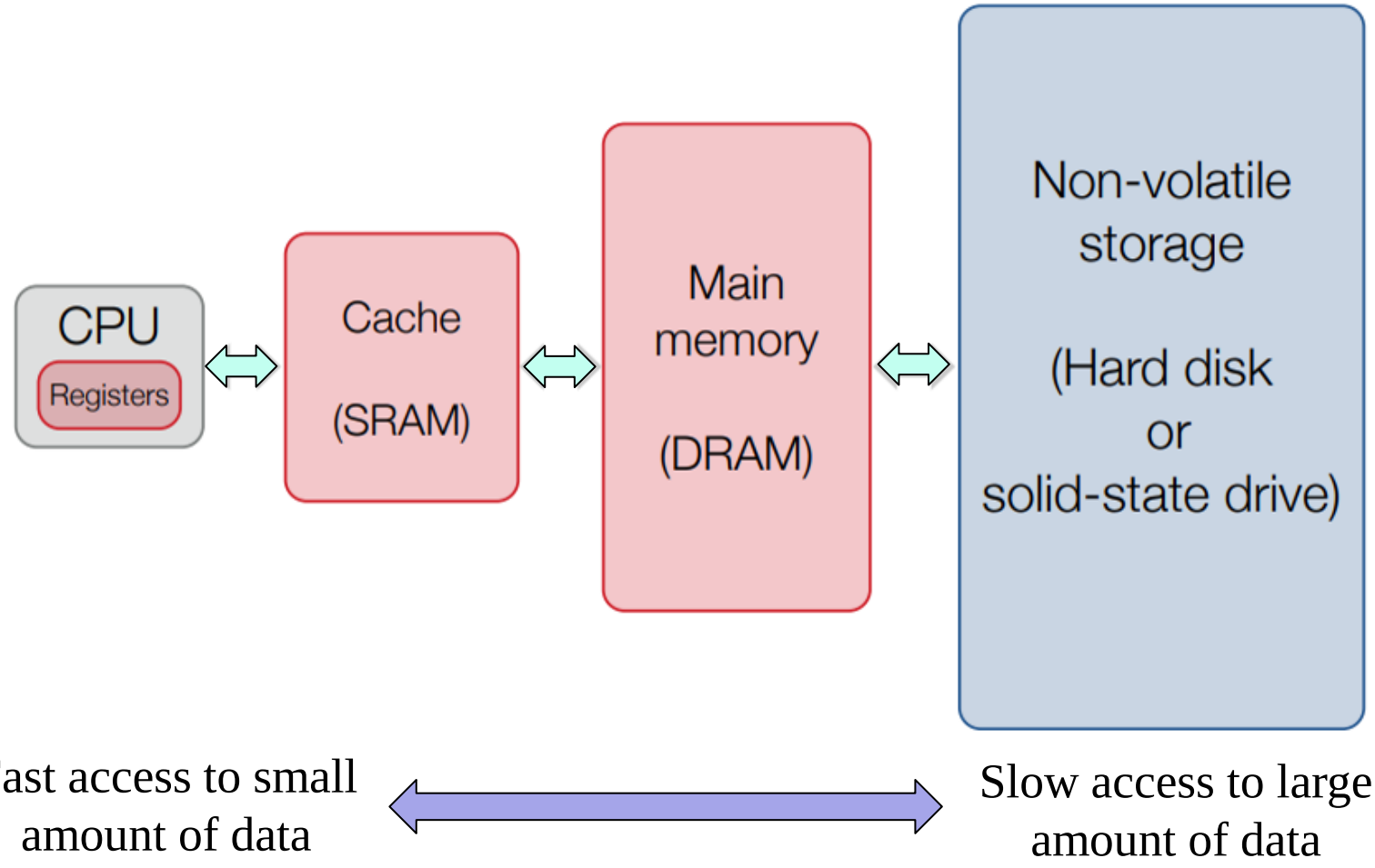
- **The speed disparity between CPU and memory continues to grow.**
- **The processor performance is limited by memory (memory wall).**





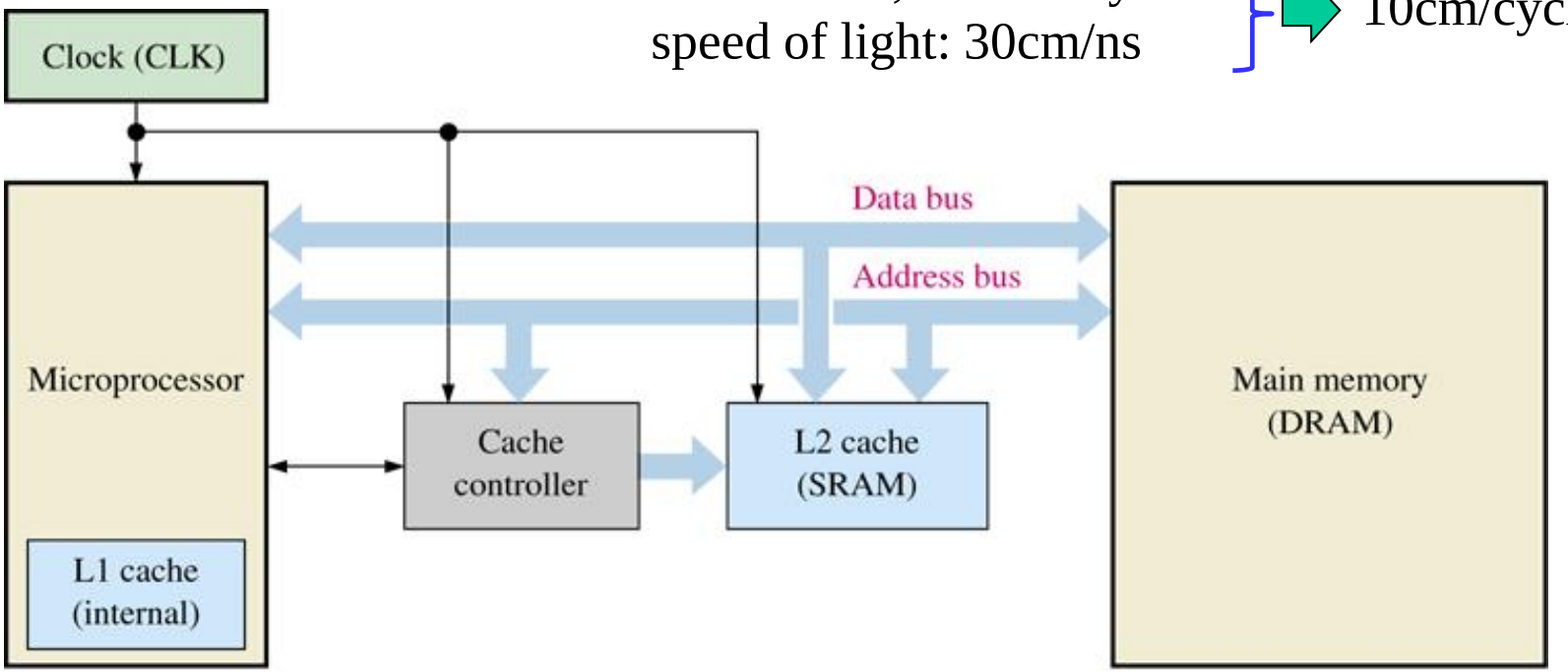
Memory Hierarchy in Computers

- **Good memory hierarchy design is important to the overall performance of a computer system.**



Memory Hierarchy in Computers (continued)

3GHz CPU, 0.33 ns/cycle
speed of light: 30cm/ns
→ 10cm/cycle



	L1 Cache	L2 Cache	Main memory
Speed (ns) :	0.1 - 1	1 - 10	10 - 100
Size (bytes):	K	10K	M – G
Cost :	highest	⟶	lowest

Latency Comparison Numbers

1	Latency Comparison Numbers (~2012)		
2	----------		
3	L1 cache reference	0.5	ns
4	Branch mispredict	5	ns
5	L2 cache reference	7	ns
6	Mutex lock/unlock	25	ns
7	Main memory reference	100	ns
8	Compress 1K bytes with Zippy	3,000	ns
9	Send 1K bytes over 1 Gbps network	10,000	ns
10	Read 4K randomly from SSD*	150,000	ns
11	Read 1 MB sequentially from memory	250,000	ns
12	Round trip within same datacenter	500,000	ns
13	Read 1 MB sequentially from SSD*	1,000,000	ns
14	Disk seek	10,000,000	ns
15	Read 1 MB sequentially from disk	20,000,000	ns
16	Send packet CA->Netherlands->CA	150,000,000	ns

from https://gist.github.com/jboner/2841832

Memory Definitions

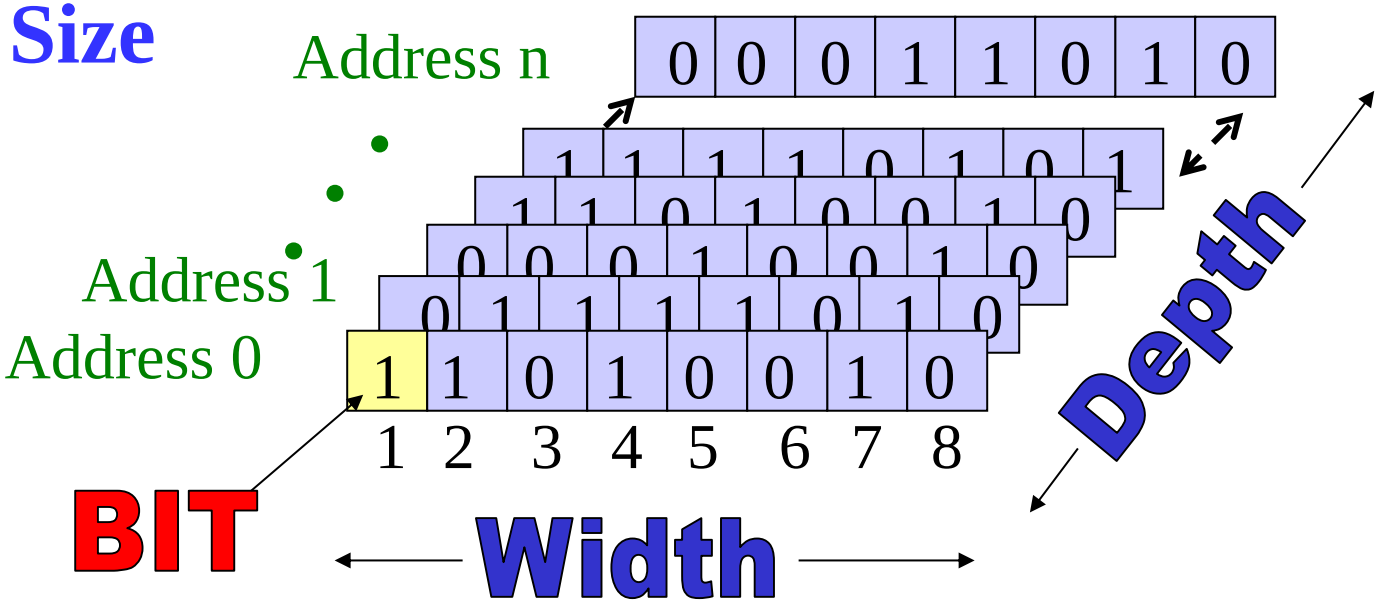
- **Memory — A collection of storage cells together with the necessary circuits to transfer information to and from them.**
- **Memory Organization — the basic architectural structure of a memory in terms of how data is accessed.**
- **Random Access Memory (RAM) — a memory organized such that data can be transferred to or from any cell (or collection of cells) in a time that is not dependent upon the particular cell selected.**
- **Memory Operations — operations on memory data supported by the memory unit. Typically, *read* and *write* operations over some data element (bit, byte, word, etc.).**

Memory Definitions (Continued)

- **Typical data elements are:**
  - **bit — a single binary digit**
  - **byte — a collection of eight bits accessed together**
  - **word — a collection of binary bits whose size is a typical unit of access for the memory. It is a power of two multiple of bytes (e.g., 2 bytes, 4 bytes, etc.)**
- **Memory Data — a bit or a collection of bits to be stored into or accessed from memory cells.**
- **Memory Address — A vector of bits that identifies a particular memory element.**
- **Memory Size — address depth × word width, e.g., 2K × 8, 32M × 16.**

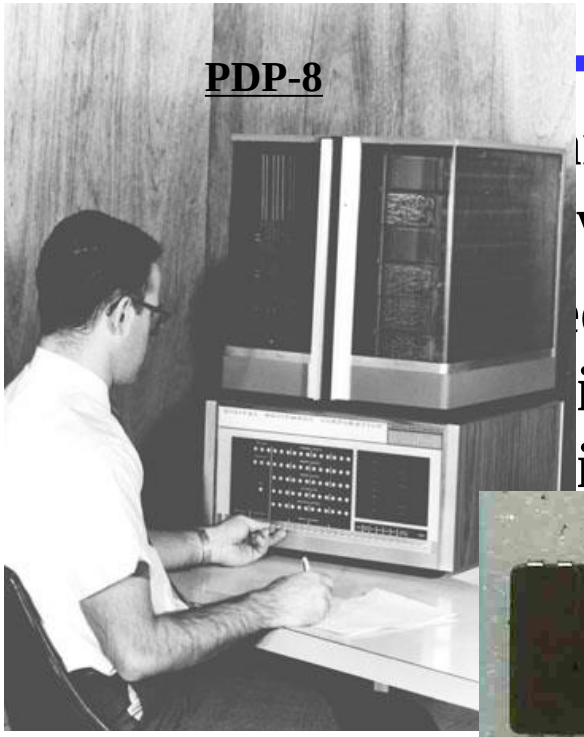
Memory Definitions (Continued)

- **Data Element: bit, byte, word**
- **Address Depth (Words #)**
- **Word Width (Bits # per word)**
- **Memory Size**

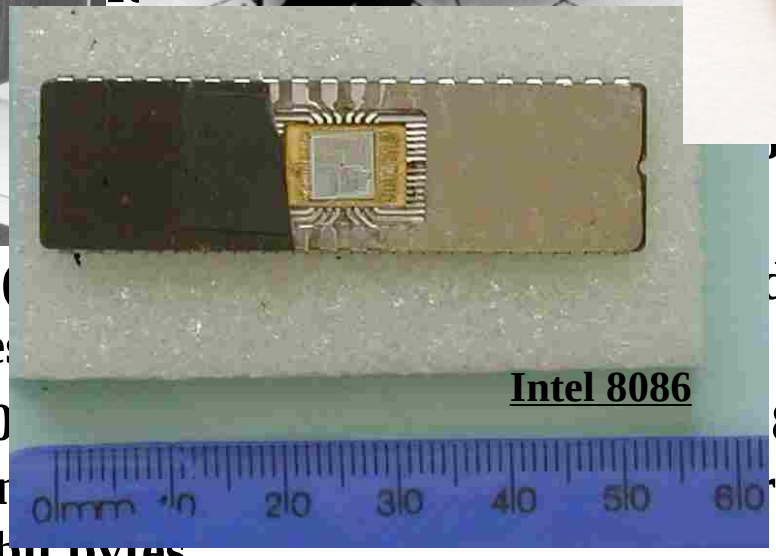


Memory Size = Address Depth × Word Width

Memory Organization



PDP-8



Intel 8086

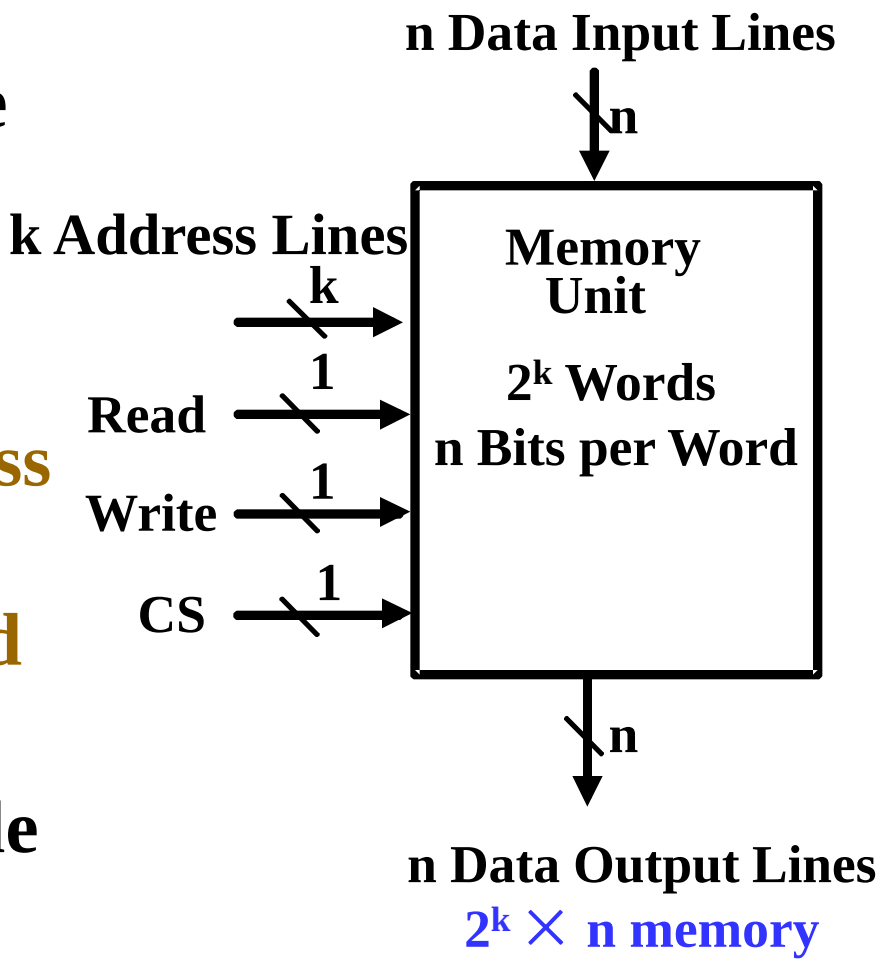
- IBM 360 (ddress 16,777,216 8-bit byte
- Intel 8080 8086 and the current Ir ress to address 65,536 8-bit bytes.

"640K ought to be enough for anybody."

——Bill Gates (1981)

Memory Block Diagram

- **Chip select (CS) or chip enable (CE) to enable the memory.**
- **k address lines are decoded to address 2^k words of memory (address depth).**
- **Each word is n bits (word width).**
- **Read and Write are single control lines defining the simplest of memory operations.**



Memory Organization Example

- **Example memory contents:**
  - **A memory with 3 address bits and 8 data bits has:**
  - **k = 3 and n = 8 so $2^3 = 8$ addresses labeled 0 to 7.**
  - **$2^3 = 8$ words of 8-bit data**

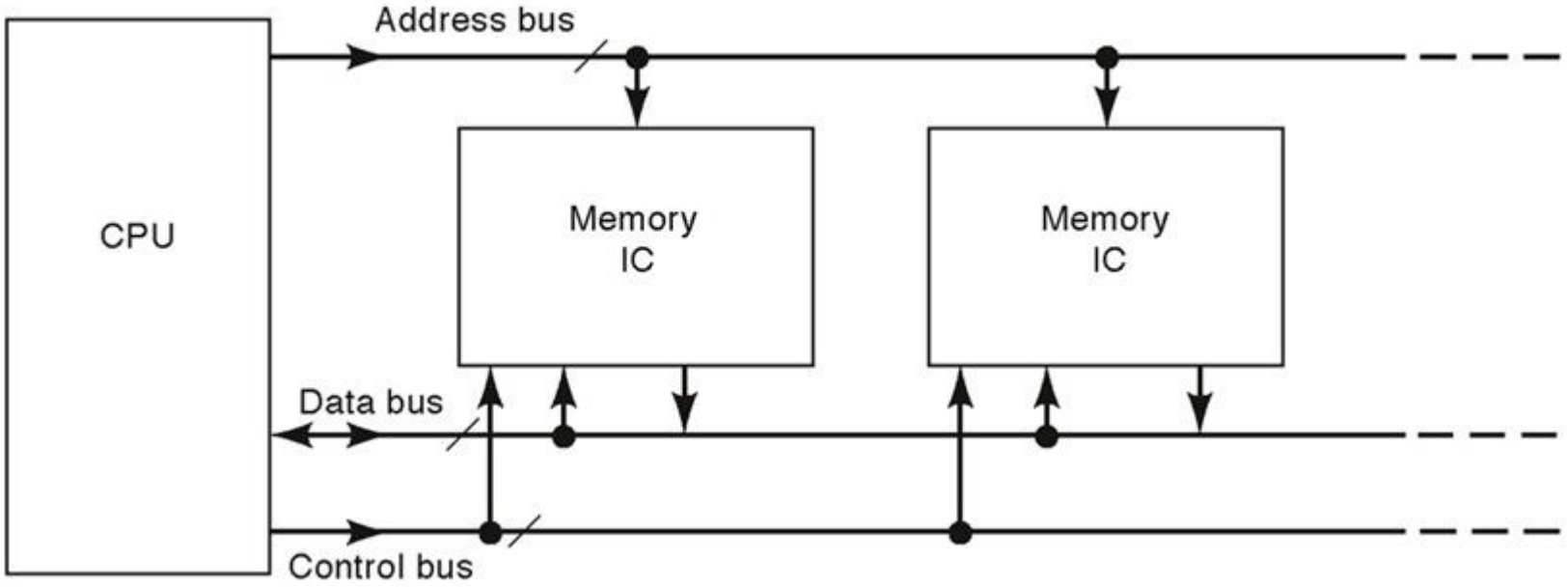
Memory Address		Memory Content
Binary	Decimal	
000	0	1 0 0 0 1 1 1 1
001	1	1 1 1 1 1 1 1 1
010	2	1 0 1 1 0 0 0 1
011	3	0 0 0 0 0 0 0 0
100	4	1 0 1 1 1 0 0 1
101	5	1 0 0 0 0 1 1 0
110	6	0 0 1 1 0 0 1 1
111	7	1 1 0 0 1 1 0 0

Basic Memory Operations

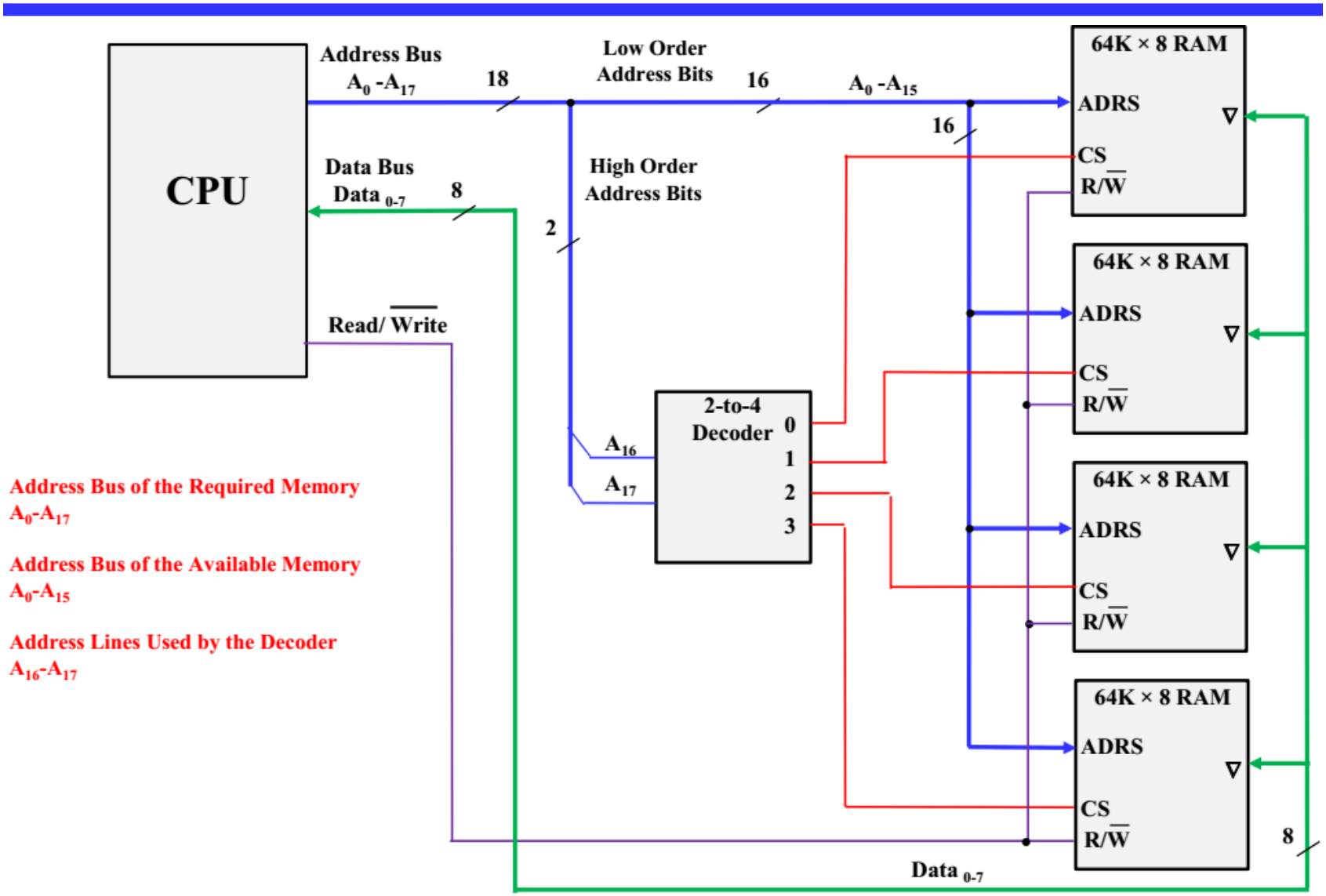
- **Memory operations require the following:**
  - ***Address*** **— specifies the memory location to operate on. The address lines carry this information into the memory. Typically: n bits specify locations of 2^n words.**
  - ***Data*** **— data written to, or read from, memory as required by the operation.**
  - **An operation — Information sent to the memory and interpreted as control information which specifies the type of operation to be performed. Typical operations are READ and WRITE. Others are READ followed by WRITE and a variety of operations associated with delivering blocks of data. Operation signals may also specify timing information.**

Basic Memory Operations

- **Three buses connect the memory to CPU:**
  - ***Address bus*** **— specifies the memory location.**
  - ***Data bus*** **— contains data to/from memory location.**
  - ***Control bus*** **— additional control signals such as chip select, read/write, etc.**

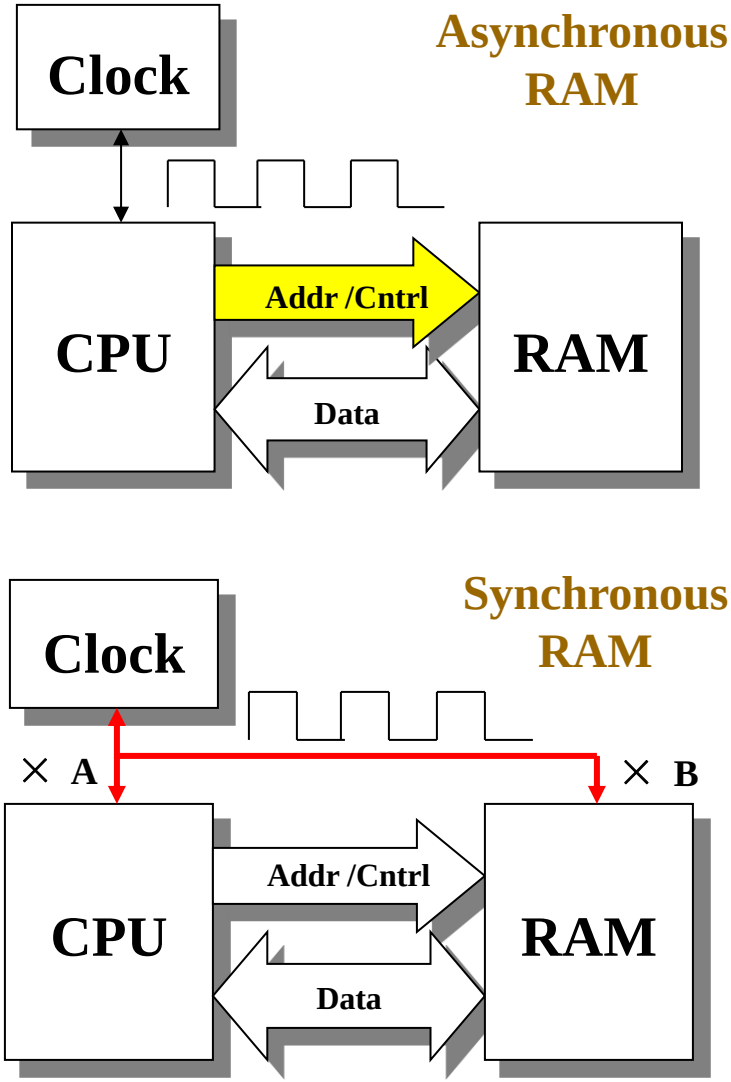


Example: CPU Interface with 256KB RAM



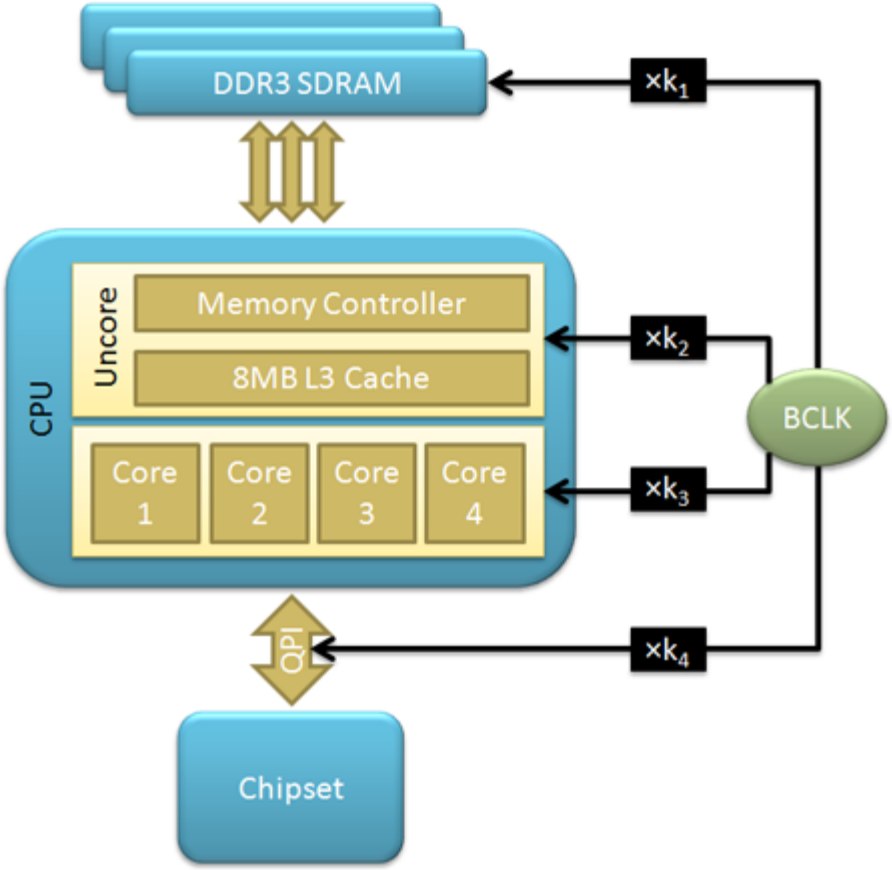
Asynchronous vs. Synchronous RAM

- An asynchronous RAM does not depend on the state of an external clock. It will read or write data as soon as it receives the instruction.
- The synchronous RAM (e.g., DDR) is synchronized with an external clock. It will read and write data into the memory on particular states of the clock.



Clock Frequency in a Computer

- **BCLK, or Base Clock**
- **CPU frequency**
  - **BCLK × CPU multiplier**
- **Uncore frequency**
  - **BCLK × Uncore multiplier**
- **Memory frequency**
  - **BCLK × Memory multiplier**



Basic Memory Operations (continued)

Chip Select CS	Read/$\overline{\text{Write}}$ R/$\overline{\text{W}}$	Memory Operation
0	×	None
1	0	Write to selected word
1	1	Read from selected word

- **Read Memory — an operation that reads a data value stored in memory:**
  1. **Place an address on the address lines.**
  2. **Toggle the memory read control line.**
  3. **Wait for the read data to become stable.**

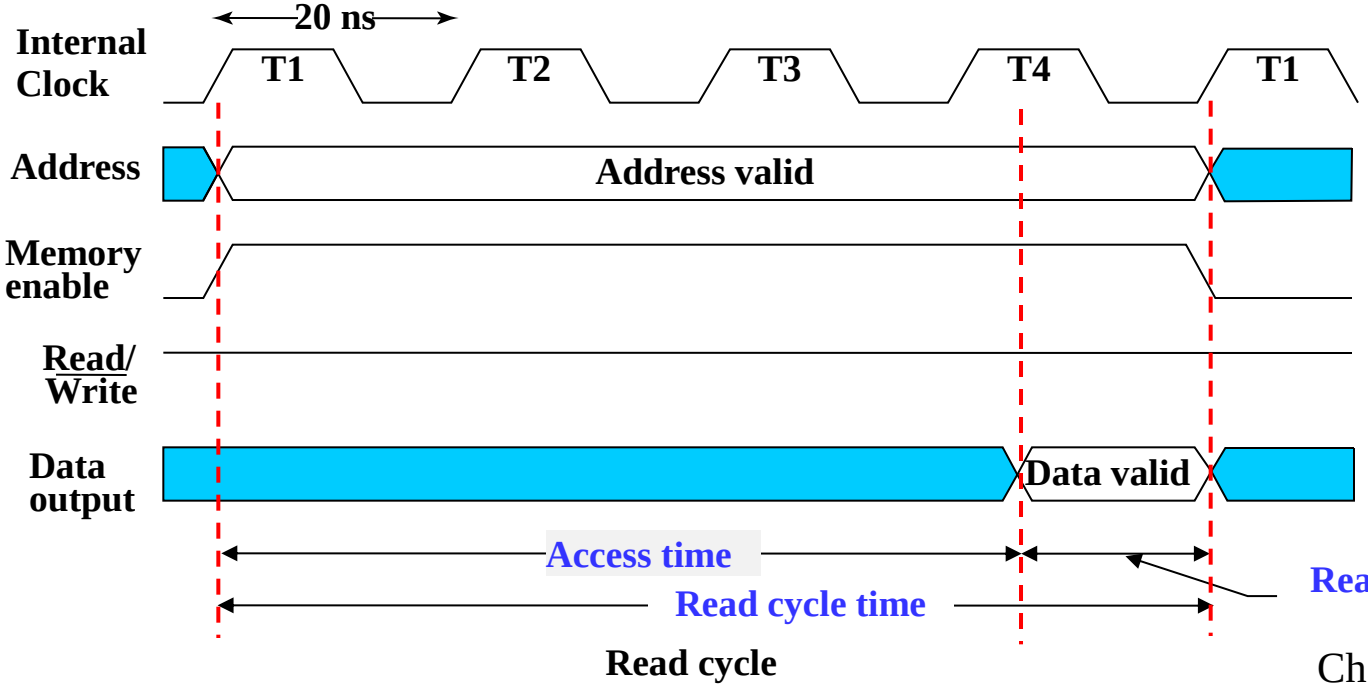
Basic Memory Operations (continued)

Chip Select CS	Read/Write R/$\overline{W}$	Memory Operation
0	×	None
1	0	Write to selected word
1	1	Read from selected word

- **Write Memory — an operation that writes a data value to memory:**
  1. **Place an address on the address lines and valid data on the data lines.**
  2. **Toggle the memory write control line.**
- **Sometimes the read or write enable line is defined as a clock with precise timing information (e.g., Read Clock, Write Strobe).**
  - **Otherwise, it is just an interface signal.**
  - **Sometimes memory must acknowledge that it has completed the operation.**

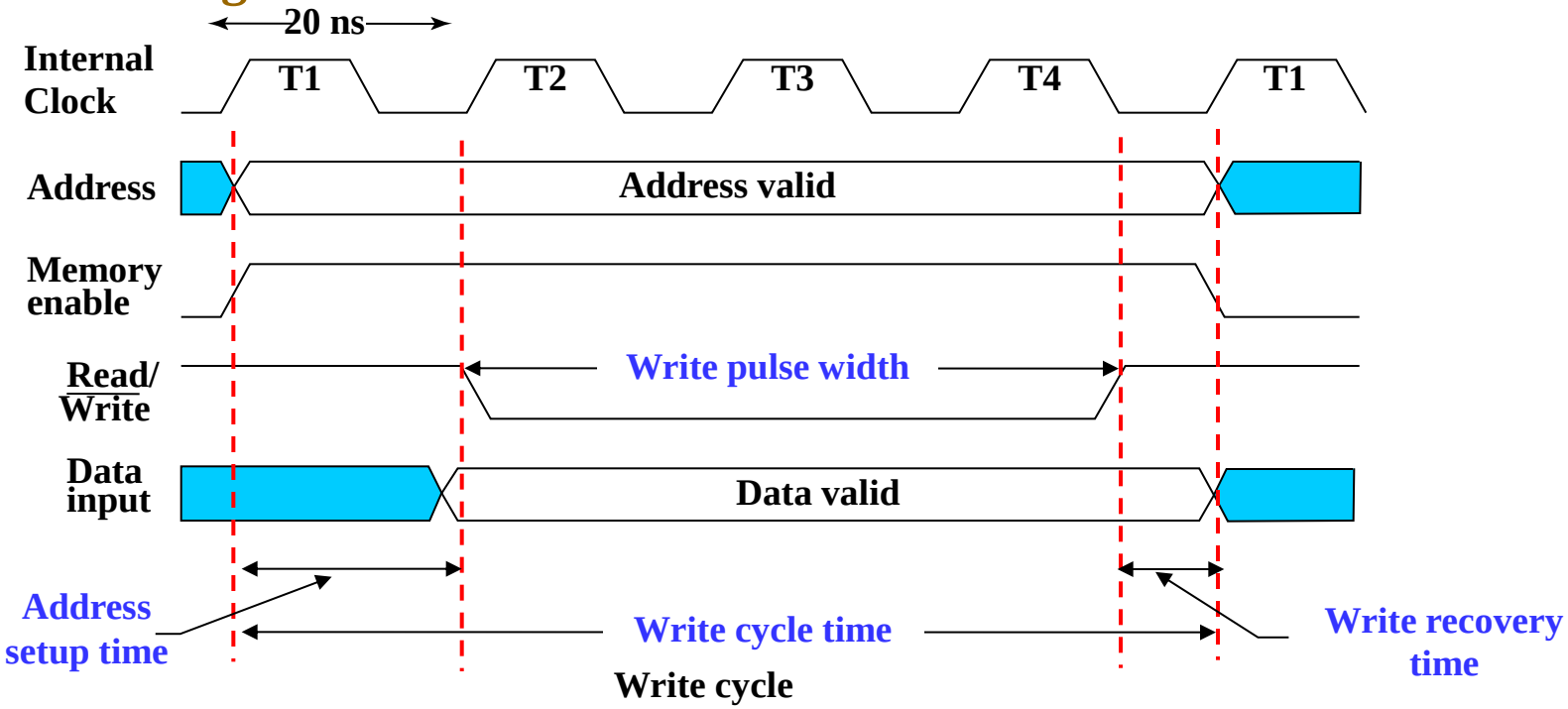
Memory Operation Timing

- Most basic memories are asynchronous
  - Storage in latches or storage of electrical charge
  - No external clock
- Controlled by control inputs and address
- Timing of signal changes and data observation is critical to the operation
- Read timing:



Memory Operation Timing

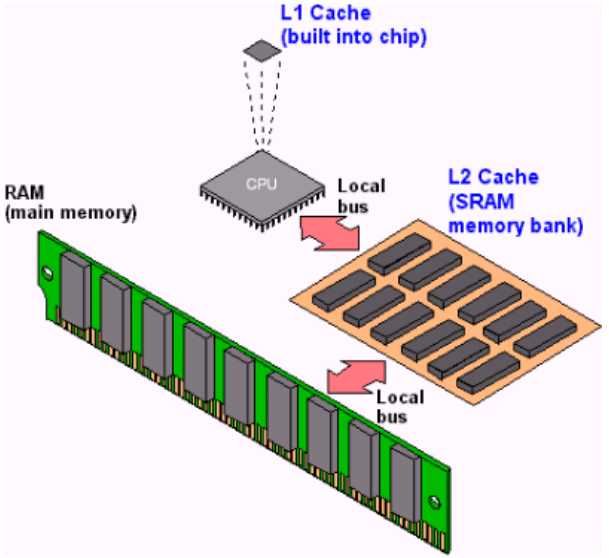
- **Write timing:**



- **Critical times measured with respect to edges of write pulse (1-0-1):**
  - **Address must be established at least a specified time before 1-0 and held for at least a specified time after 0-1 to avoid disturbing stored contents of other addresses.**
  - **Data must be established at least a specified time before 1-0 and held for at least a specified time after 0-1 to write correctly.**

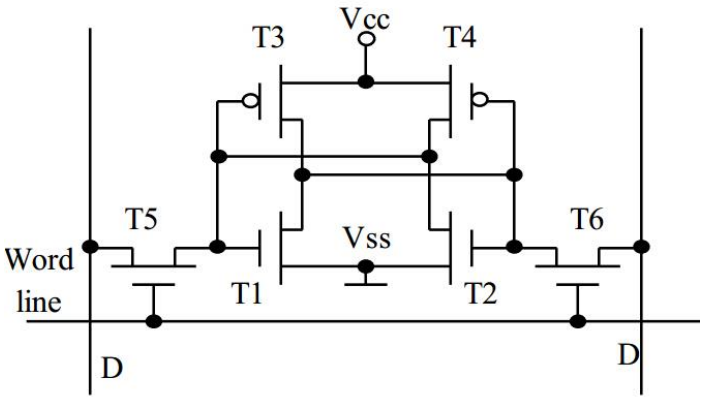
RAM Integrated Circuits

- **Types of random access memory**
  - ***Static*** **– information stored in latches**
  - ***Dynamic*** **– information stored as electrical charges on capacitors**
    - **Charge "leaks" off**
    - **Periodic *refresh* of charge required**
- **Dependence on Power Supply**
  - ***Volatile*** **– loses stored information when power turned off**
  - ***Non-volatile*** **– retains information when power turned off**

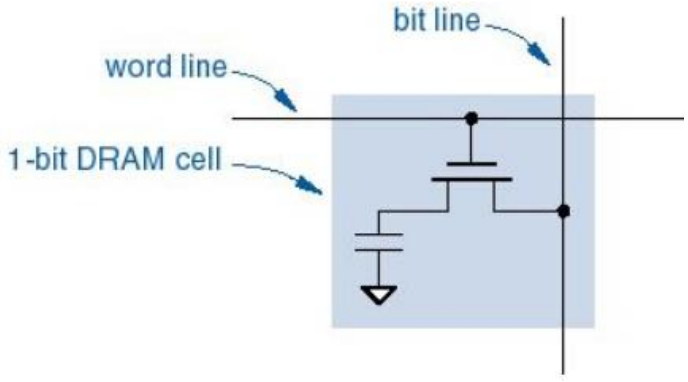


Static vs. Dynamic RAM

Static RAM	Dynamic RAM
SRAM uses transistor to store data	DRAM uses capacitor to store data
SRAM does not need periodic refreshment to maintain data	DRAM needs periodic refreshment to maintain data
SRAM's structure is complex	DRAM's structure is simple
SRAM is expensive	DRAM is less expensive
SRAM is faster	DRAM is slower
SRAM is used in Cache memory	DRAM is used in main memory



SRAM Cell



DRAM Cell

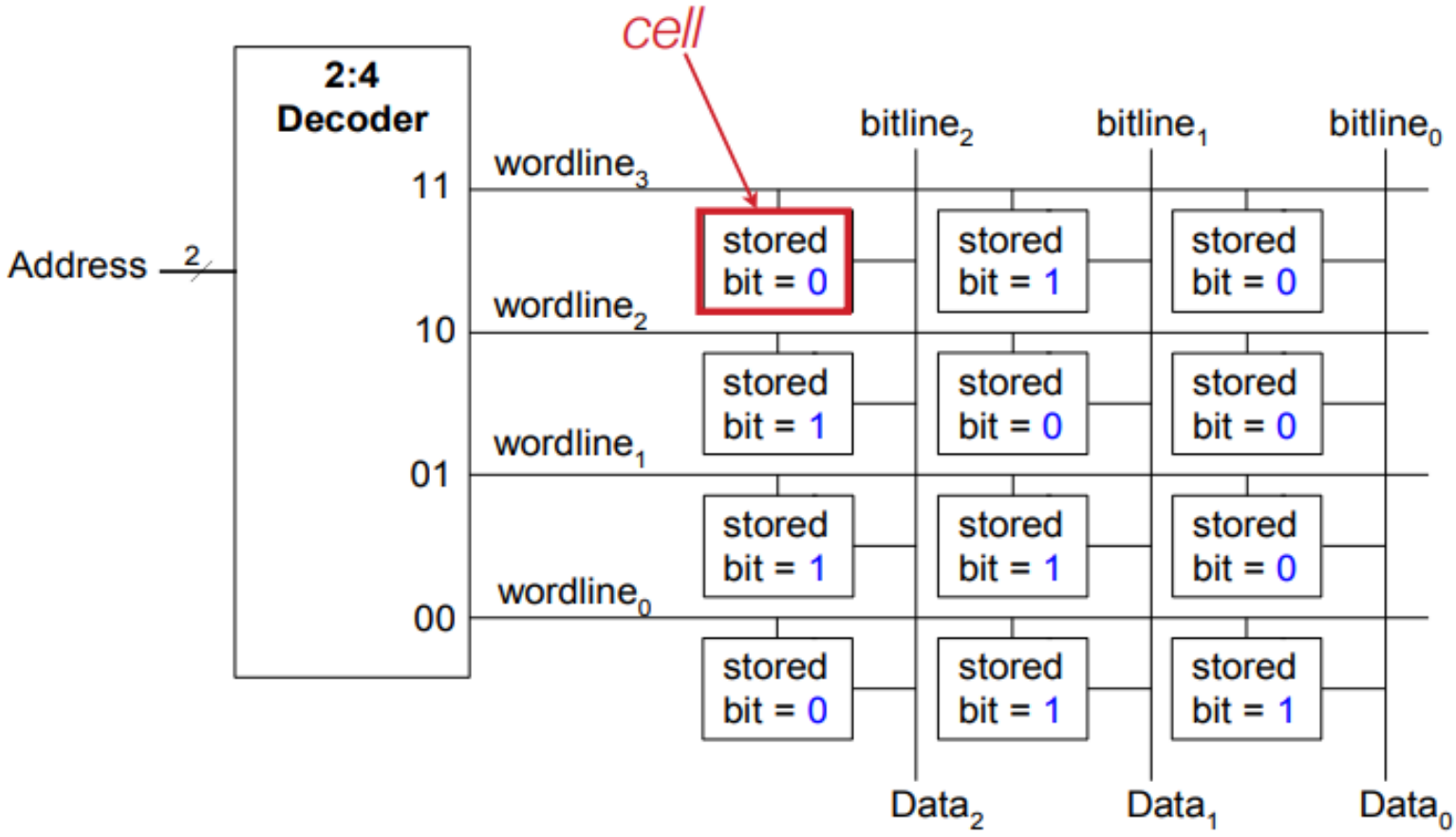
refresh

Volatile storage (RAM) comparisons

	Flip-flop	SRAM	DRAM
Transistors / bit	~20	6	1
Density	Low	Medium	High
Access time	Fast	Medium	Slow
Destructive read?	No	No	Yes (refresh required)
Power consumption	High	Medium	Low

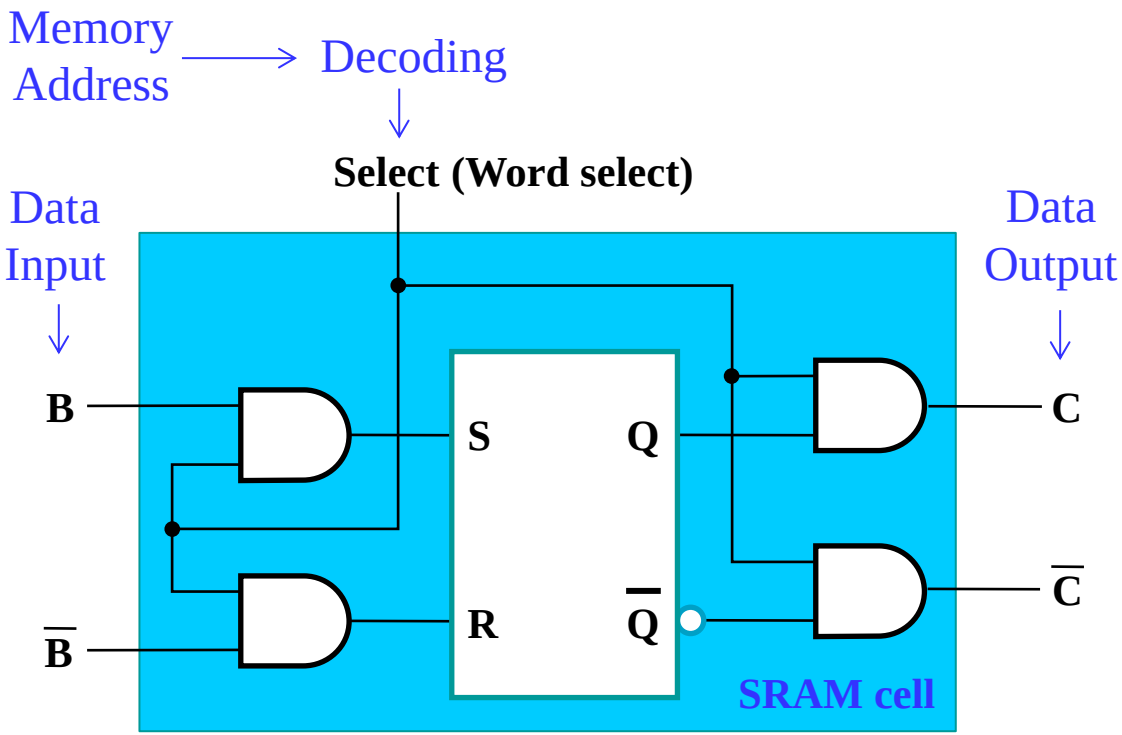
Memory Array Architecture

- **Memory is a 2D array of bits. Each bit stored in a cell.**



Static RAM (SRAM) – Cell

- **Array of storage cells used to implement static RAM**
- **Storage Cell**
  - **SR Latch**
  - **Select input for control**
  - **Dual Rail Data Inputs B and $\overline{B}$**
  - **Dual Rail Data Outputs C and $\overline{C}$**



Problem

- **Problem : An SRAM has address lines from A_0 to A_{15} and data width from D_0 to D_7. What is the total capacity of the SRAM ?**

  A. 512 KB

  B. 256 KB

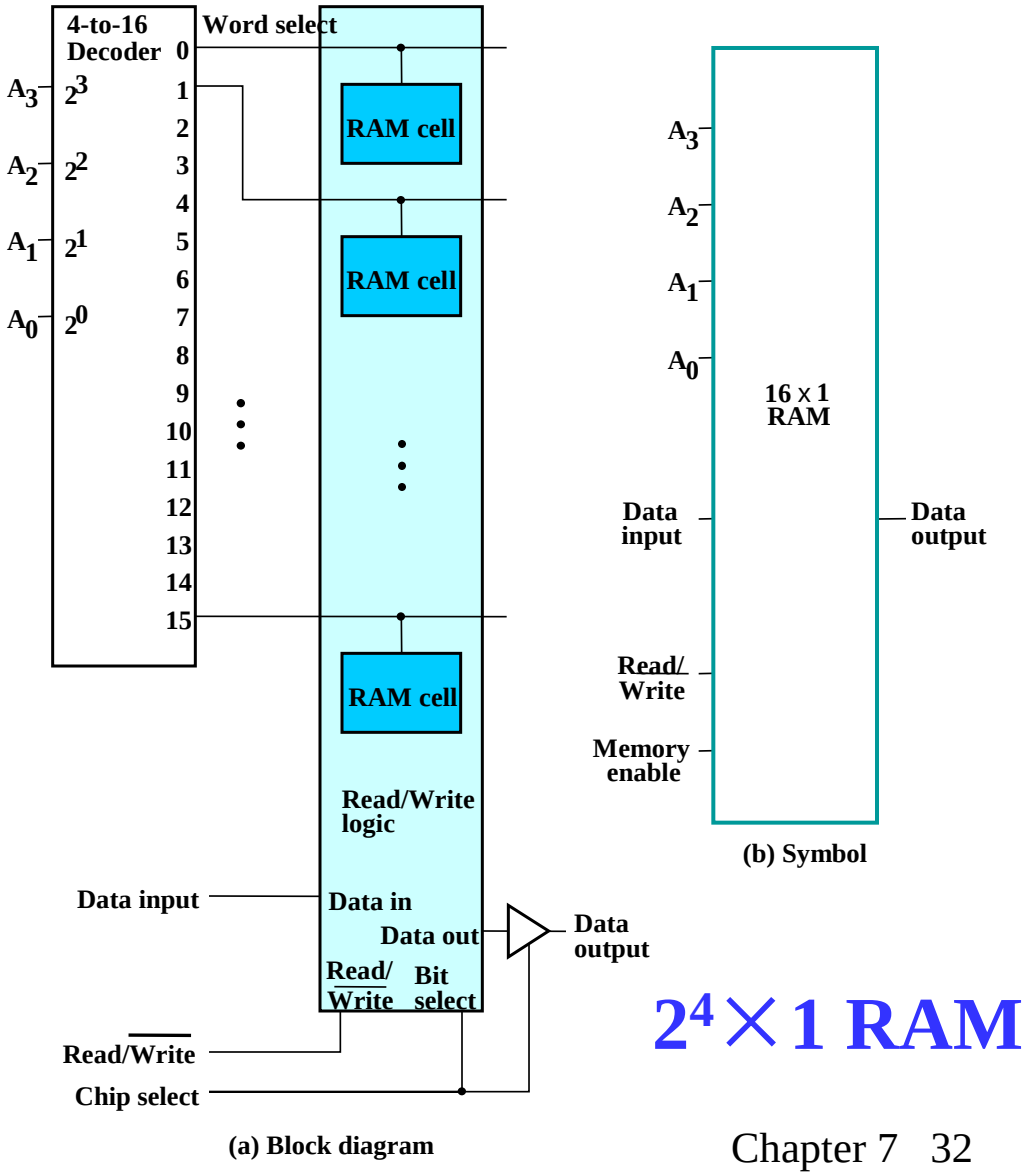
  C. 128 KB

  D. 64 KB

- **Answer:**

2^n-Word × 1-Bit RAM IC

- **To build a RAM IC from a RAM slice, we need:**
  - **Decoder — decodes the n address lines to 2^n word select lines**
  - **A 3-state buffer on the data output permits RAM ICs to be combined into a RAM with 2^n words**

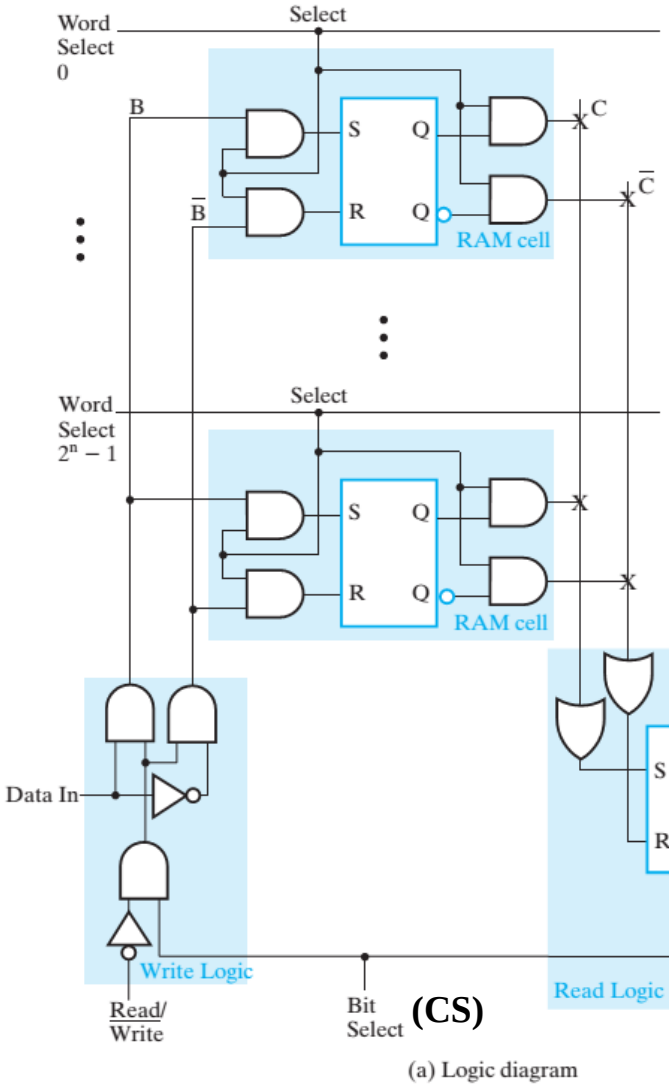


$2^4 \times 1$ RAM

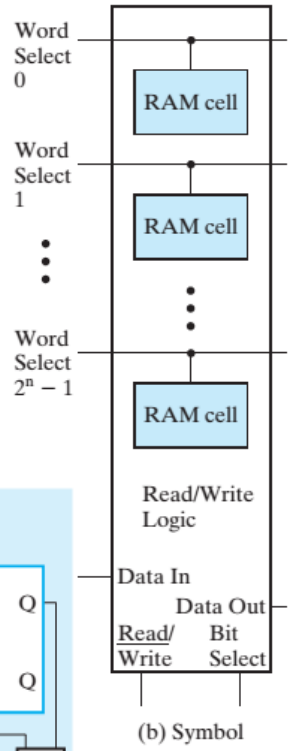
Static RAM (SRAM) – Bit Slice

- **Represents all circuitry that is required for 2^n 1-bit words**
  - **Multiple RAM cells**
  - **Control Lines:**
    - **Word select i – one for each word**
    - **Bit Select**
    - **Read / $\overline{\text{Write}}$**
  - **Data Lines:**
    - **Data in**
    - **Data out**

bidirectional pins

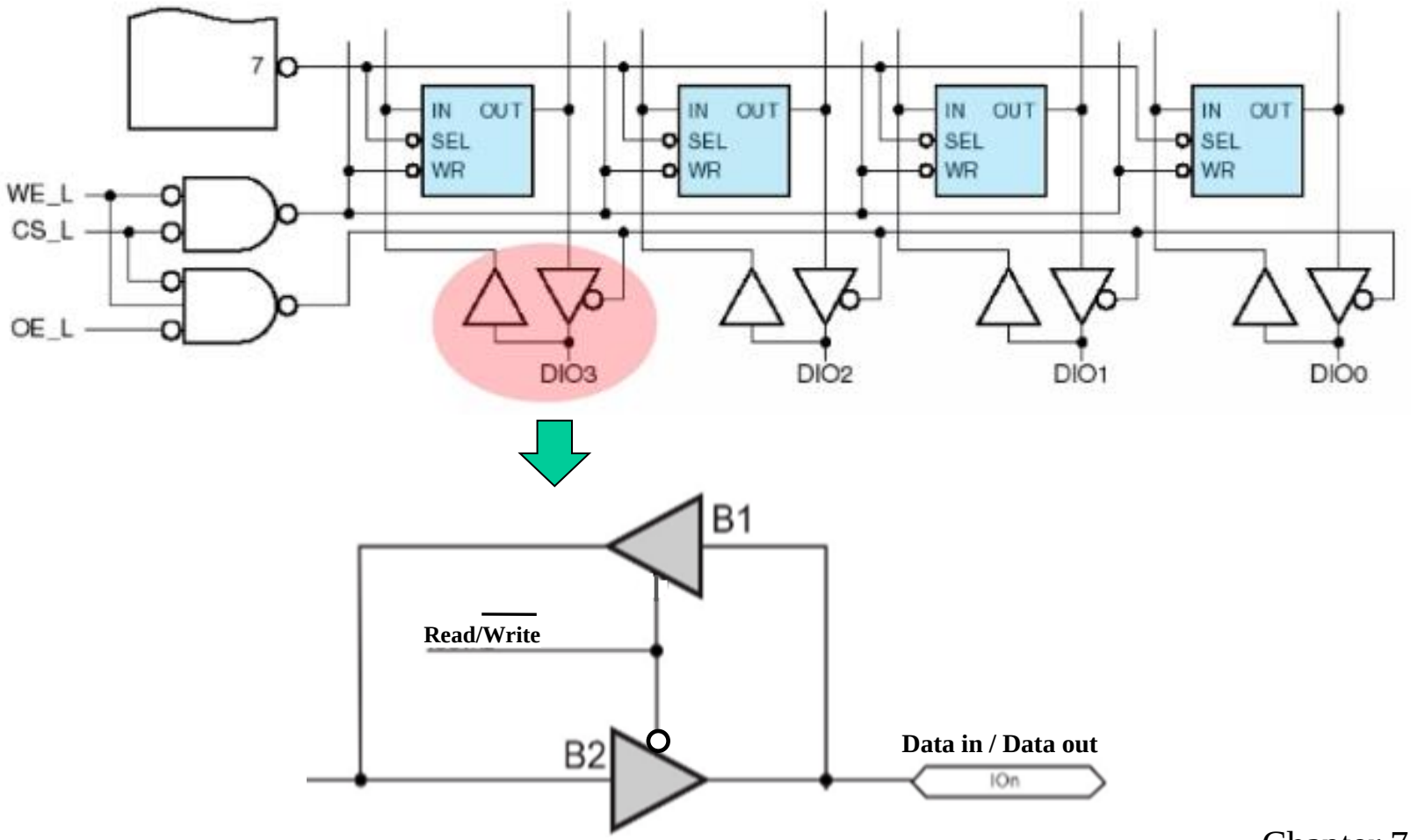


$2^n \times 1$ RAM



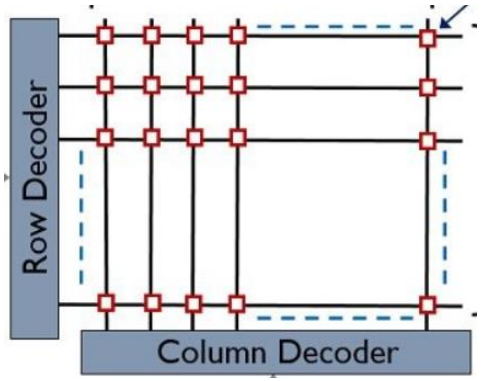
Bidirectional In/Out Data Pins

- **Uses the same data pins for reads and writes**

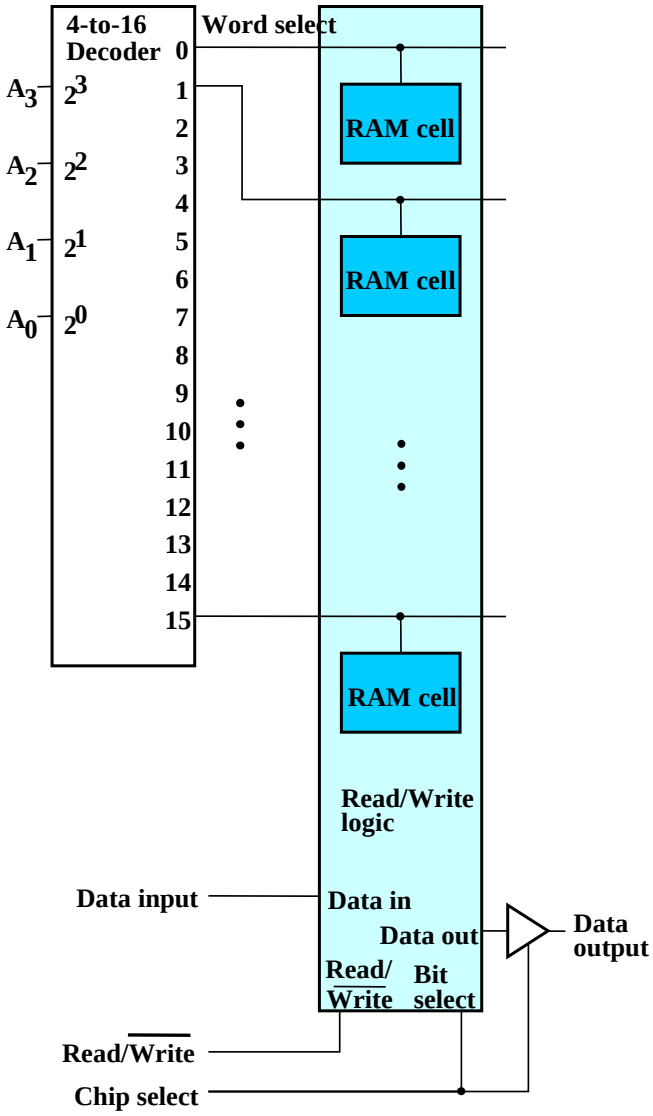


Cell Arrays and Coincident Selection

- **Memory arrays can be very large =>**
  - **Large decoders**
  - **Large fanouts for the bit lines**
  - **The decoder size and fanouts can be reduced by approximately $\sqrt{n}$ by using a coincident selection in a 2-dimensional array**
  - **Coincident selection is a 2D address decoding technique:**
    - **Uses two decoders, one for words and one for bits**
    - **Word select becomes Row select**
    - **Bit select becomes Column select**
- **For 1K*1 memory, with a single 10-to-1024-line decoder, we need 1,024 AND gates with 10 inputs.**



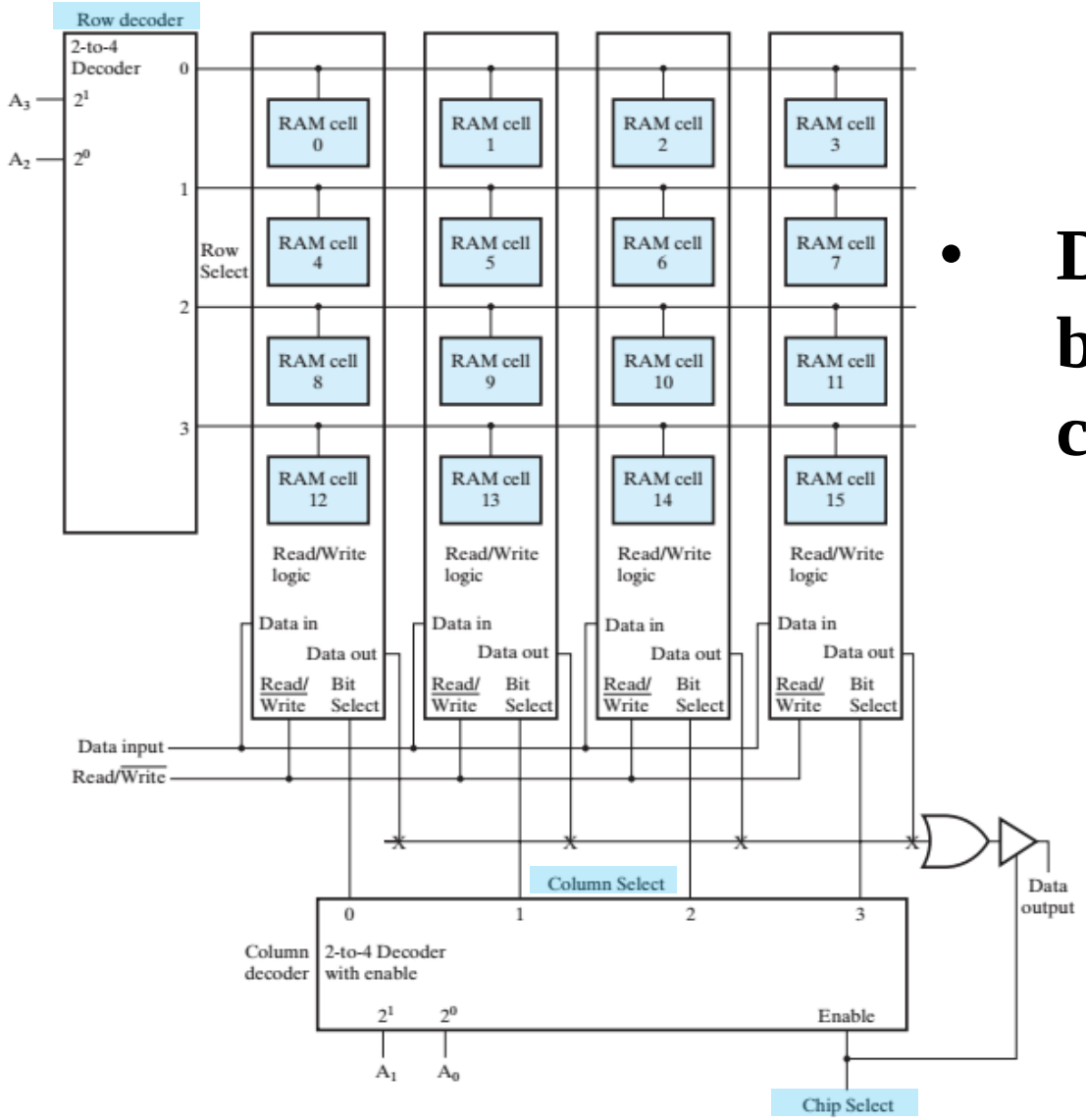
$2^4 \times 1$ RAM without Coincident Selection



- **Using a single 4-to-16-line decoder**

$2^4 \times 1$ bit RAM

$2^4 \times 1$ RAM with Coincident Selection

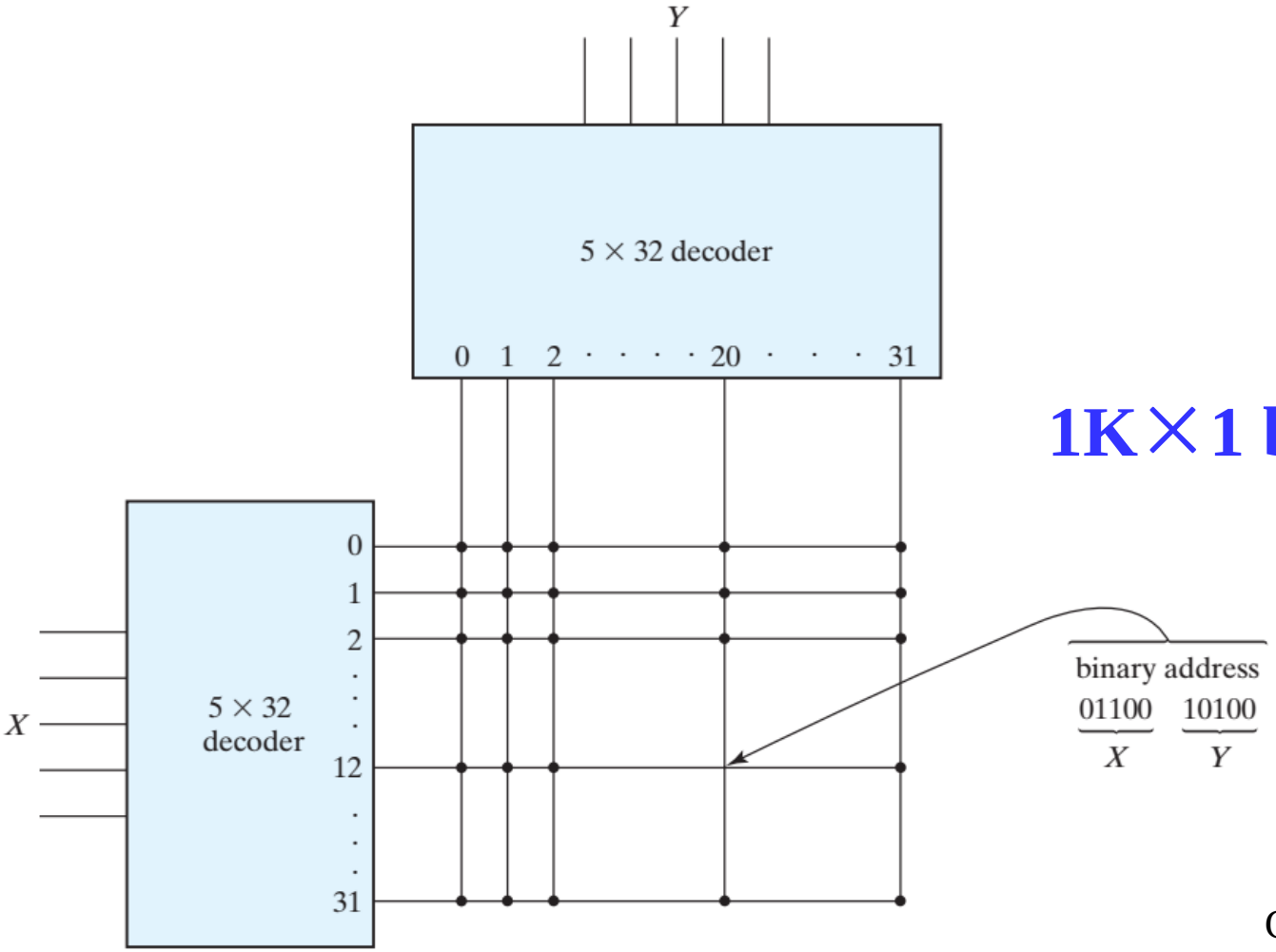


- **Decode address into both row and column select signals**

$2^4 \times 1$ bit RAM

Cell Arrays and Coincident Selection (continued)

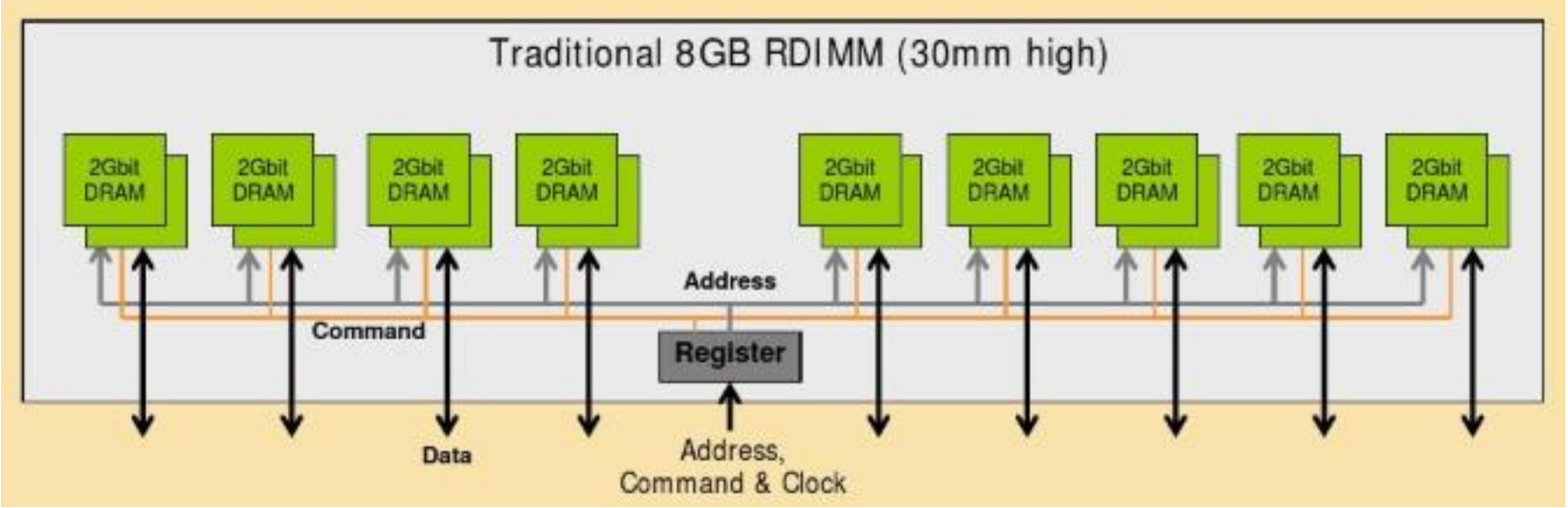
- **For 1K*1 memory, with two 5-to-32-line decoders, we need 32*2=64 AND gates with 5 inputs in each.**



1K×1 bit RAM

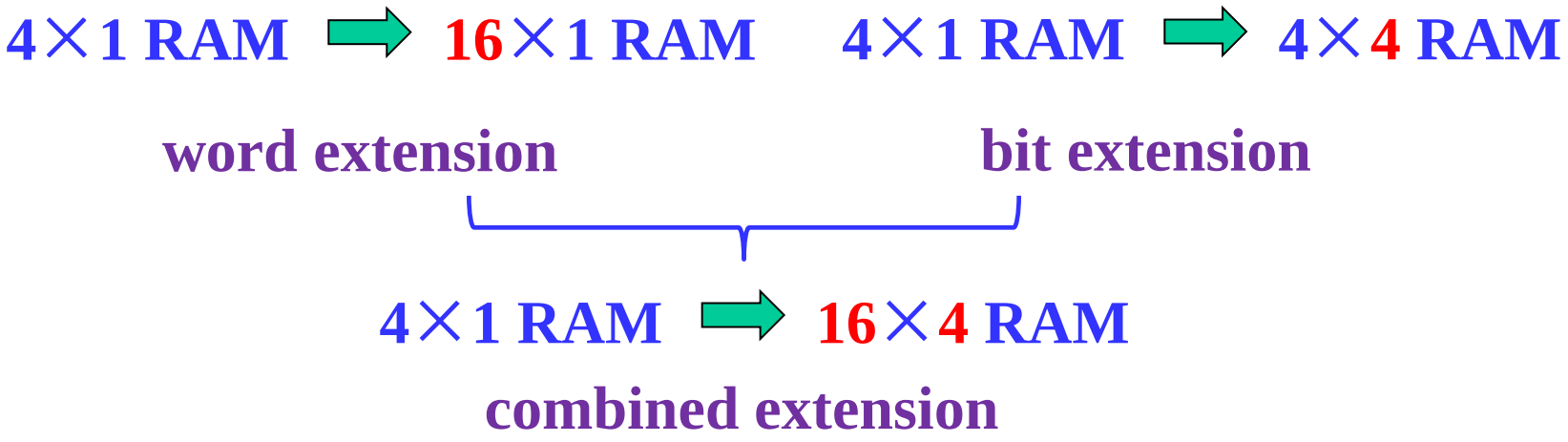
RAM ICs with > 1 Bit/Word

- **To better balance the number of words and word length, use ICs with > 1 bit/word**



RAM ICs with > 1 Bit/Word

- **Memory expansion method**
  - **Address depth or space expansion: word extension (address bits are increased)**
  - **Word width expansion: bit extension**



Problem

- **Problem : How many 4M×32 RAM chips are needed to provide a memory of capacity of 128MB?**

  A. 64

  B. 32

  C. 16

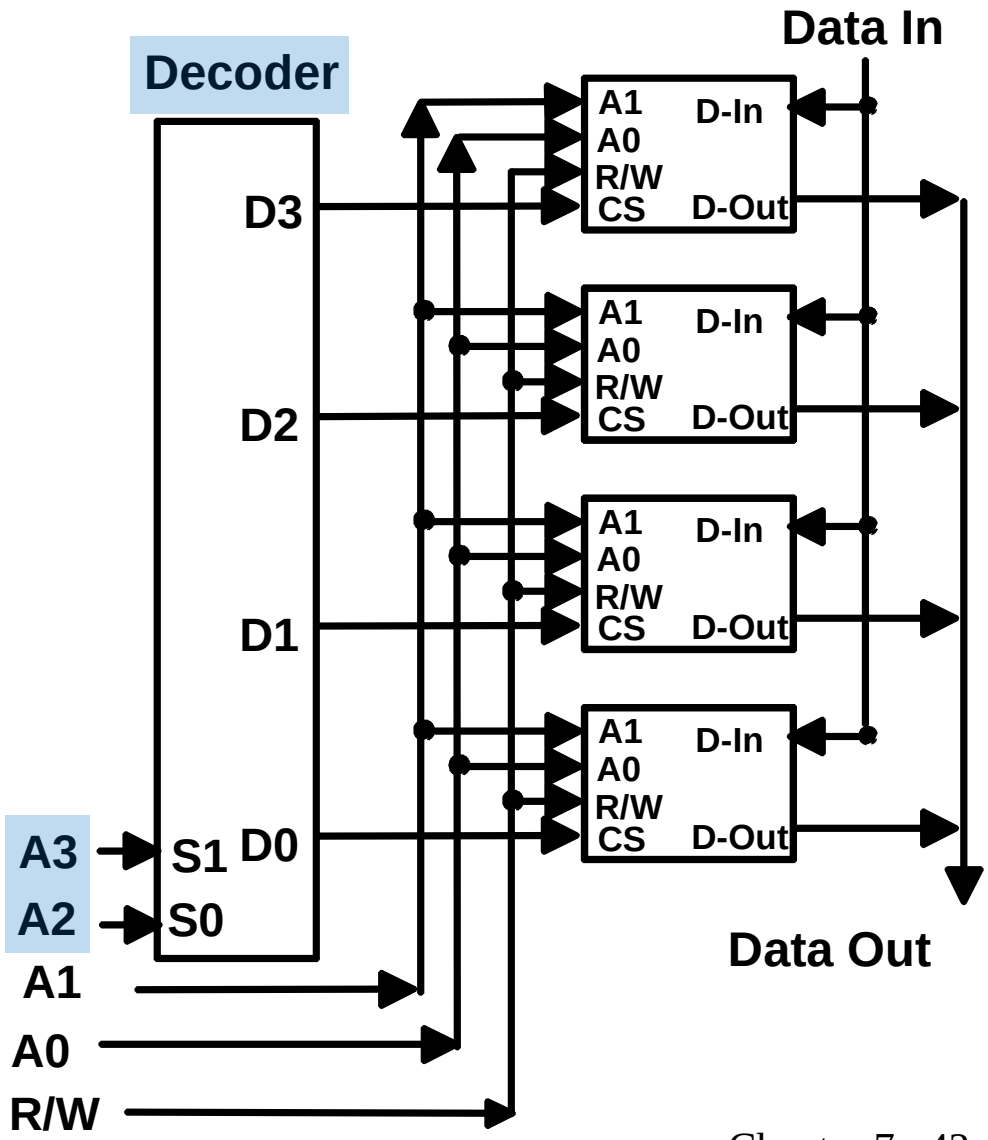
  D. 8

- **Answer:**

Making Larger Memories: Word extension

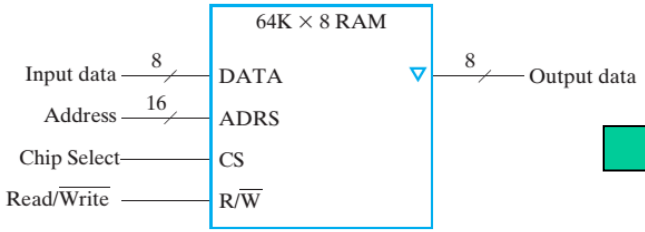
- Using the CS lines, we can make larger memories from smaller ones by tying all address, data, and R/W lines in parallel, and using the decoded higher order address bits to control CS.
- Using the 4-Word by 1-Bit memory from before, we construct a 16-Word by 1-Bit memory. ⇒

4×1 RAM ➡ 16×1 RAM

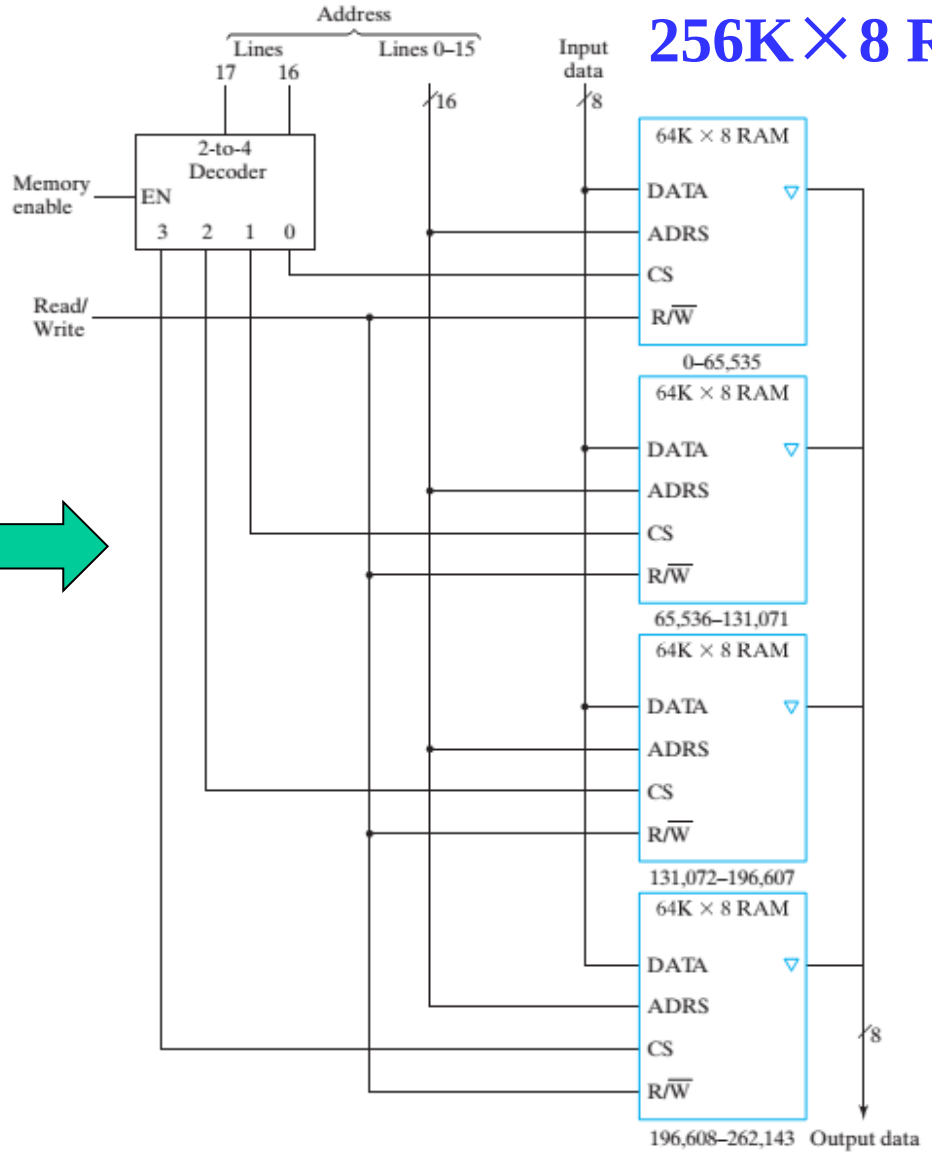


Making Larger Memories: Word extension (continued)

64K×8 RAM

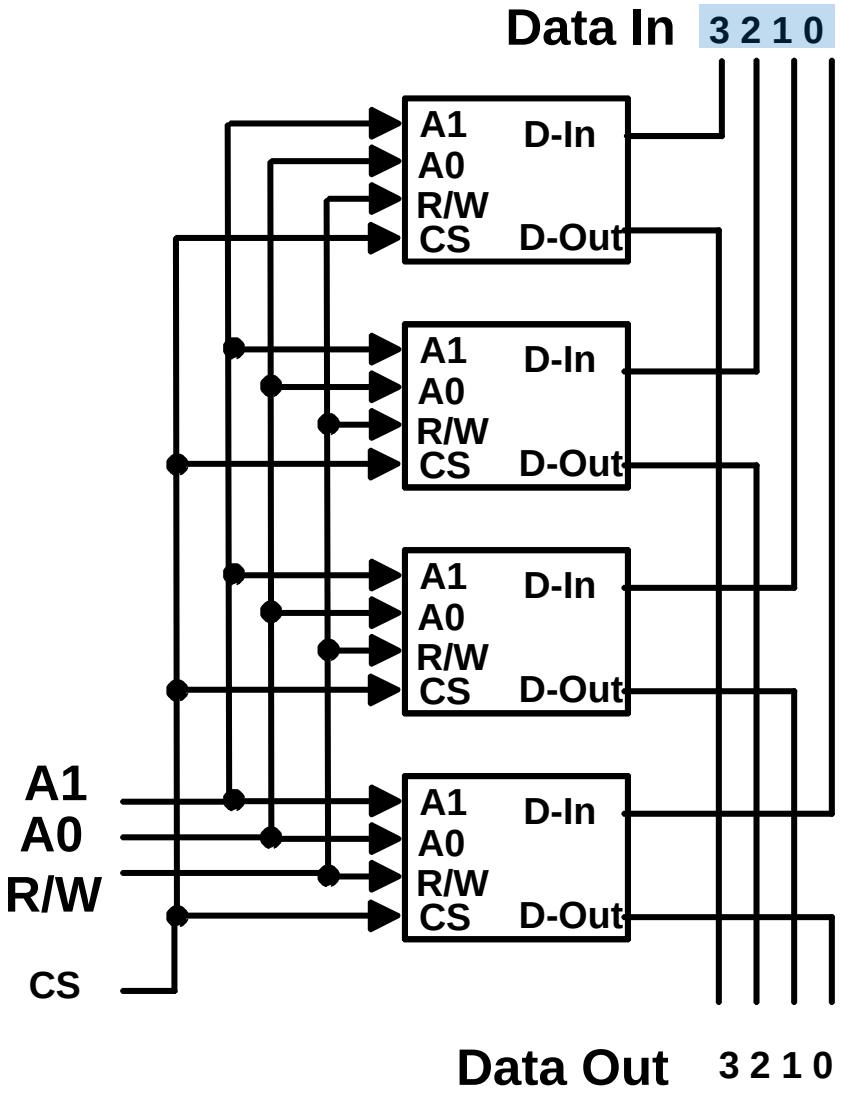


256K×8 RAM



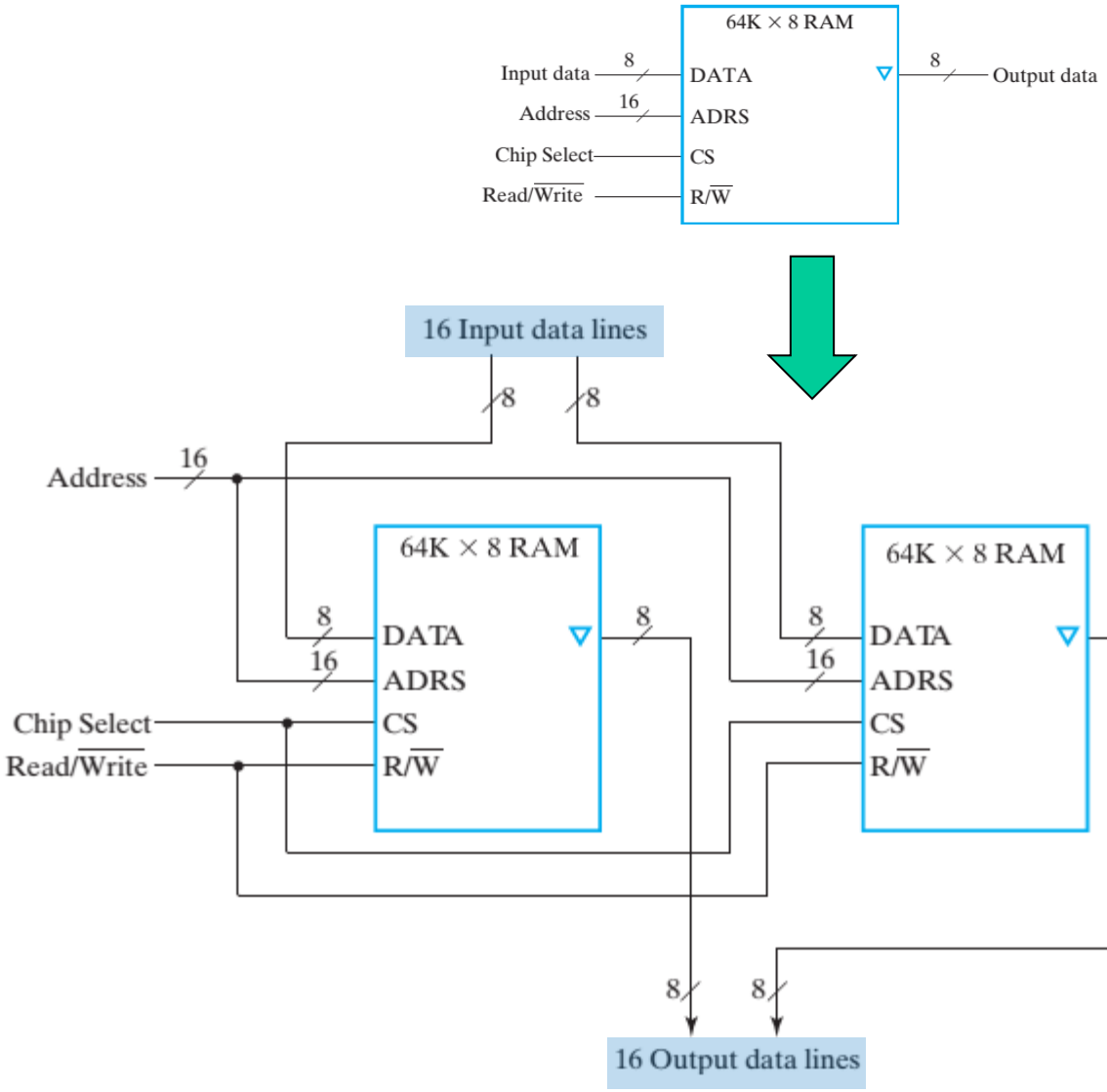
Making Wider Memories: Bit extension

- To construct wider memories from narrow ones, we tie the address and control lines in parallel and keep the data lines separate.
- For example, to make a 4-word by 4-bit memory from 4, 4-word by 1-bit memories ⇒
- Note: Both 16×1 and 4×4 memories take 4-chips and hold 16 bits of data.



4×1 RAM ⇨ 4×4 RAM

Making Wider Memories: Bit extension (continued)



64K×8 RAM

64K×16 RAM

Problem: Coincident Decoding without a Square Configuration

- **Problem 1: A 64K * 16 RAM chip uses coincident decoding by splitting the internal decoder into row select and column select. What is the size of each decoder, and how many AND gates are required for decoding an address?**

- **Answer:**

  64 K = 2^{16} = $2^8 \times 2^8$ = 256 × 256

  Row decoder size = Column decoder size = 2^8

  Row decoder = Column decoder size = 8 to 256, AND gates = 2^8 = 256 (assumes 1 level of gates with 8 inputs)

  Total AND gates required = 256 + 256 = 512

Problem: Coincident Decoding with a Square Configuration

- **Problem 2: A 64K * 16 RAM chip uses coincident decoding by splitting the internal decoder into row select and column select. Assuming that the RAM cell array is square, what is the size of each decoder, and how many AND gates are required for decoding an address?**

- **Answer:**

  Number of bits in array = $2^{16} \times 2^4 = 2^{10} \times 2^{10}$

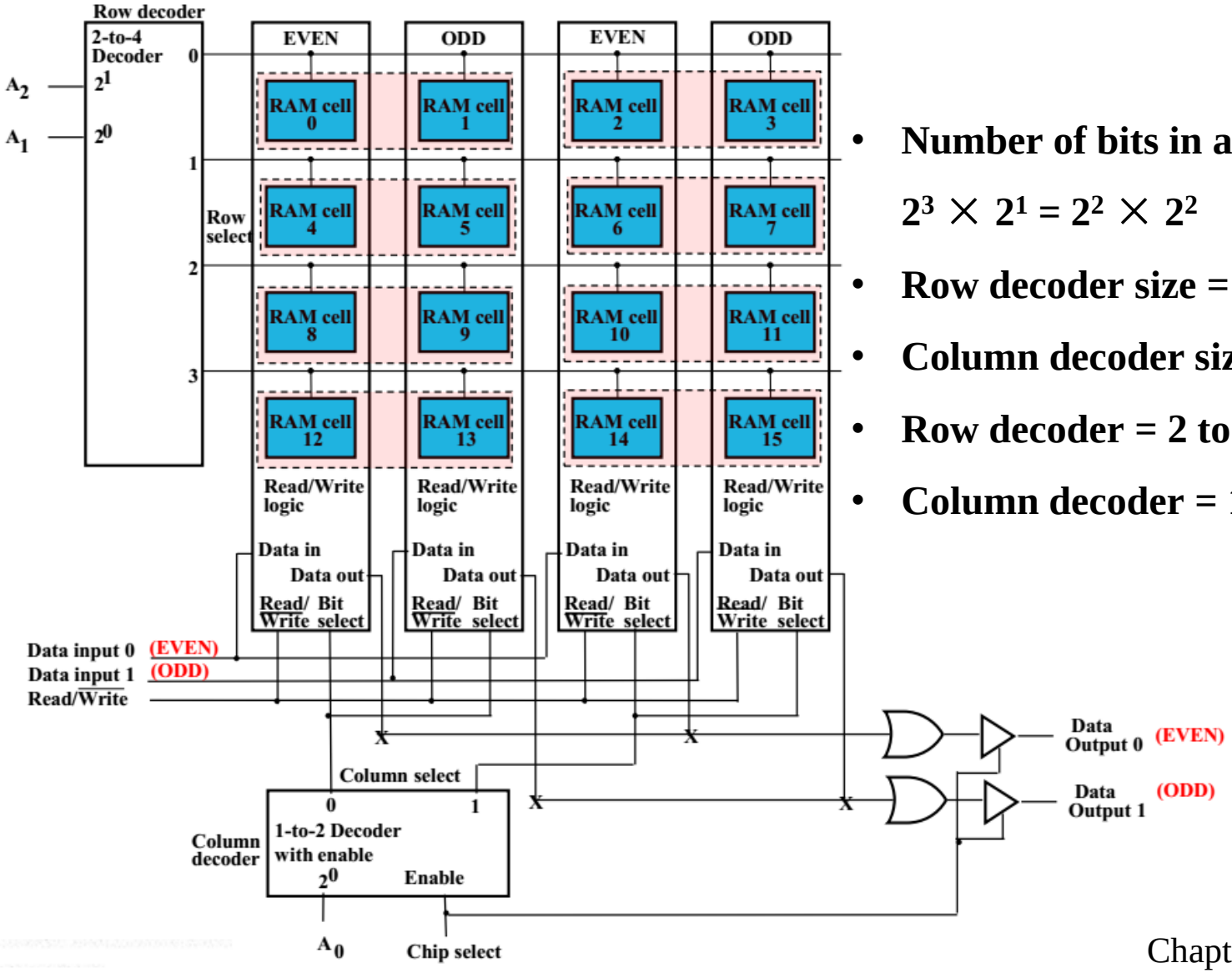
  Row decoder size = 2^{10}, Column decoder size = 2^6

  Row decoder = 10 to 1024, AND gates = $2^{10} = 1024$ (assumes 1 level of gates with 10 inputs)

  Column decoder = 6 to 64, AND gates = $2^6 = 64$ (assumes 1 level of gates with 6 inputs)

  Total AND gates required = 1024 + 64 =1088

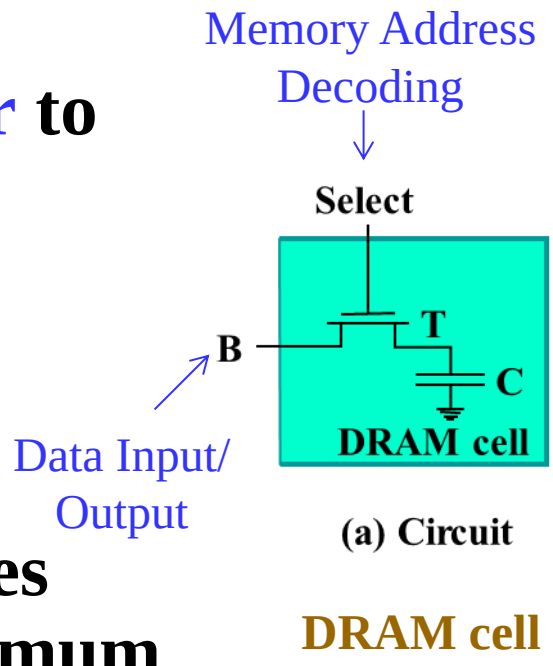
Coincident Selection with a Square Configuration (e.g., 8×2 RAM)



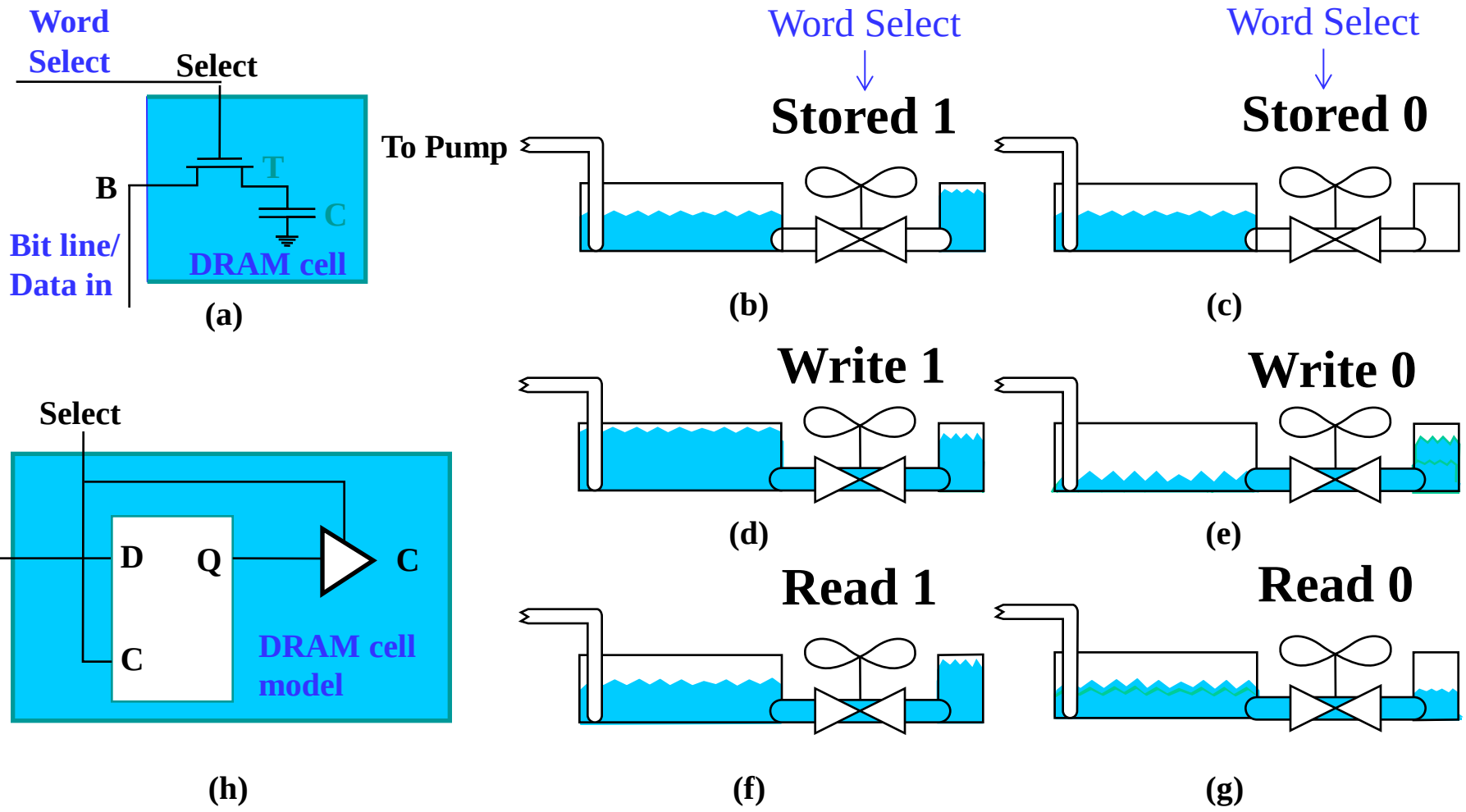
- Number of bits in array:
  $2^3 \times 2^1 = 2^2 \times 2^2$
- Row decoder size = 2^2
- Column decoder size = 2^1
- Row decoder = 2 to 4
- Column decoder = 1 to 2

Dynamic RAM (DRAM)

- **Basic Principle: Storage of information on capacitors.**
- **Charge and discharge of capacitor to change stored value**
- **Use of transistor as "switch" to**
  - **Store charge**
  - **Charge or discharge**
- **Each row of a DRAM chip requires refreshing within a specified maximum refresh time**



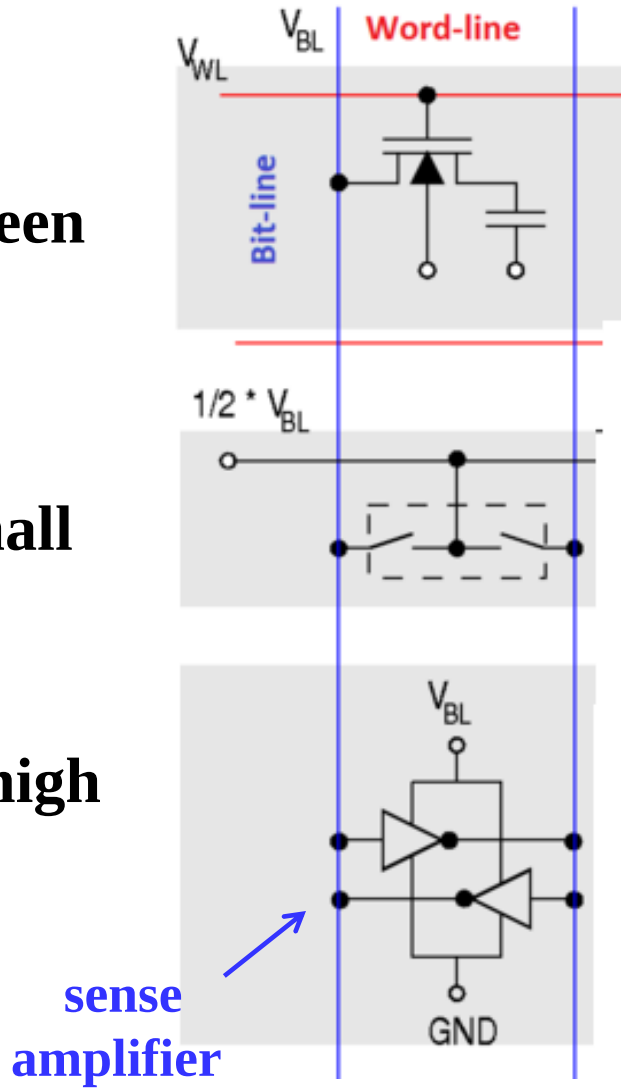
Dynamic RAM (continued)



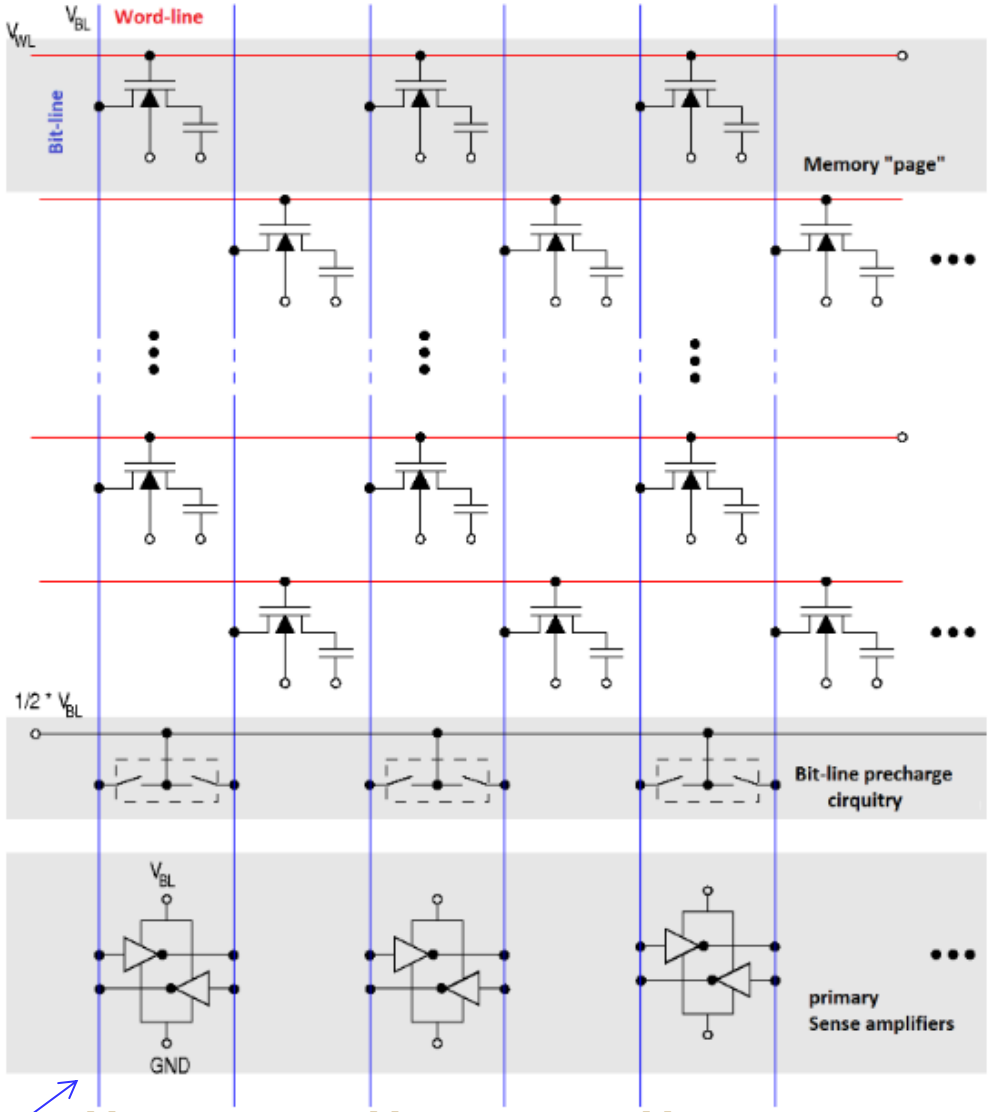
hydraulic analogy of cell operation

Dynamic RAM - Basic Operation

- **Read from a DRAM:**
  - **bit-lines are precharged to the metastable point (voltages between high and low logic levels)**
  - **word-line goes high to open the row**
  - **sense amplifier amplifies the small voltage with positive feedback**
- **Write to a DRAM**
  - **sense amplifier is forced to the high or low voltage state**
  - **open the row to charge or discharge the cell**

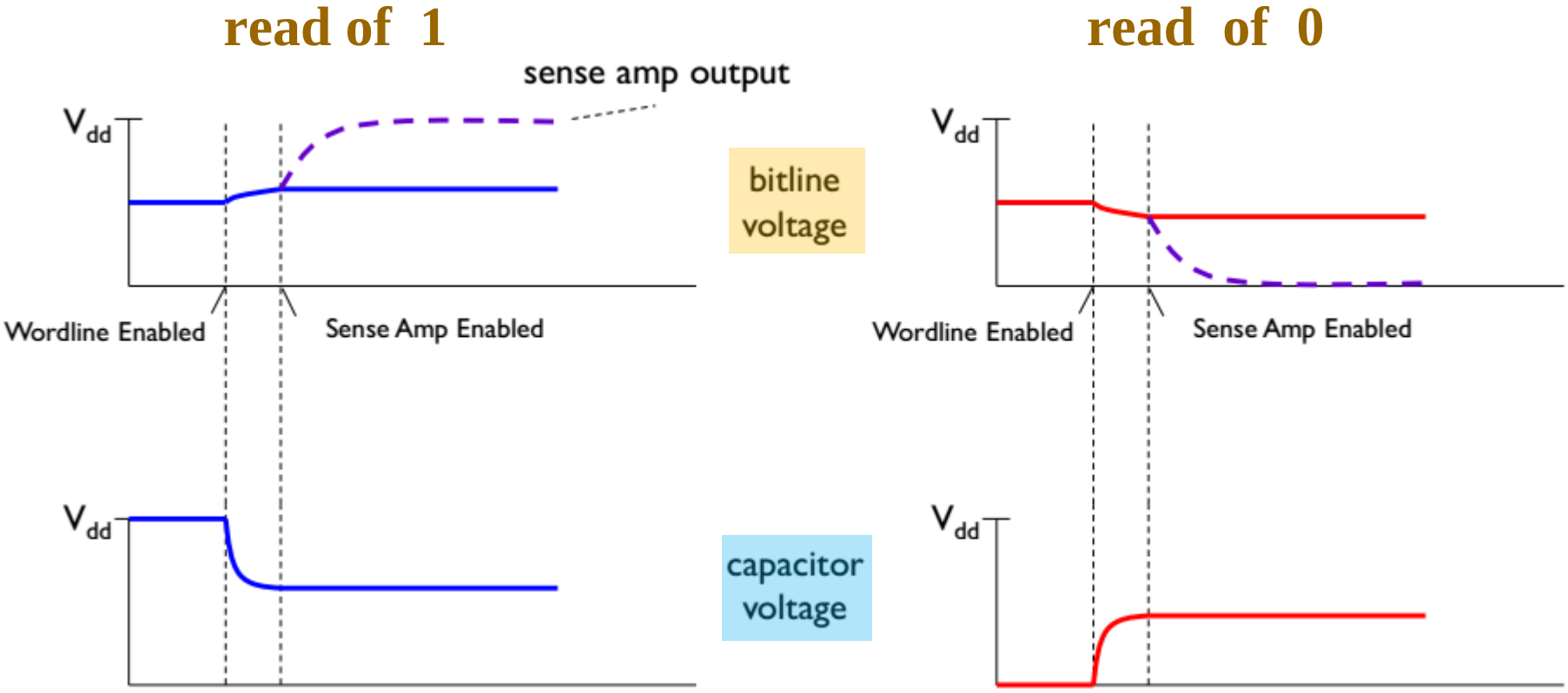


Dynamic RAM - DRAM Cell Array



Dynamic RAM - Destructive Read

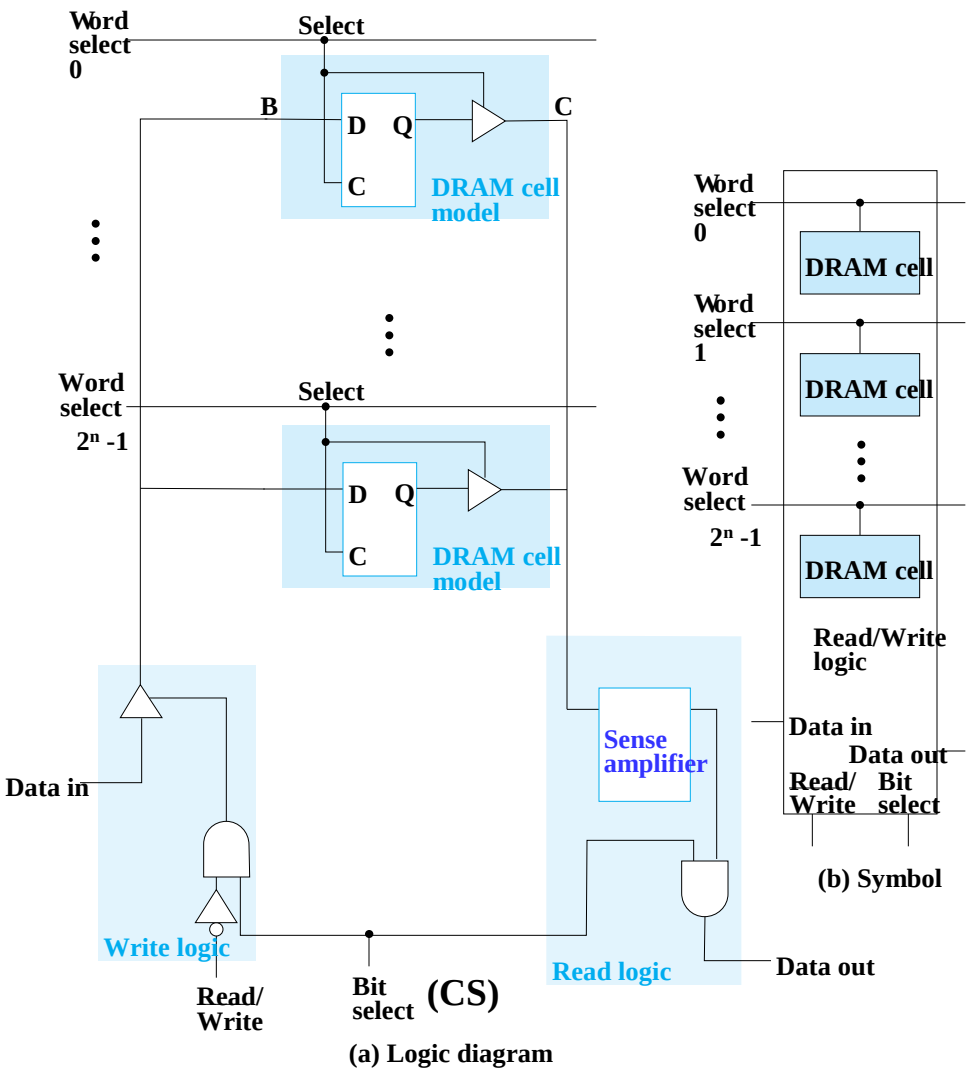
- **Reads are destructive: contents are erased by reading.**



- **After read of 0 or 1, cell contents close to ½.**

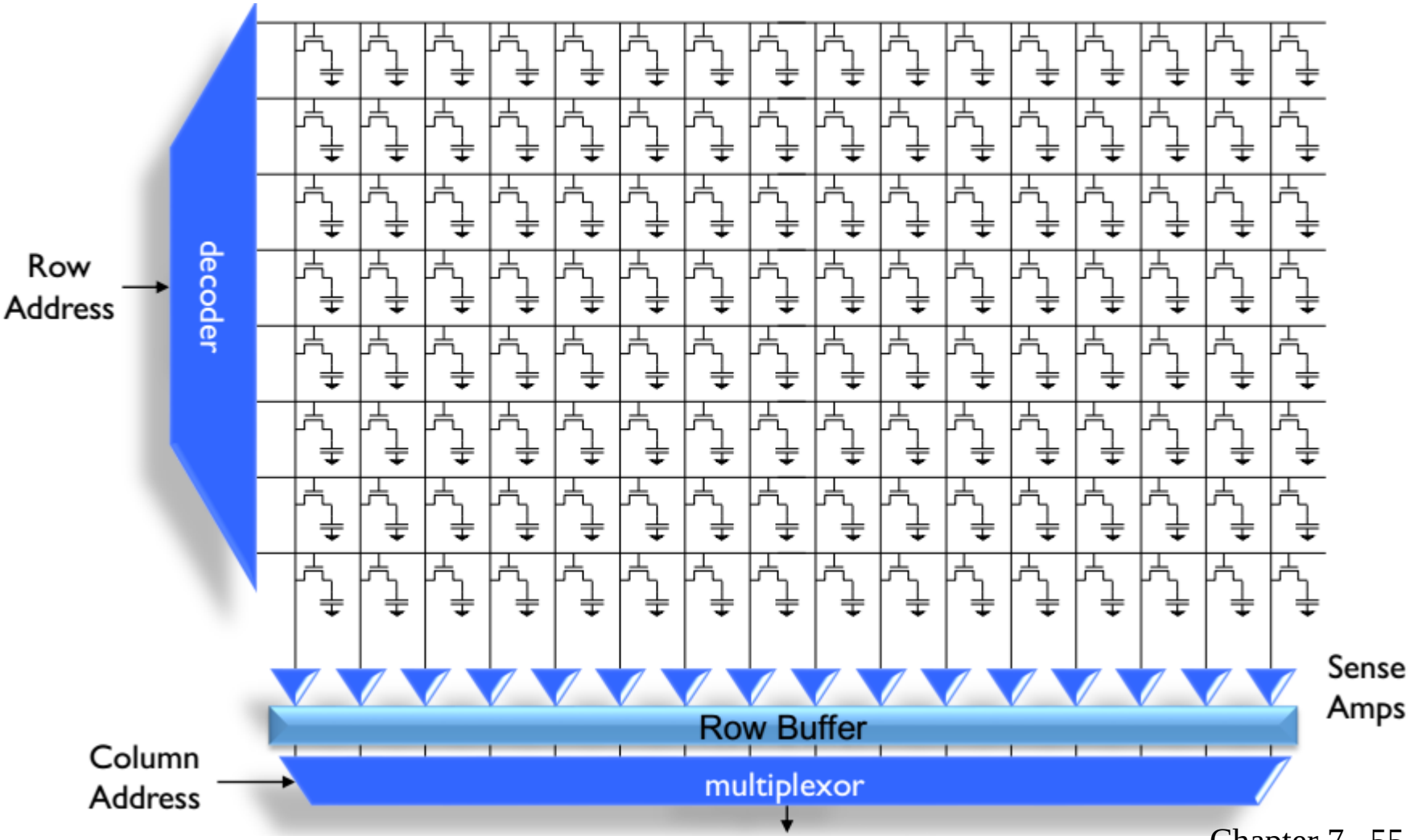
Dynamic RAM - Bit Slice

- C is driven by 3-state drivers
- Sense amplifier is used to change the small voltage change on C into H or L
- In the electronics, B, C, and the sense amplifier output are connected to make destructive read into non-destructive read



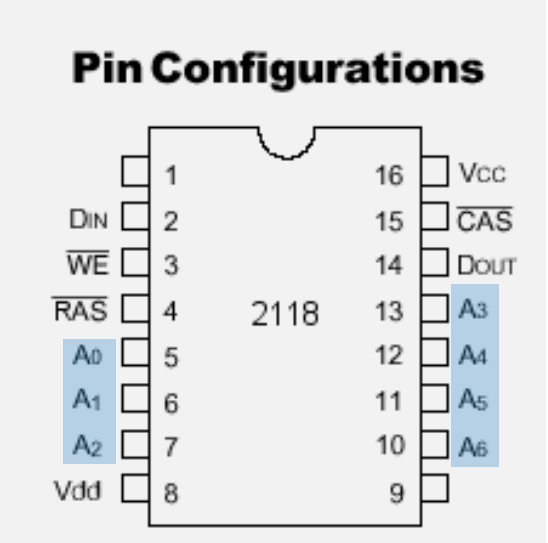
Dynamic RAM - Address Multiplexing (1/5)

- **DRAM is much denser than SRAM.**



Dynamic RAM - Address Multiplexing (2/5)

- In order to reduce the number of pins on the chip and thus reduce the size of the DRAM chip, most DRAM chips multiplex the row and column addresses onto the same set of pins.
- One of the earliest, simplest, and most important DRAM chips was the 2118, introduced by Intel in 1979.
- The 2118 was a 16K × 1 DRAM with only half of the address lines (14/2=7).



Dynamic RAM - Address Multiplexing (3/5)

- **This multiplexing of the address pins means that the number of address pins for a DRAM can be cut in half, which is a fundamental feature of DRAMs.**
- **The address is applied in two steps:**
  - **the row address first**
  - **the column address second**
- **Two strobes, RAS (row address strobe) and CAS (column address strobe) are used to tell the chip which part of the address is currently on the address pins.**

Dynamic RAM - Address Multiplexing (4/5)

- **DRAM organizes its memory into a 2D array of cells:**
  - **Externally, address multiplexing (RAS# and CAS#) is used to supply the row and column addresses separately using the same lines.**
  - **Internally, coincident decoding is used to activate a specific row and column line.**

Technique	Purpose	Used in DRAM?	Benefit
Address Multiplexing	Reduces external pin count	Yes	Fewer address lines needed
Coincident Decoding	Reduces internal decoding complexity	Yes	Fewer transistors and wires needed

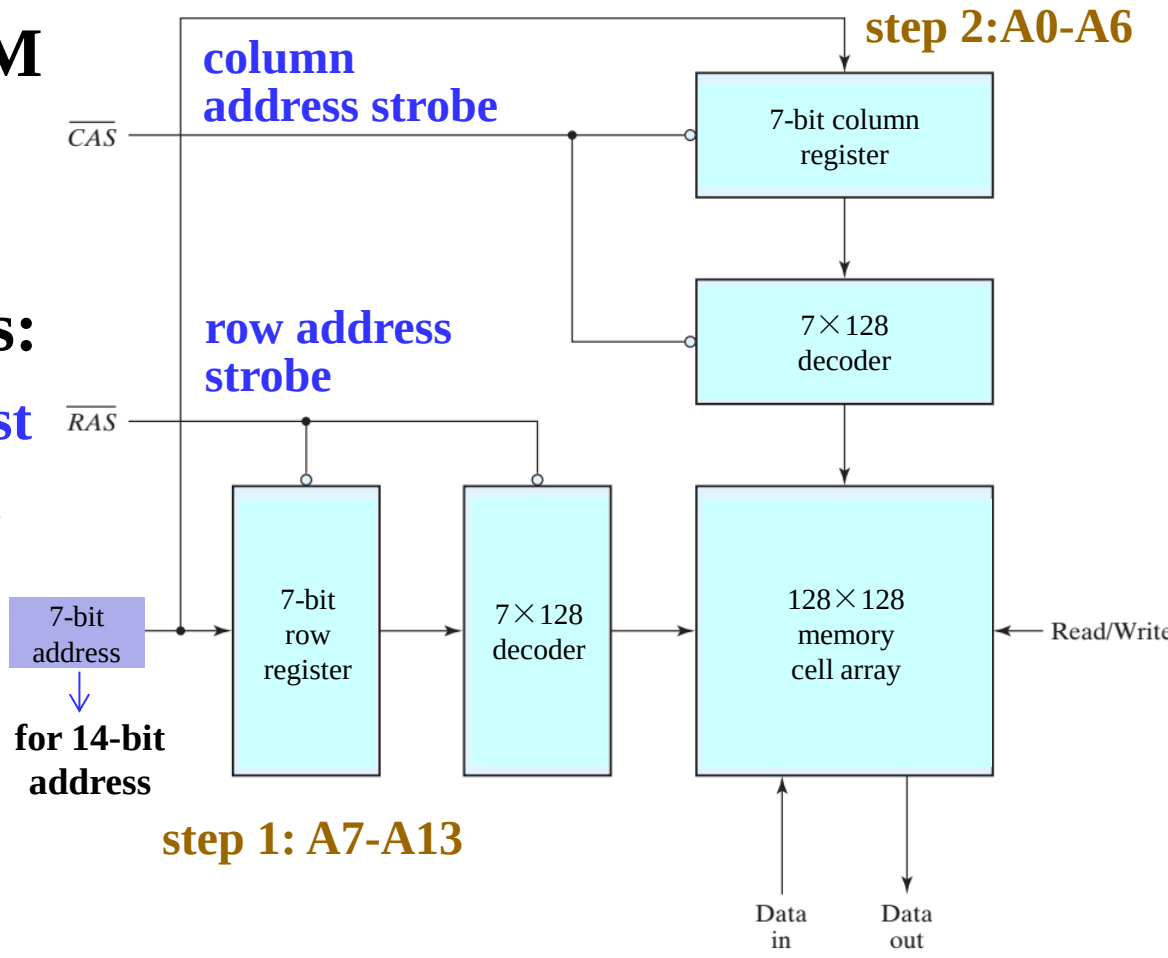
Why SRAM Typically Does *Not* Use Address Multiplexing

- **SRAM is usually smaller and used for faster-access memory like caches.**
  - **Since it's smaller, it's feasible to have all address bits provided in parallel — so no need to multiplex.**
  - **SRAM's key focus is speed, and address multiplexing adds latency due to sequential row/column address latching, which is unacceptable for high-speed cache access.**

Feature	DRAM	SRAM
Address Multiplexing	Yes: RAS/CAS mechanism	No
Reason for Use	Reduce pin count	Speed is more critical
Typical Use Case	Main memory (large capacity)	CPU caches (small but fast)
Access Time	Slower	Very fast

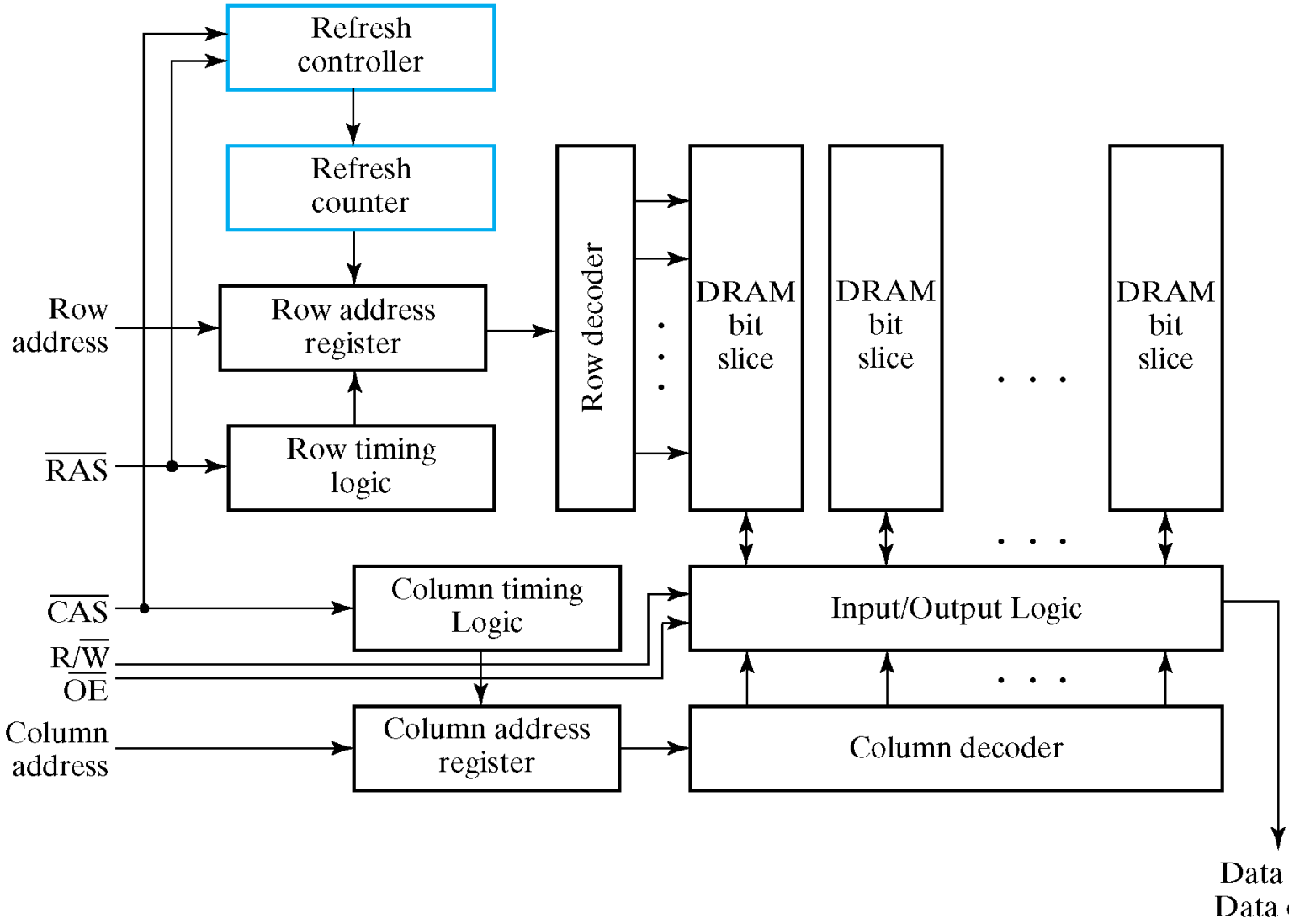
Dynamic RAM - Address Multiplexing (5/5)

- E.g., 16K×1 DRAM
  - 7 address lines
- The address is applied in two steps:
  - the row address first
  - the column address second
- Two strobes:
  - RAS (row address strobe)
  - CAS (column address strobe)



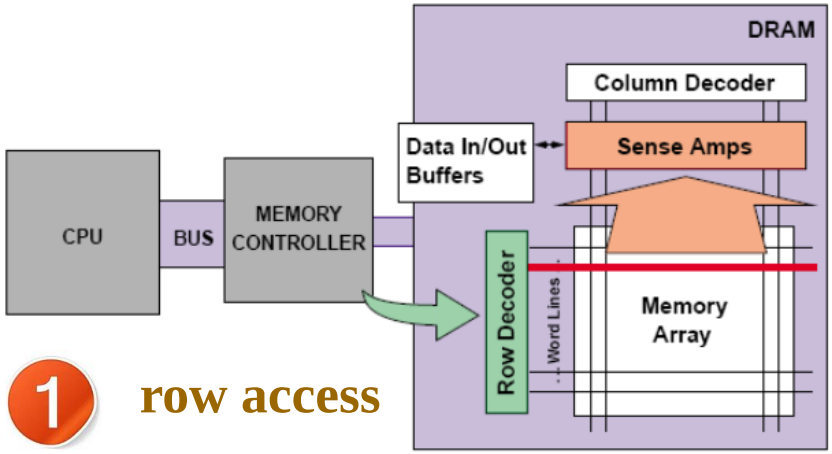
64K×1 DRAM

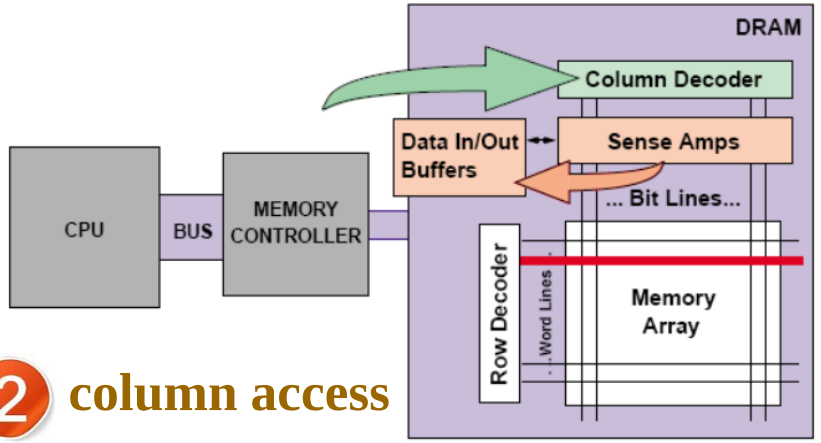
Dynamic RAM - Block Diagram

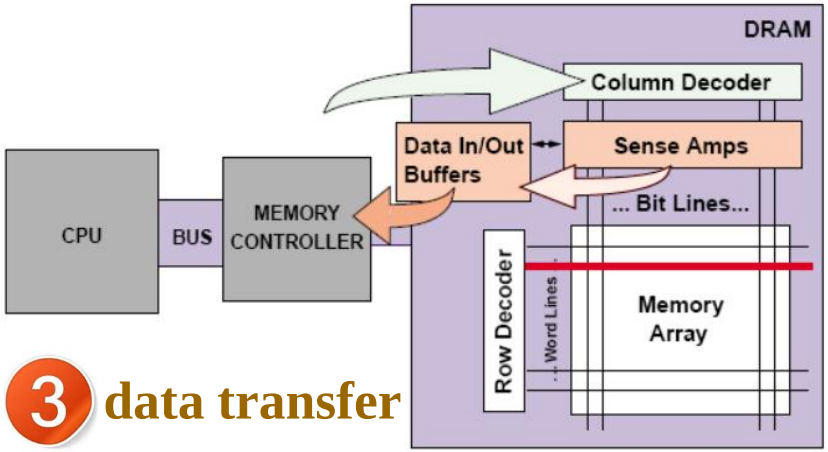


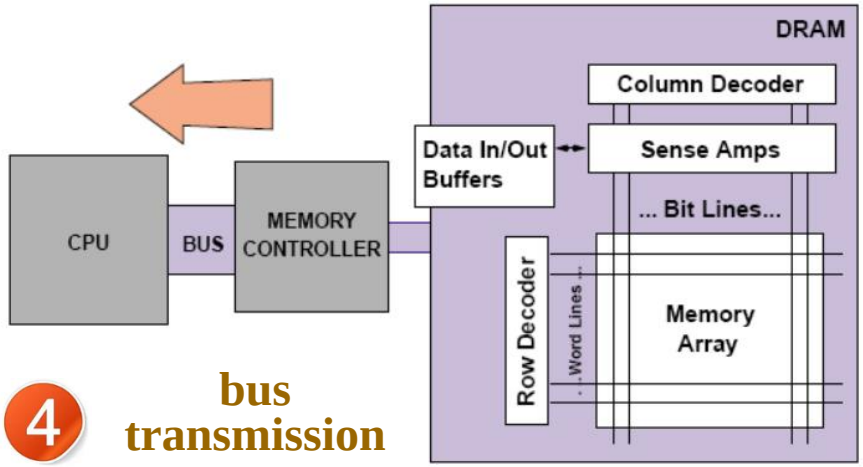
Dynamic RAM - DRAM Access Sequence

- **Why is the row address applied first?**

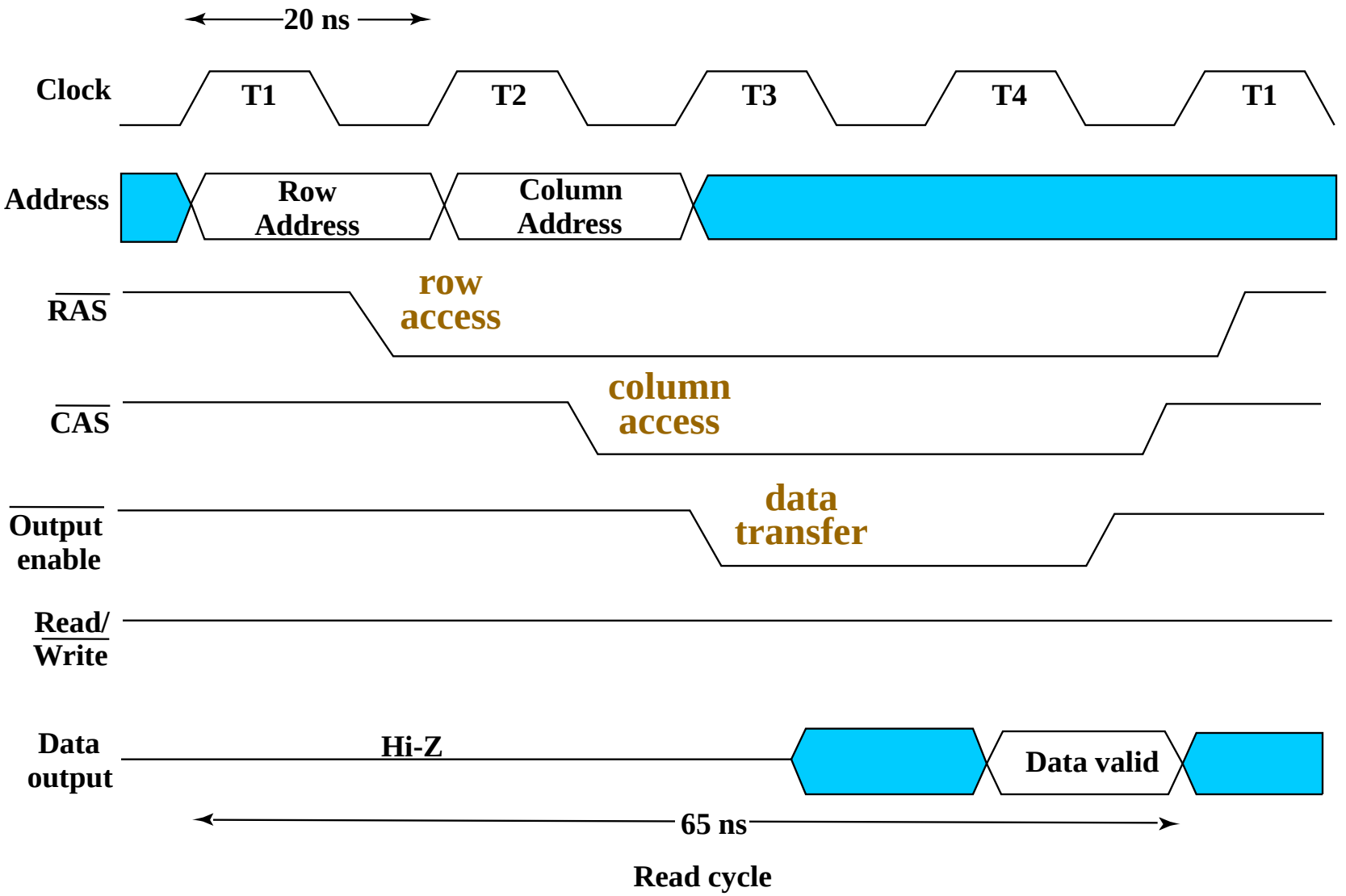








Dynamic RAM Read Timing



Problem: Address Multiplexing

- **Problem : A 4 Gb DRAM uses 4-bit data and has equal-length row and column addresses. How many address pins does the DRAM have?**

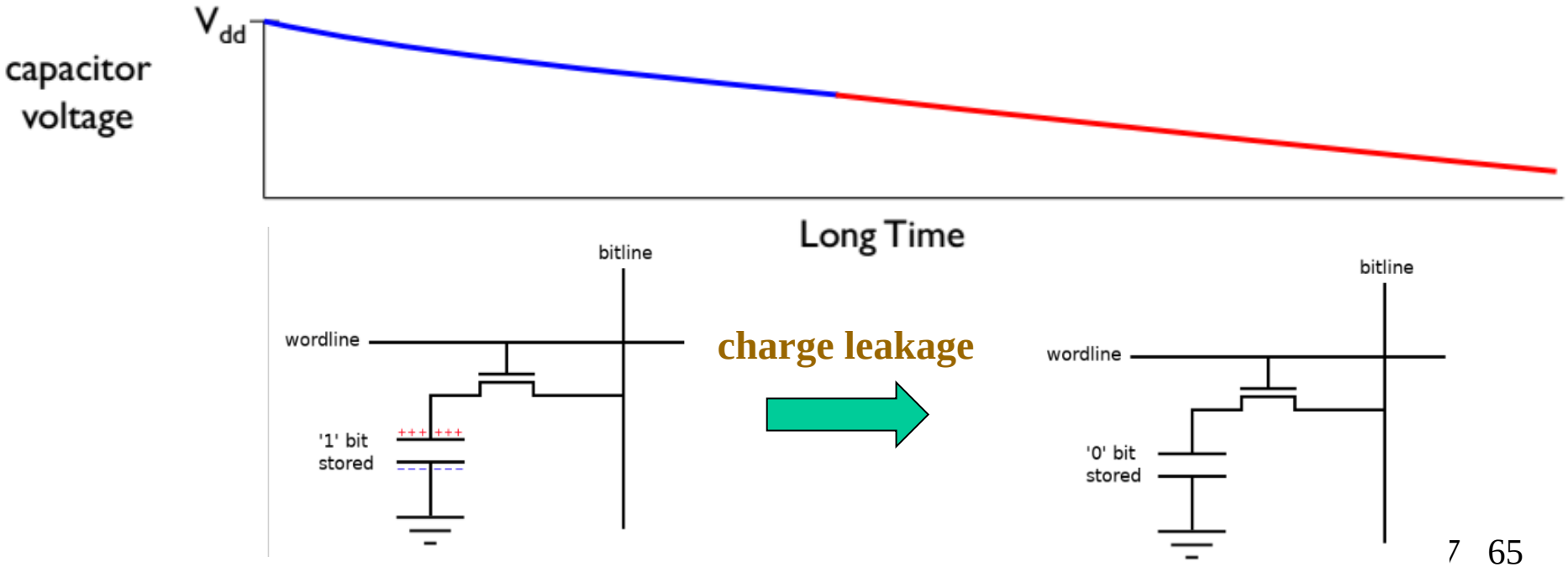
- **Answer:**

  Memory Size: 4 Gb = 2^{32} = $2^{30} \times 2^{2}$ (1 word = 4 bits)
  DRAM contains 2^{30} words (address lines = 30)

  Since the DRAM uses address multiplexing, the number of address pins is 30/2=15.

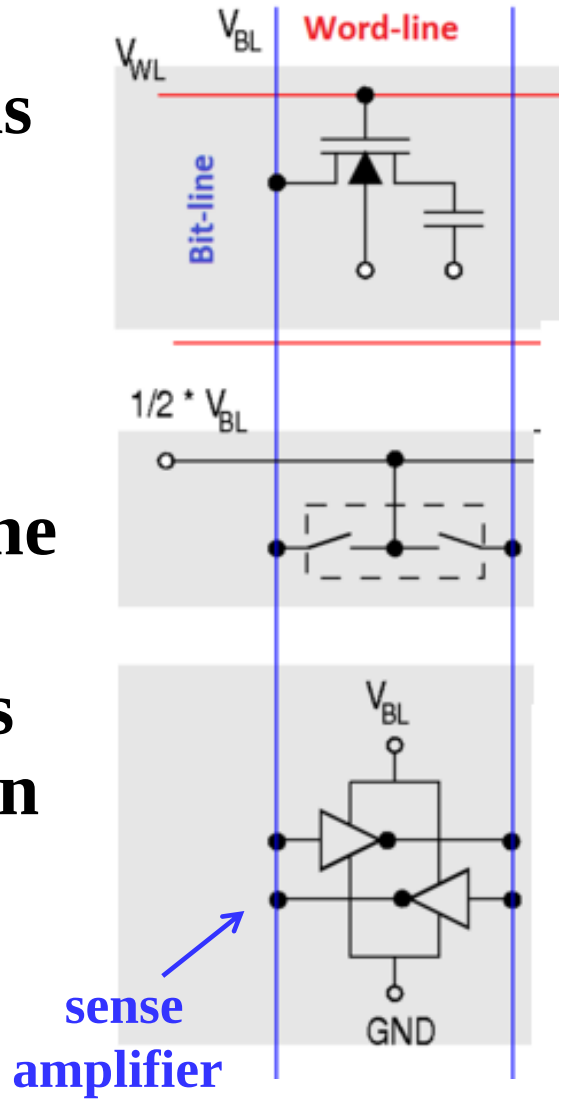
Dynamic RAM – DRAM Refresh

- **Gradually, DRAM cell loses contents**
  - **Even if it is not accessed**
  - **This is why DRAM is called "dynamic"**
- **DRAM must be regularly read and re-written**
  - **JEDEC standard: DRAM refresh rate ≤ 64 ms**



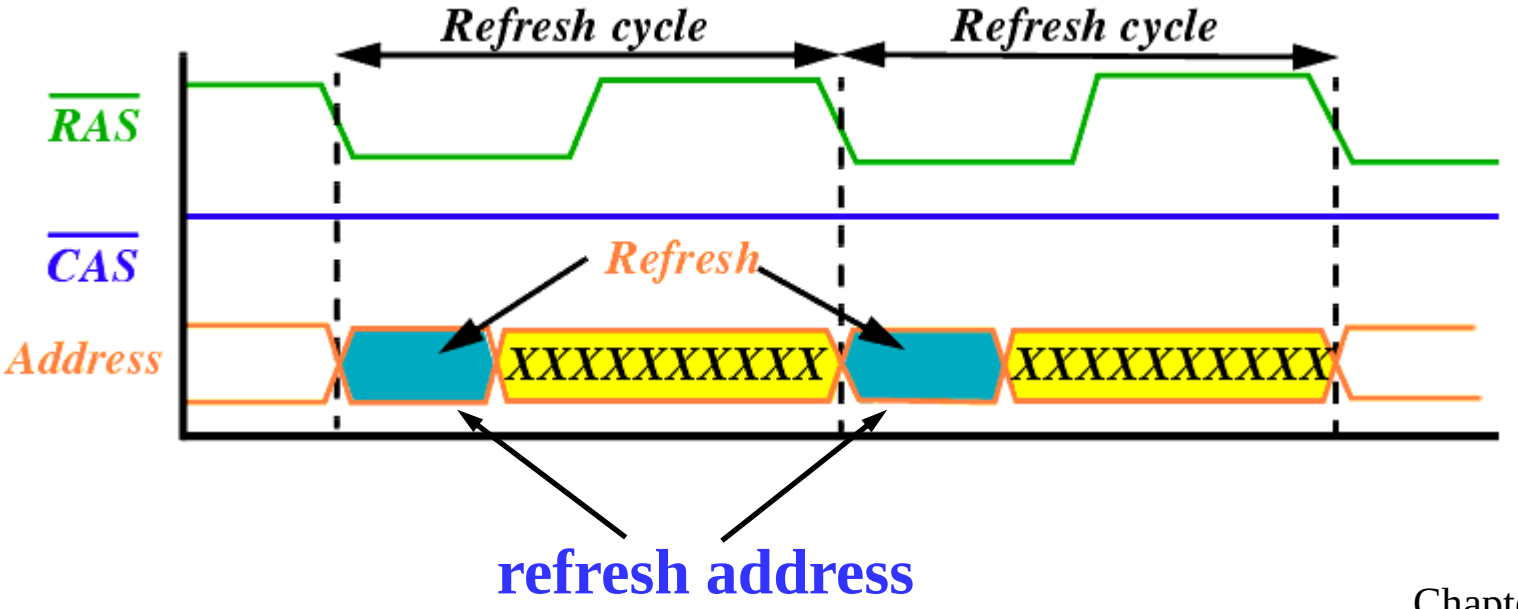
Dynamic RAM – DRAM Refresh

- While reading of columns in an open row is occurring, current is flowing back up the bit-lines from the output of the sense amplifiers and recharging the storage cells.
- This reinforces (i.e. refreshes) the charge in the storage cell.
- Due to the length of the bit-lines there is a fairly long propagation delay for the charge to be transferred back to the cell's capacitor.



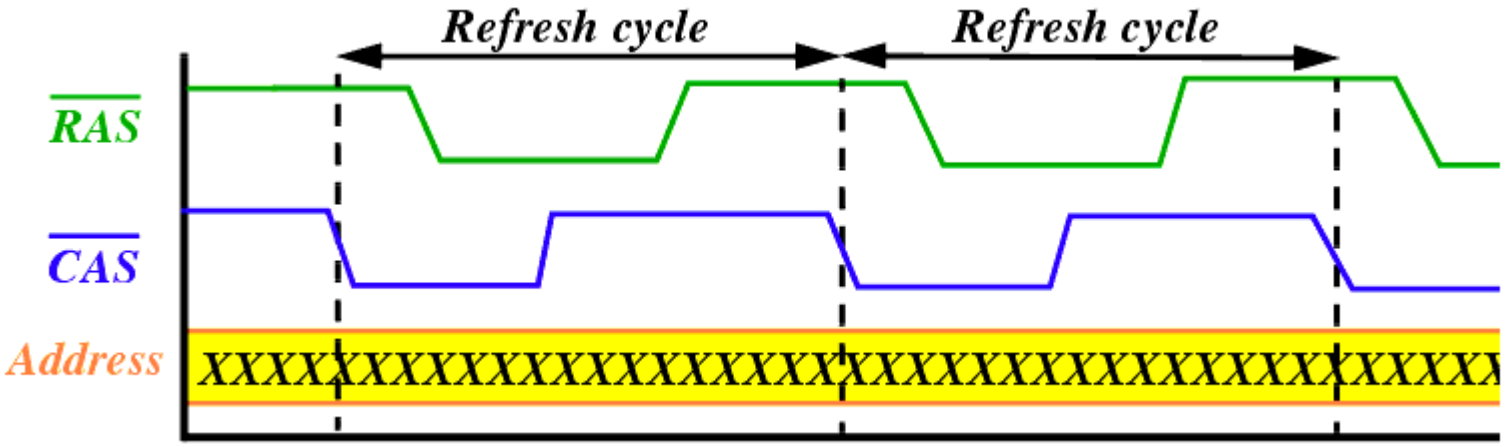
Dynamic RAM – Refresh Policies

- **RAS-only refresh (row access only without column access)**
  - **RAS is activated and a row address (refresh address) is applied to the DRAM, CAS inactive.**
  - **DRAM internally reads one row and amplifies the read data. Not transferred to the output pins as CAS is disabled.**
  - **The main disadvantage of this refresh method is that an external logic device is required to generate the row addresses in succession.**



Dynamic RAM – Refresh Policies

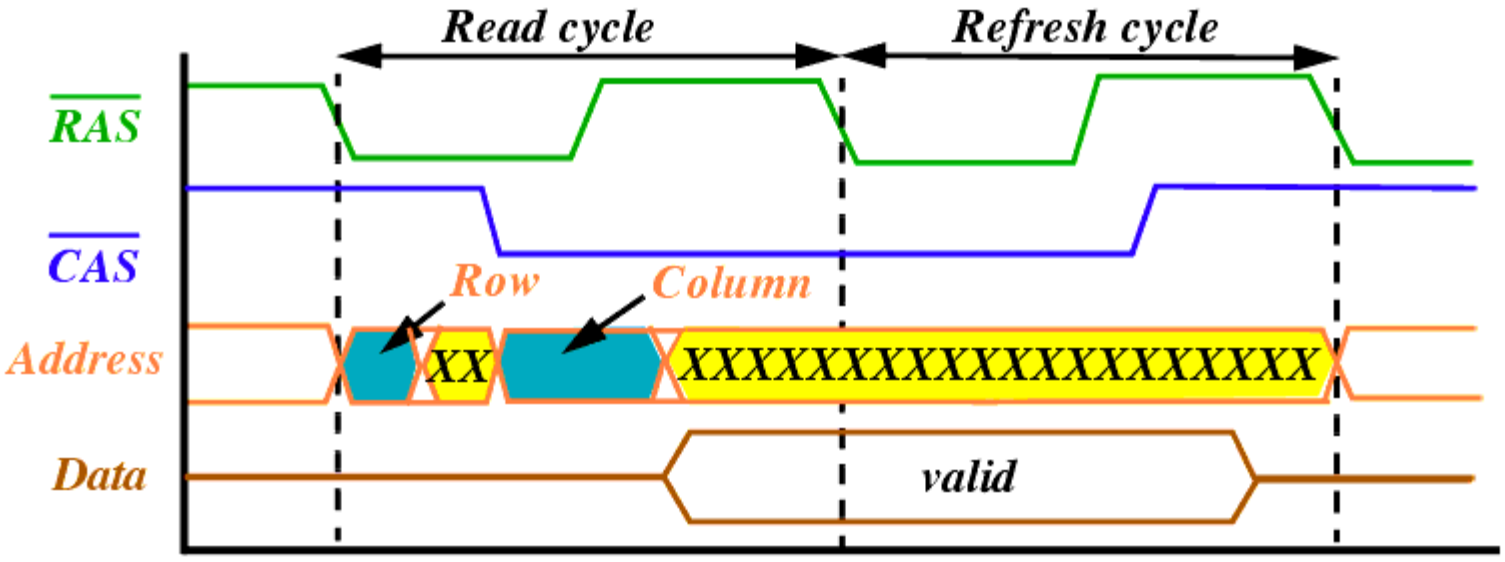
- **CAS before RAS refresh**
  - **DRAM has its own refresh logic with an address counter. When the sequence is applied on CAS and RAS, the internal refresh logic generates an address and refreshes the associated cells.**
  - **After every cycle, the internal address counter is incremented.**
  - **The memory controller just needs to issue the signals.**



No address is involved !

Dynamic RAM – Refresh Policies

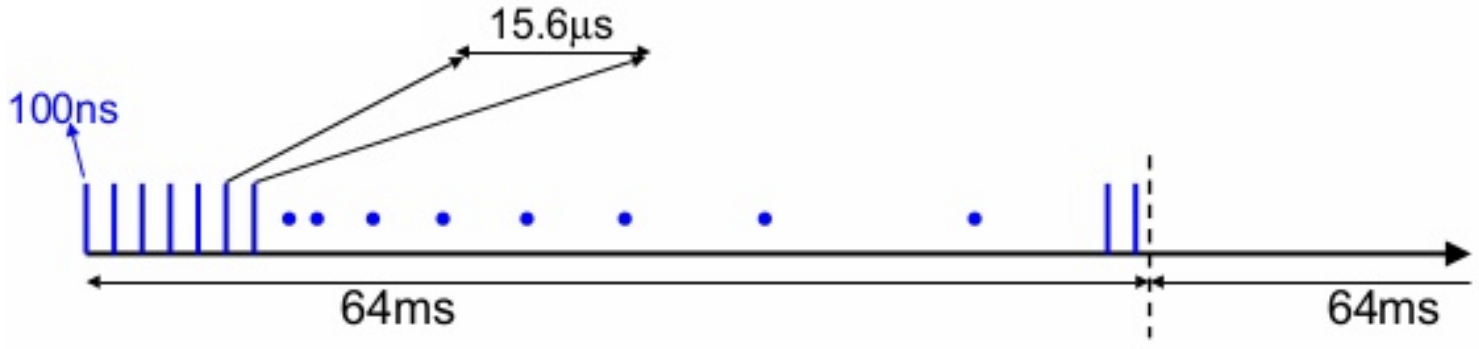
- **Hidden refresh**
  - **Following a normal read or write, CAS is left at 0 and RAS is cycled, effectively performing a CAS-before-RAS refresh. During a hidden refresh, the output data from the prior read remains valid. Thus, the refresh is hidden.**



Dynamic RAM – Refresh (continued)

- **Refresh mode**
  - **Burst mode: stop the work and refresh all memory (e.g., per 16~64ms)**

  - **Distributed mode: space out refresh one row at a time, thus avoid blocking memory for a long time (e.g., 15.6µs)**



DRAM Types

- **DRAM Types**
  - **Asynchronous DRAM**
  - **Synchronous DRAM (e.g., SDRAM, DDR, RDRAM)**

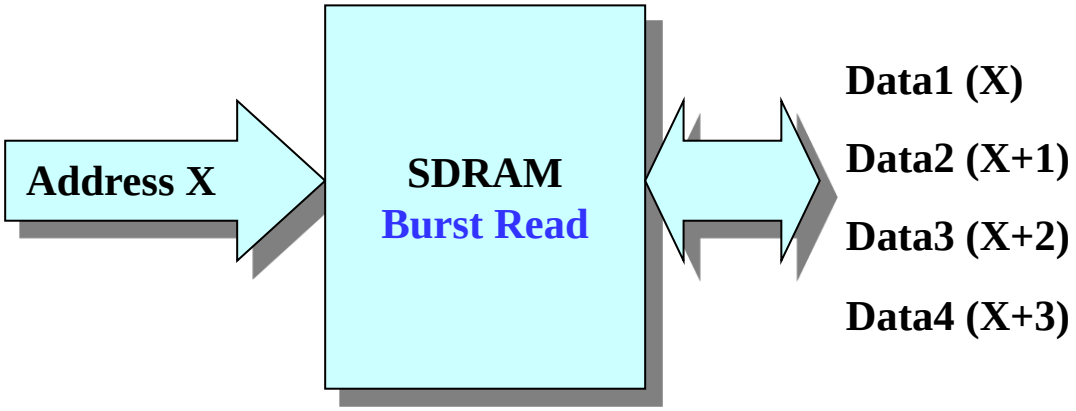
	Asynchronous DRAM	Synchronous DRAM
Clock dependency	Operates independently of the system clock	Synchronized to system clock (e.g., CPU clock)
Data Transfer	Typically single access at a time	Can do burst read/write
Speed	Lower	Faster (better pipelining, burst)
Control complexity	Higher (timing-sensitive)	Lower (thanks to clocked control)
Usage	Legacy or embedded systems	Modern systems (SDRAM, DDR)

Synchronous DRAM

- **All signals are referenced to an external clock**
  - **Makes timing more precise with other system (CPU)**
  - **Synchronous registers appear on address input、data input and data output**
- **Burst read/write in SDRAM refers to a high-speed data transfer mode where multiple data words are read or written in a sequence, starting from a specific memory address, with only one address and command issued at the beginning.**

Synchronous DRAM (continued)

- **Burst Read**
  - **Controller issues an ACTIVATE command to open a row.**
  - **READ command is issued with a starting column address and burst length (typically 2, 4, 8, or more words).**
  - **SDRAM returns a sequence of words, one per clock cycle (after CAS latency).**

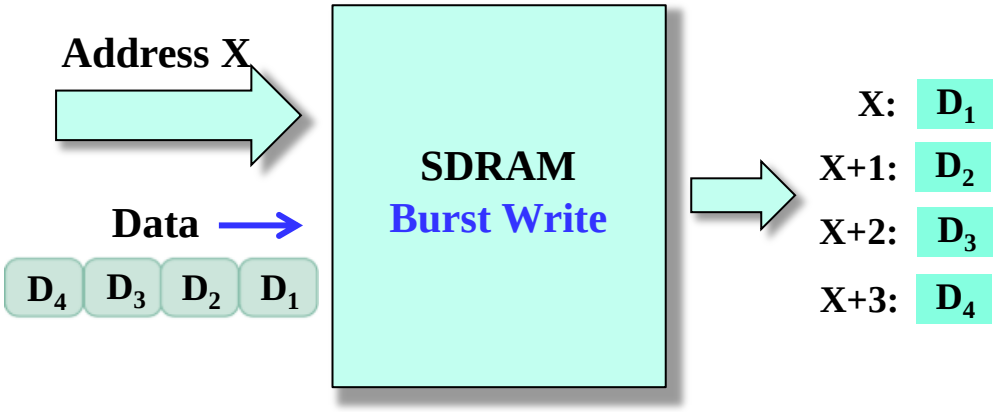


Burst Read with Burst length = 4

Synchronous DRAM (continued)

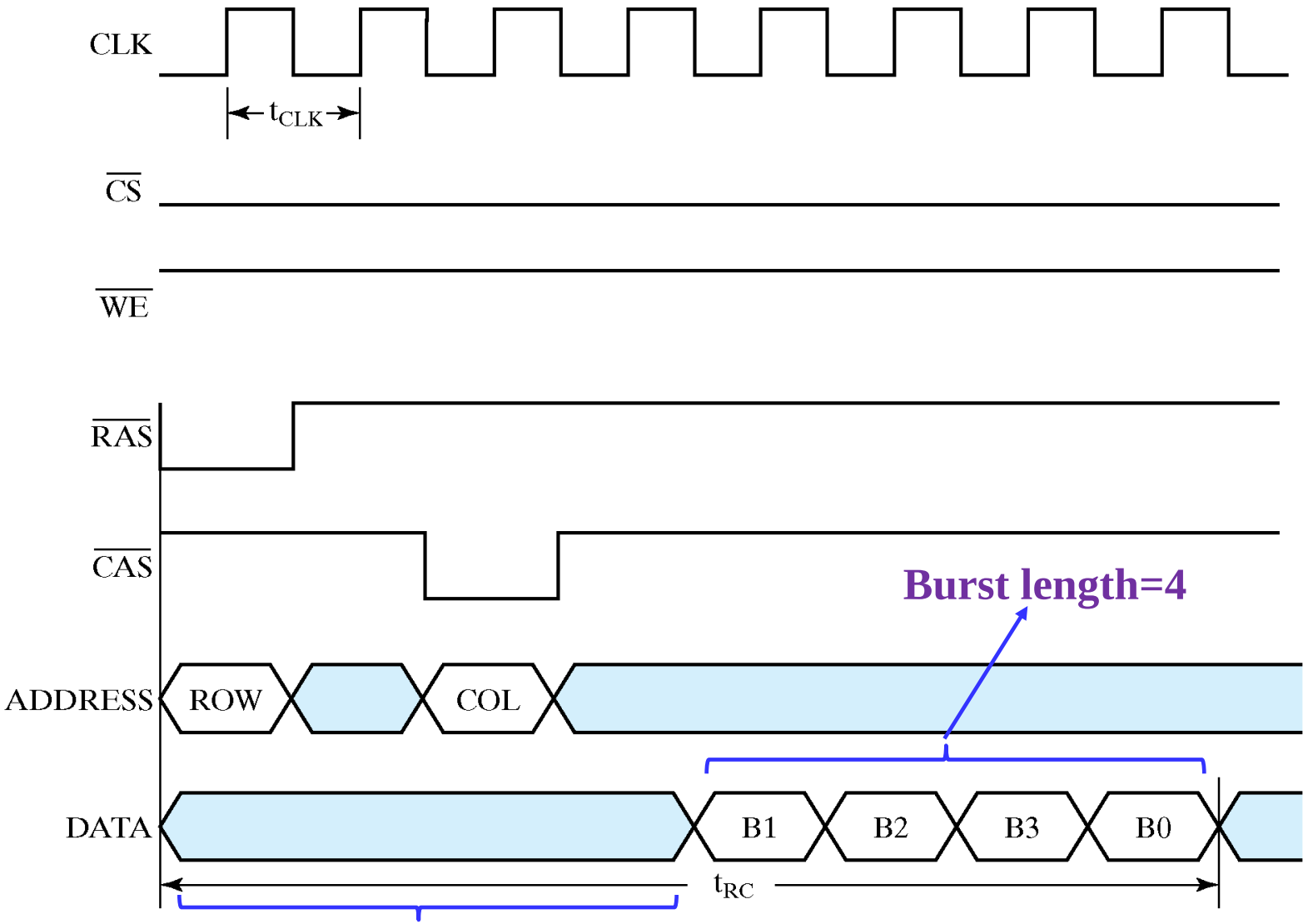
- **Burst Write**
  - **ACTIVATE the row.**
  - **WRITE command is issued with a starting address and burst length.**
  - **Controller provides data in successive cycles for each word in the burst.**

note: For write bursts, the controller must provide data for each cycle of the burst on the data bus.



Burst Write with Burst length = 4

SDRAM burst time——burst read



Synchronous DRAM (Continued)

- **Memory bandwidth is the speed of the memory. It is the maximum rate at which data can be read from or stored into a memory by the processor.**
- **Memory bandwidth is measured in megabytes per second (MB/s) or gigabytes per second (GB/s).**
- **The memory bandwidth of SDRAM is related with burst size.**

Synchronous DRAM (Continued)

- **For example**
  - **Memory data path width: 1 byte**
  - **Memory clock period: 7.5 ns**
  - **Latency time (from application of row address until first word available): 4 clock cycles**

- **If burst size: 8 bytes**
  - **Read cycle time: (4 + 8) * 7.5 ns = 90 ns**
  - **Memory Bandwidth: $8/(90 * 10^{-9}) \approx 88.89$ Mbytes/sec**

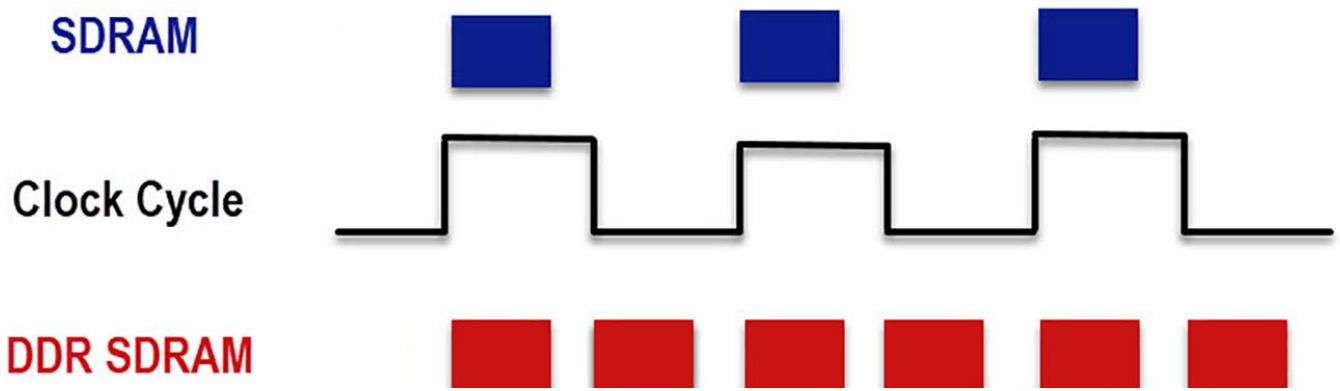
- **If burst size: 2048 bytes**
  - **Read cycle time: (4 + 2048) * 7.5 ns = 15390 ns**
  - **Memory Bandwidth: $2048/(15390 * 10^{-9}) \approx 133.07$ Mbytes/sec**

Synchronous DRAM (Continued)

- **Example**
  - **Memory data path width: 1 word = 4 bytes**
  - **Burst size: 8 words = 32 bytes**
  - **Memory clock frequency: 5 ns**
  - **Latency time: 4 clock cycles**
  - **Read cycle time: (4 + 8) * 5 ns = 60 ns**
  - **Memory Bandwidth: $32/(60 * 10^{-9}) \approx 533$ Mbytes/sec**

Double Data Rate Synchronous DRAM

- Transfers data on both edges of the clock
- Provides a transfer rate of 2 data words per clock cycle

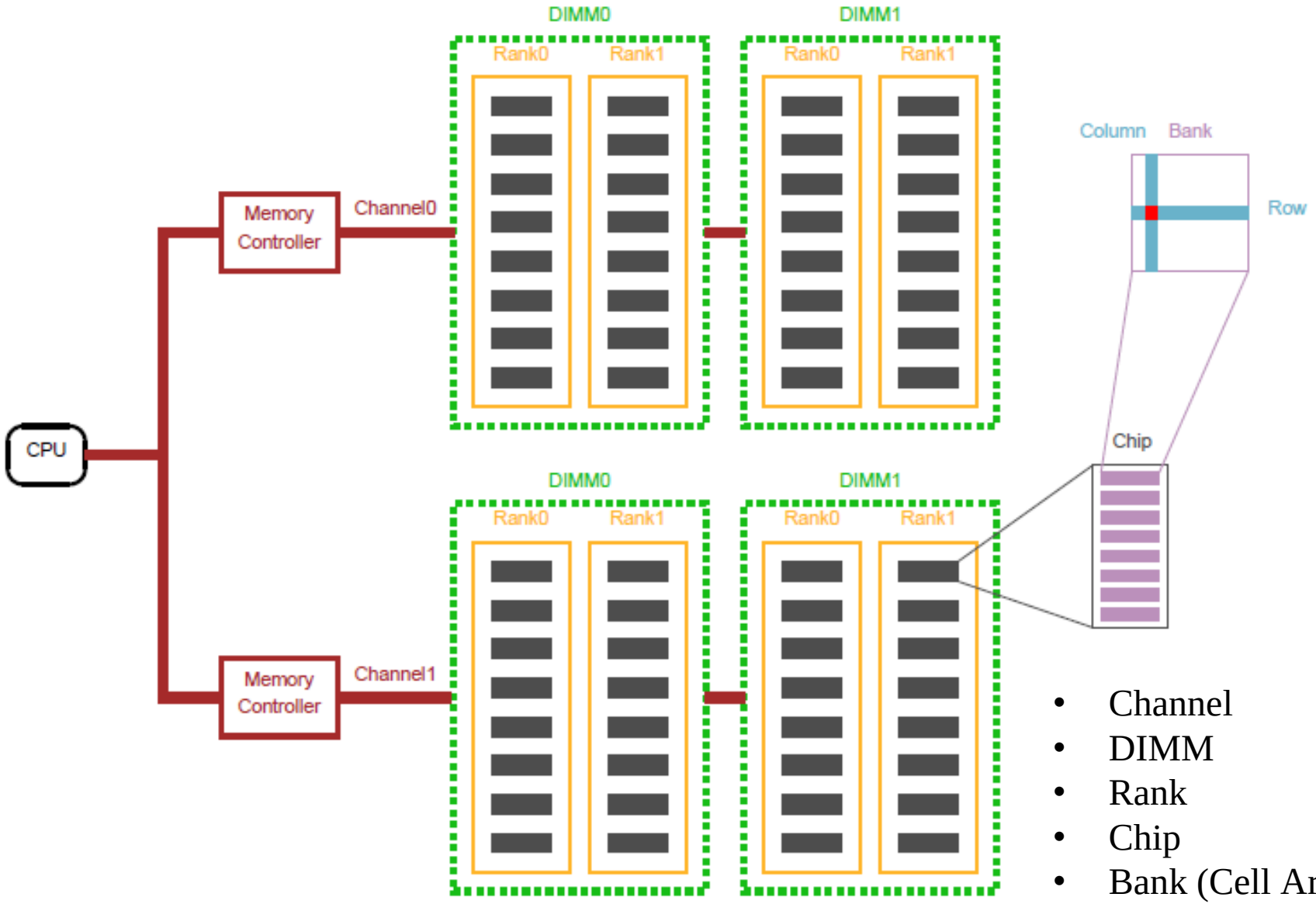


- Example: Same as for synchronous DRAM
  - Read cycle time = 60 ns
  - Memory data path width: 1 word = 4 bytes
  - Burst size: 8 — 8 * 2 words = 8*2*4 = 64 bytes
  - Memory Bandwidth: 64/(60 * 10^{-9}) ≈ 1.066 Gbytes/sec

Arrays of DRAM Integrated Circuits

- **Similar to arrays of SRAM ICs, but there are differences typically handled by an IC called a *DRAM controller*:**
  - **Separation of the address into row address and column address and timing their application**
  - **Providing $\overline{\text{RAS}}$ and $\overline{\text{CAS}}$ and timing their application**
  - **Performing refresh operations at required intervals**
  - **Providing status signals to the rest of the system (e.g., indicating whether or not the memory is active or is busy performing refresh)**

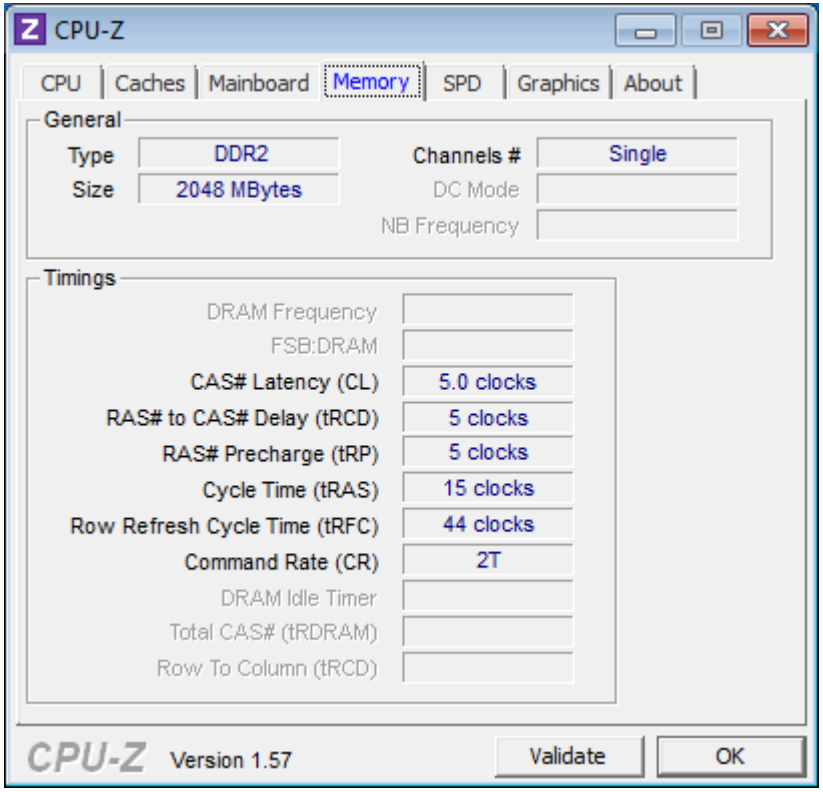
DRAM Organization



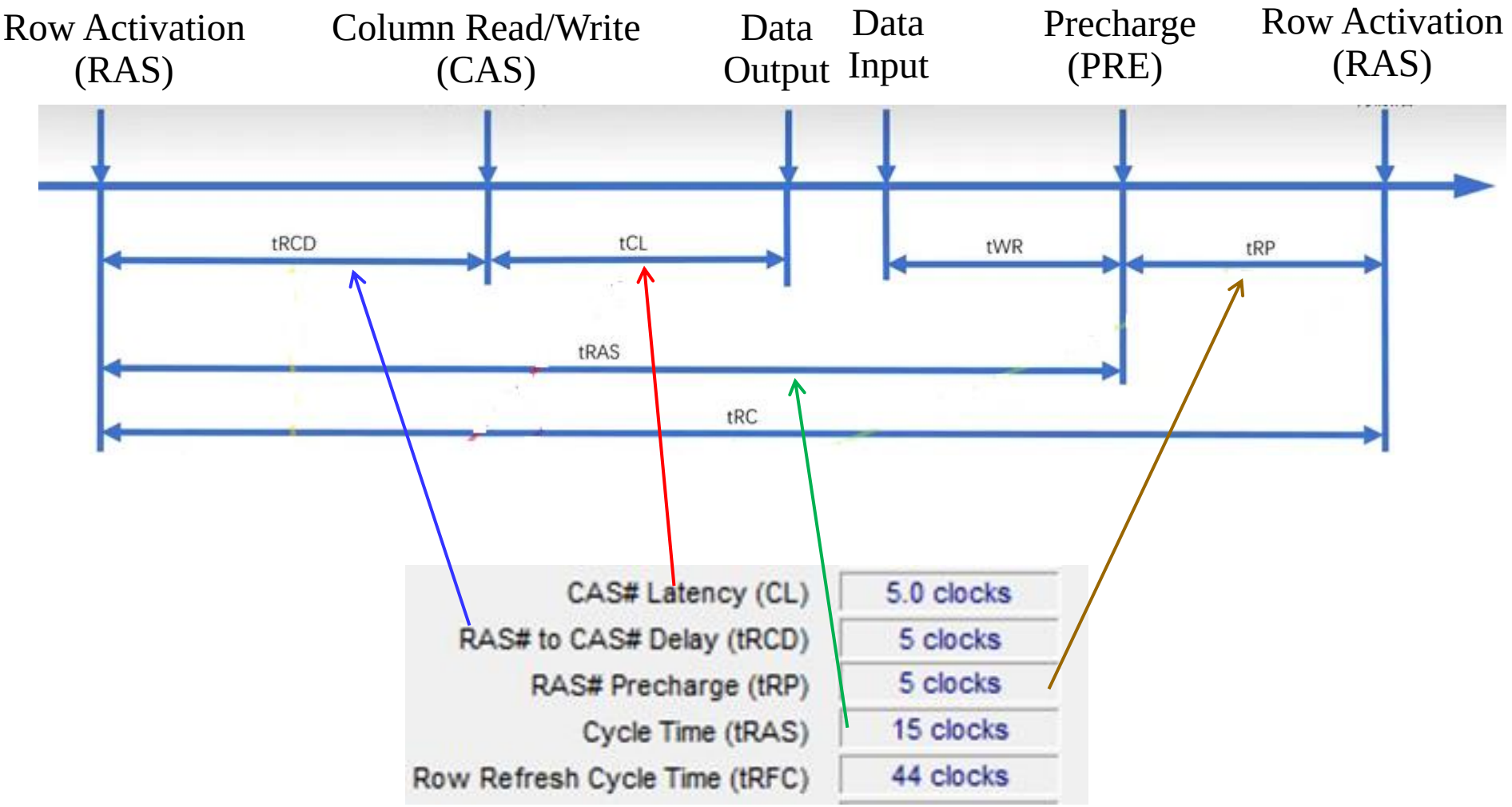
- Channel
- DIMM
- Rank
- Chip
- Bank (Cell Arrays)
- Cell (Row/Column)

DRAM Specification (1/2)

- As of now, are you familiar with the memory specifications reported by CPU-Z?



DRAM Specification (2/2)



Assignments

Reading:

- 7.1-7.7

Problem assignment:

- 7-1、7-4、7-5、7-8

Example: Candy Vending Machine

Design a candy vending machine:

- The vending machine accepts nickels (N, 5 cents) and dimes (D, 10 cents)
- When the machine has received 15 cents it delivers a package of candy.
- If too much money has been added, the machine returns the difference.
- When the candy has been released, the machine will be back to the original state.

Vending Machine Specification

- **Vending machine specification:**
  - **Input: N, D**

  1) **N = 0 (no nickel); N = 1 (a nickel inserted)**
  2) **D = 0 (no dime); D = 1 (a dime inserted)**

  note: only one of the inputs N or D are asserted at one time, so the input combinations are (0,0), (0,1) and (1,0).

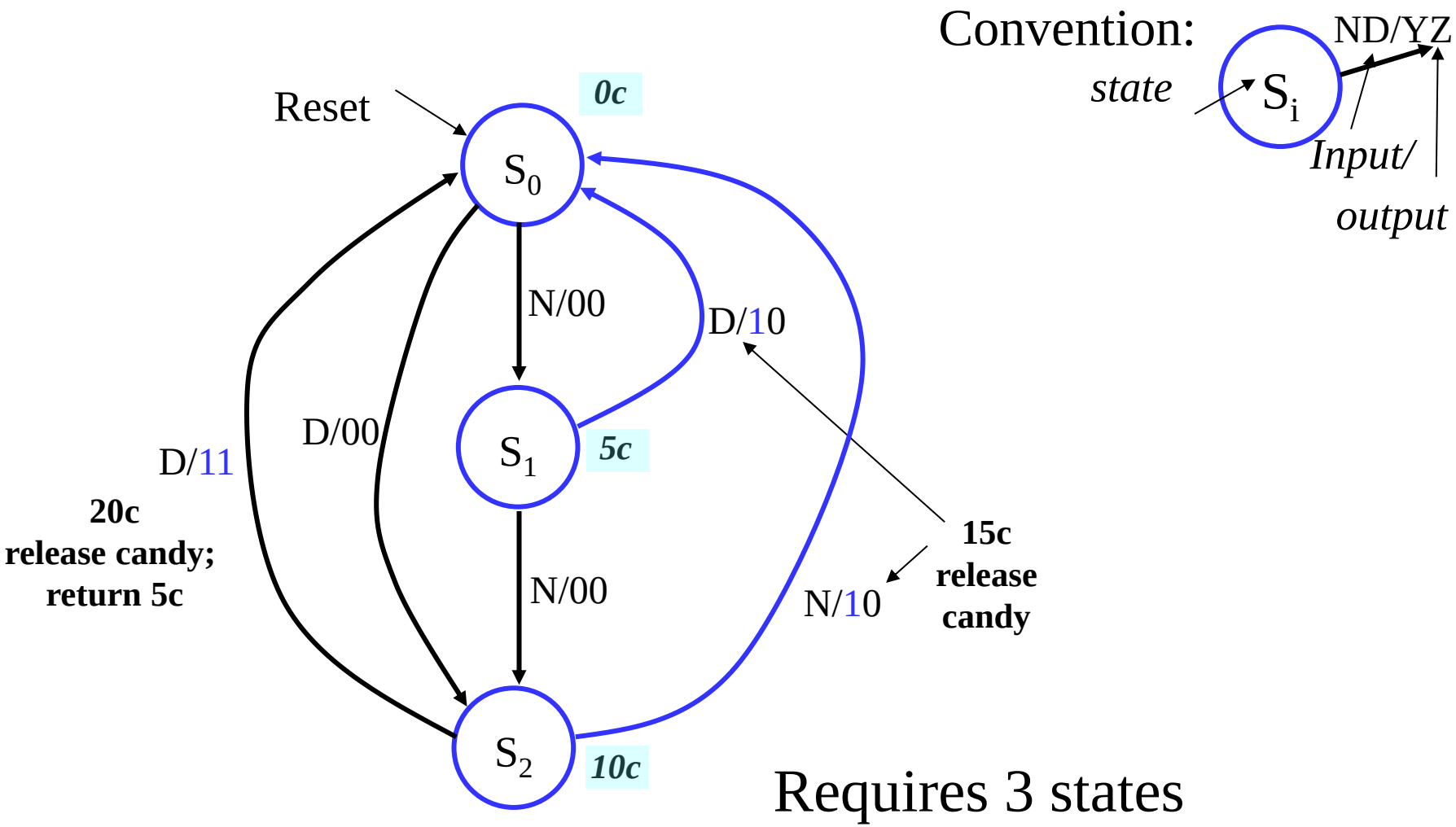
  - **State: money received**
  - **Output: Y, Z**

  1) **Y = 0 (no candy); Y = 1 (candy provided)**
  2) **Z = 0 (no change); Z = 1 (change returned: 5 cents)**

Vending Machine Specification—Mealy Model

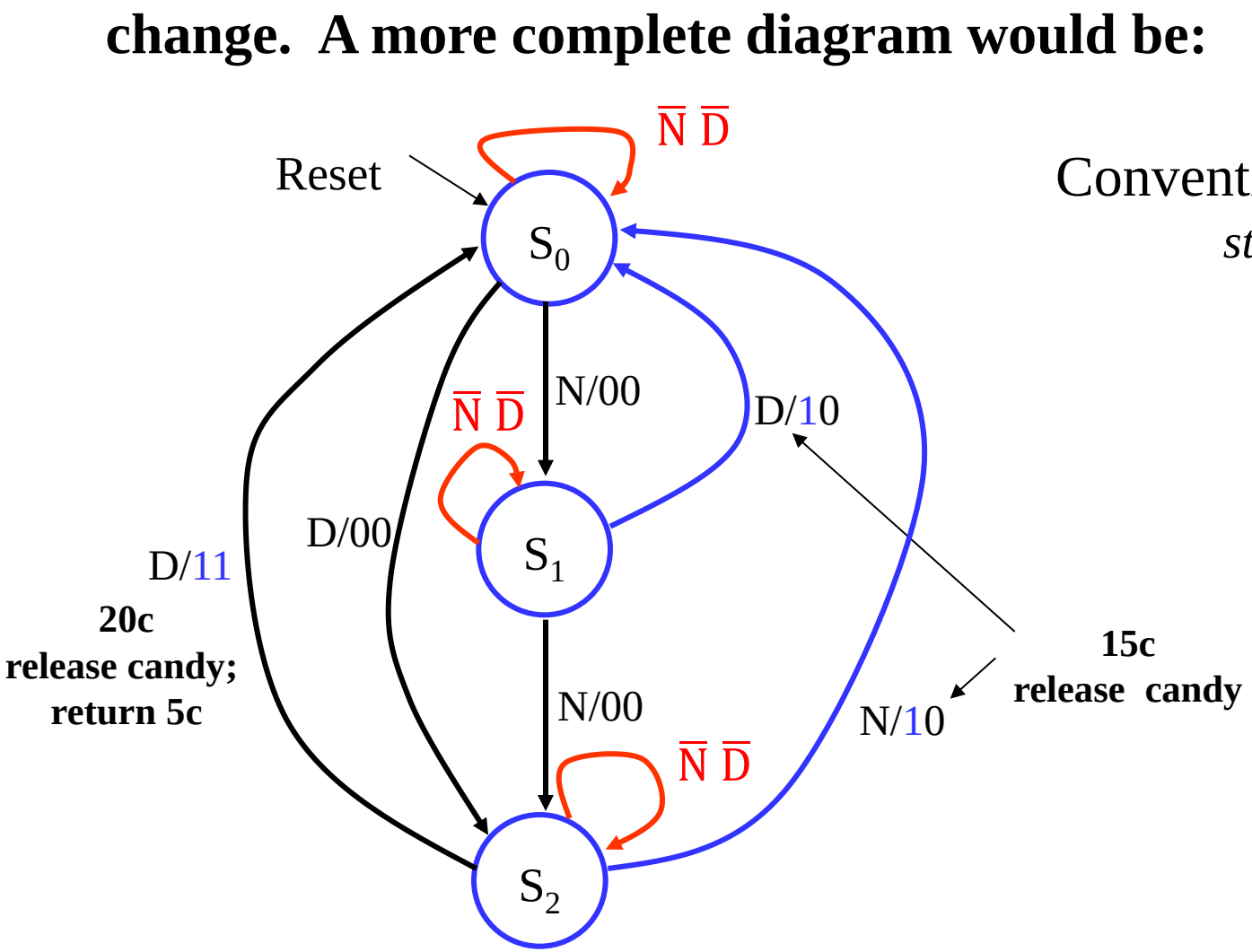
- **Vending machine specification:**
  - **Input: N, D**
  1) **N = 0 (no nickel); N = 1 (a nickel inserted)**
  2) **D = 0 (no dime); D = 1 (a dime inserted)**
  - **State: money received**
  - **Output: Y, Z**
  1) **Y = 0 (no candy); Y = 1 (candy provided)**
  2) **Z = 0 (no change); Z = 1 (change returned)**
  - **Function**
  1) $Y = \begin{cases} 0 & \text{if (money received + N/D} < 15 \text{ cents)} \\ 1 & \text{otherwise} \end{cases}$
  2) $Z = \begin{cases} 0 & \text{if (money received + N/D} \leq 15 \text{ cents)} \\ 1 & \text{otherwise} \end{cases}$

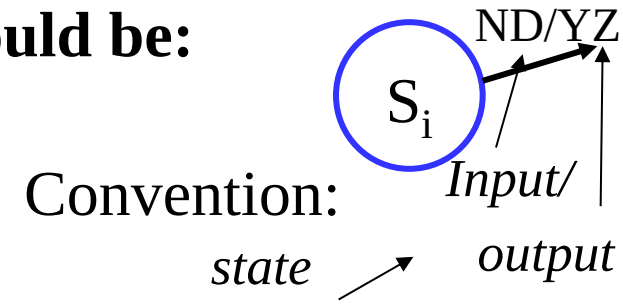
State Diagram (Mealy)



State Diagram (Mealy)

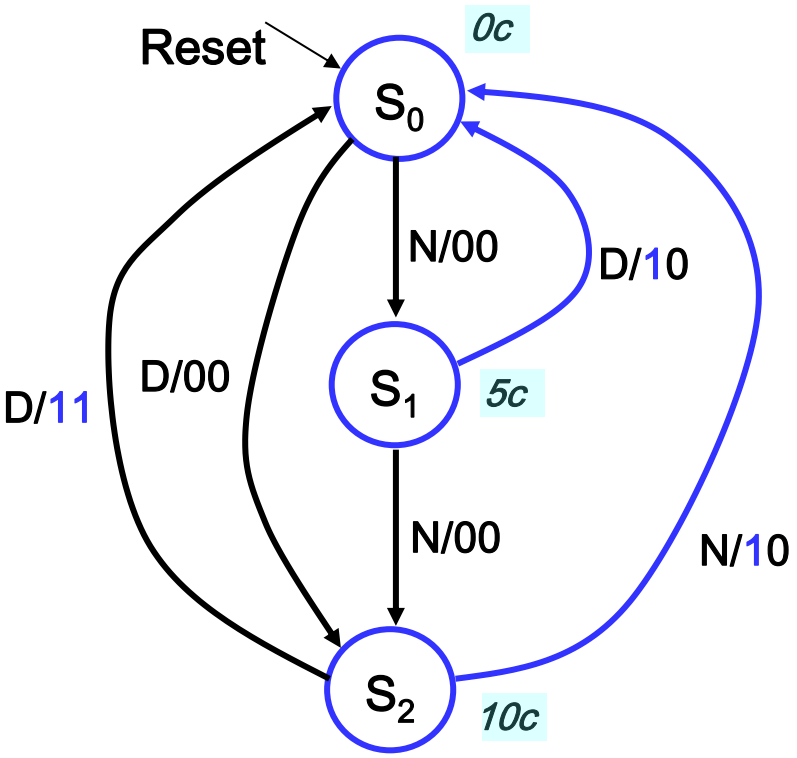
- **We assumed that when an input=0 there is no change. A more complete diagram would be:**





State Table for Mealy Model

Present State	Inputs N	Inputs D	Next States	Outputs Y	Outputs Z
S_0	0	0	S_0	0	0
S_0	0	1	S_2	0	0
S_0	1	0	S_1	0	0
S_0	1	1	x	x	x
S_1	0	0	S_1	0	0
S_1	0	1	S_0	1	0
S_1	1	0	S_2	0	0
S_1	1	1	x	x	x
S_2	0	0	S_2	0	0
S_2	0	1	S_0	1	1
S_2	1	0	S_0	1	0
S_2	1	1	x	x	x
S_3	0	0	x	x	x
S_3	0	1	x	x	x
S_3	1	0	x	x	x
S_3	1	1	x	x	x



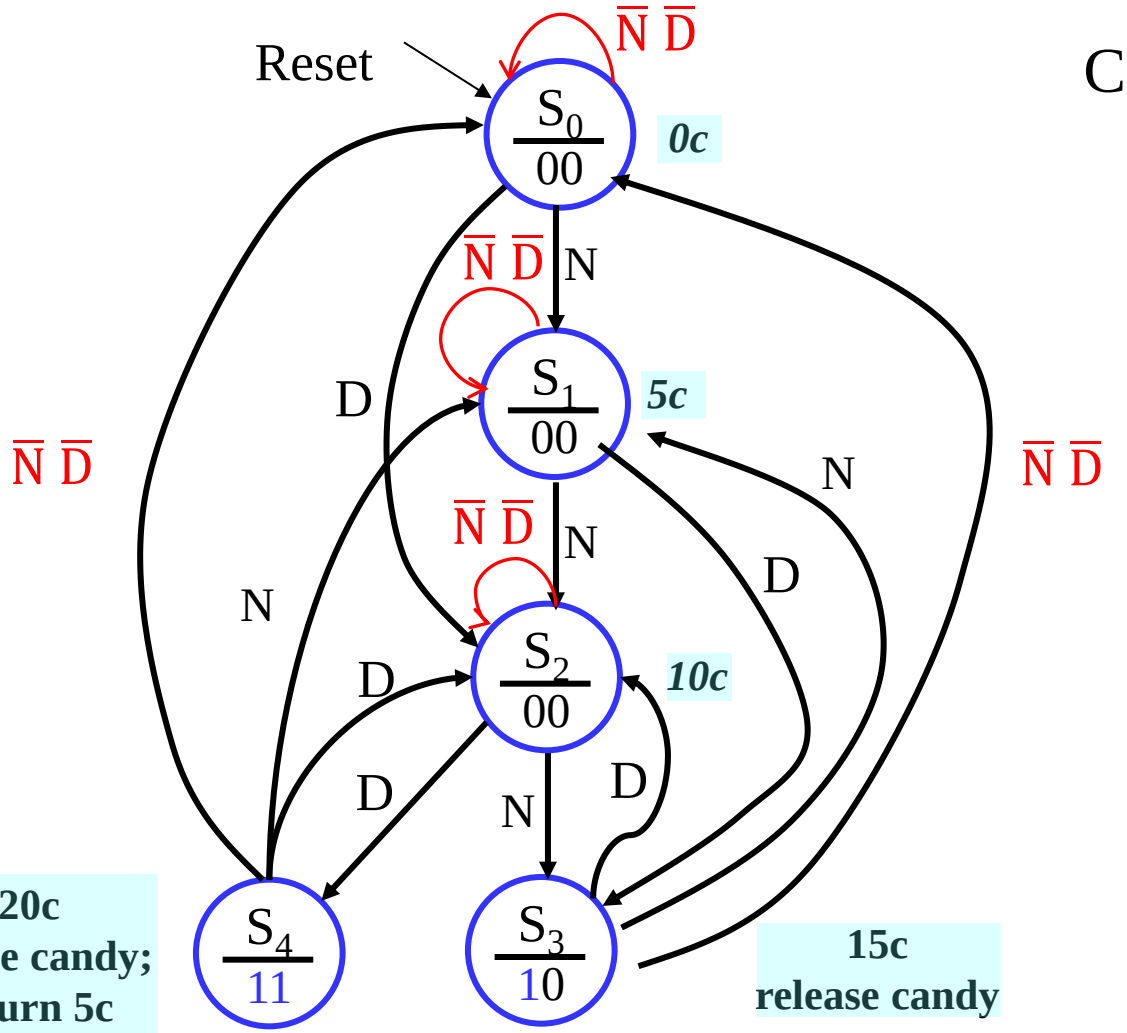
Vending Machine Specification—Moore Model

- **Vending machine specification:**
  - **Input: N, D**
  1) **N = 0 (no nickel); N = 1 (a nickel inserted)**
  2) **D = 0 (no dime); D = 1 (a dime inserted)**
  - **State: money received**
  - **Output: Y, Z**
  1) **Y = 0 (no candy); Y = 1 (candy provided)**
  2) **Z = 0 (no change); Z = 1 (change returned)**
  - **Function**

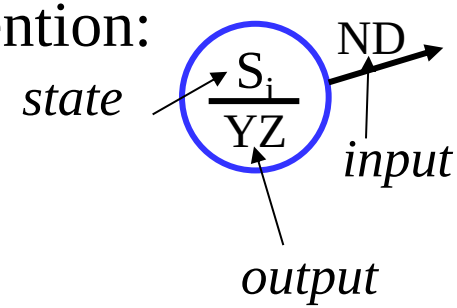
1) $Y = \begin{cases} 0 & \text{if (money received} < 15 \text{ cents)} \\ 1 & \text{otherwise} \end{cases}$

2) $Z = \begin{cases} 0 & \text{if (money received} \leq 15 \text{ cents)} \\ 1 & \text{otherwise} \end{cases}$

State Diagram (Moore)



Convention:



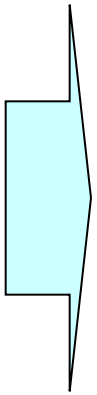
Requires 5 states

State Encoding and FF Type Selection

- **Three states requires 2 flip-flops: A and B**
- **Use the following encoding:**
  - **S_0 = 00**
  - **S_1 = 01**
  - **S_2 = 10**
- **Encoded state table**
- **We will use D flip-flops for state A and B**

Encoded State Table

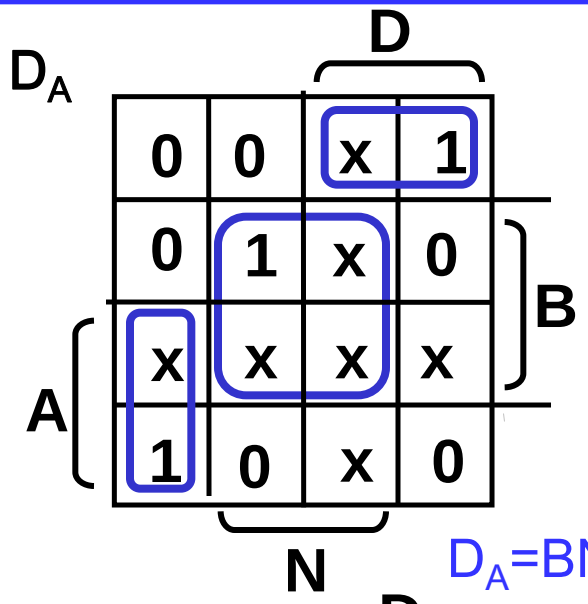
Present State	Inputs D N	Next States	Outputs Y Z
S_0	0 0	S_0	0 0
S_0	0 1	S_1	0 0
S_0	1 0	S_2	0 0
S_0	1 1	x	x x
S_1	0 0	S_1	0 0
S_1	0 1	S_2	0 0
S_1	1 0	S_0	1 0
S_1	1 1	x	x x
S_2	0 0	S_2	0 0
S_2	0 1	S_0	1 0
S_2	1 0	S_0	1 1
S_2	1 1	x	x x
S_3	0 0	x	x x
S_3	0 1	x	x x
S_3	1 0	x	x x
S_3	1 1	x	x x



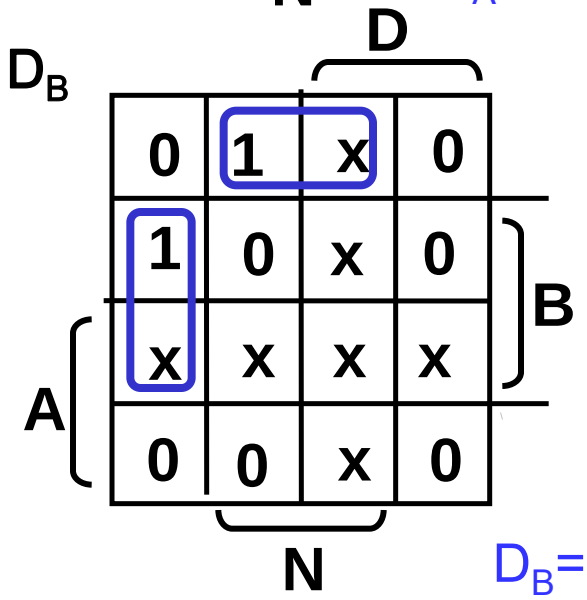
Present State A B	Inputs D N	Next States A^+ B^+	Outputs Y Z
0 0	0 0	0 0	0 0
0 0	0 1	0 1	0 0
0 0	1 0	1 0	0 0
0 0	1 1	x	x x
0 1	0 0	0 1	0 0
0 1	0 1	1 0	0 0
0 1	1 0	0 0	1 0
0 1	1 1	x	x x
1 0	0 0	1 0	0 0
1 0	0 1	0 0	1 0
1 0	1 0	0 0	1 1
1 0	1 1	x	x x
1 1	0 0	x	x x
1 1	0 1	x	x x
1 1	1 0	x	x x
1 1	1 1	x	x x

Input Equations for D_A and D_B

Present State A B	Inputs D N	Next States A^+ B^+	Outputs Y Z
0 0	0 0	0 0	0 0
0 0	0 1	0 1	0 0
0 0	1 0	1 0	0 0
0 0	1 1	x	x x
0 1	0 0	0 1	0 0
0 1	0 1	1 0	0 0
0 1	1 0	0 0	1 0
0 1	1 1	x	x x
1 0	0 0	1 0	0 0
1 0	0 1	0 0	1 0
1 0	1 0	0 0	1 1
1 0	1 1	x	x x
1 1	0 0	x	x x
1 1	0 1	x	x x
1 1	1 0	x	x x
1 1	1 1	x	x x



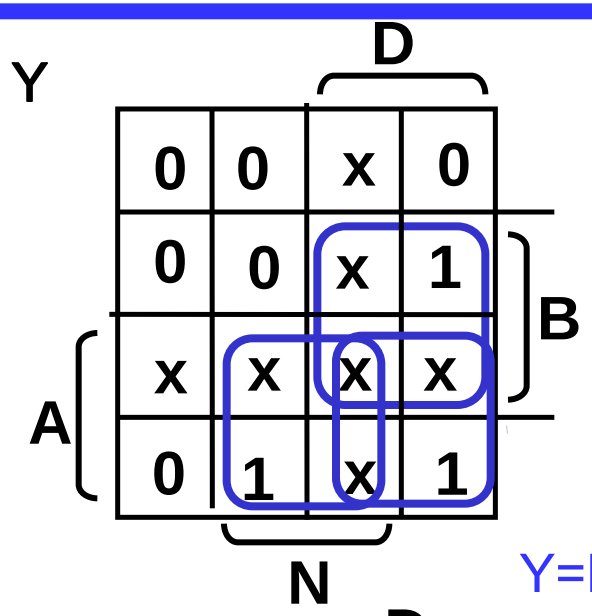
$D_A = BN + \bar{A}\bar{B}D + A\bar{N}\bar{D}$



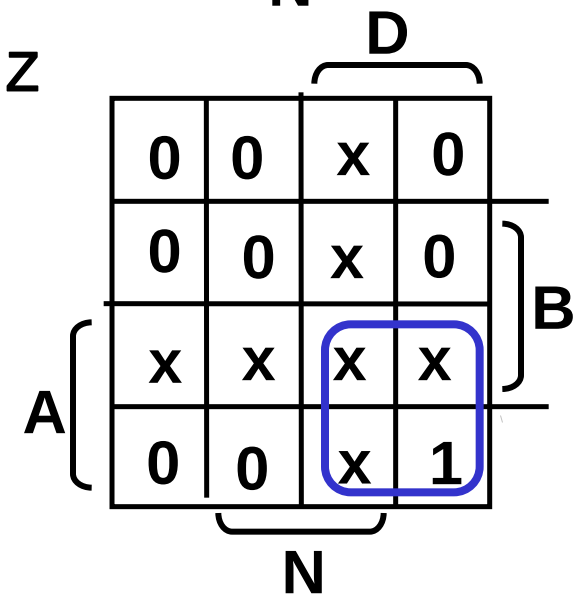
$D_B = B\bar{N}\bar{D} + \bar{A}BN$

Output Equations

Present State A B	Inputs N D	Next States A^+ B^+	Outputs Y Z
0 0	0 0	0 0	0 0
0 0	0 1	0 1	0 0
0 0	1 0	1 0	0 0
0 0	1 1	x	x x
0 1	0 0	0 1	0 0
0 1	0 1	1 0	0 0
0 1	1 0	0 0	1 0
0 1	1 1	x	x x
1 0	0 0	1 0	0 0
1 0	0 1	0 0	1 0
1 0	1 0	0 0	1 1
1 0	1 1	x	x x
1 1	0 0	x	x x
1 1	0 1	x	x x
1 1	1 0	x	x x
1 1	1 1	x	x x



$Y = BD + AN + AD$



$Z = AD$

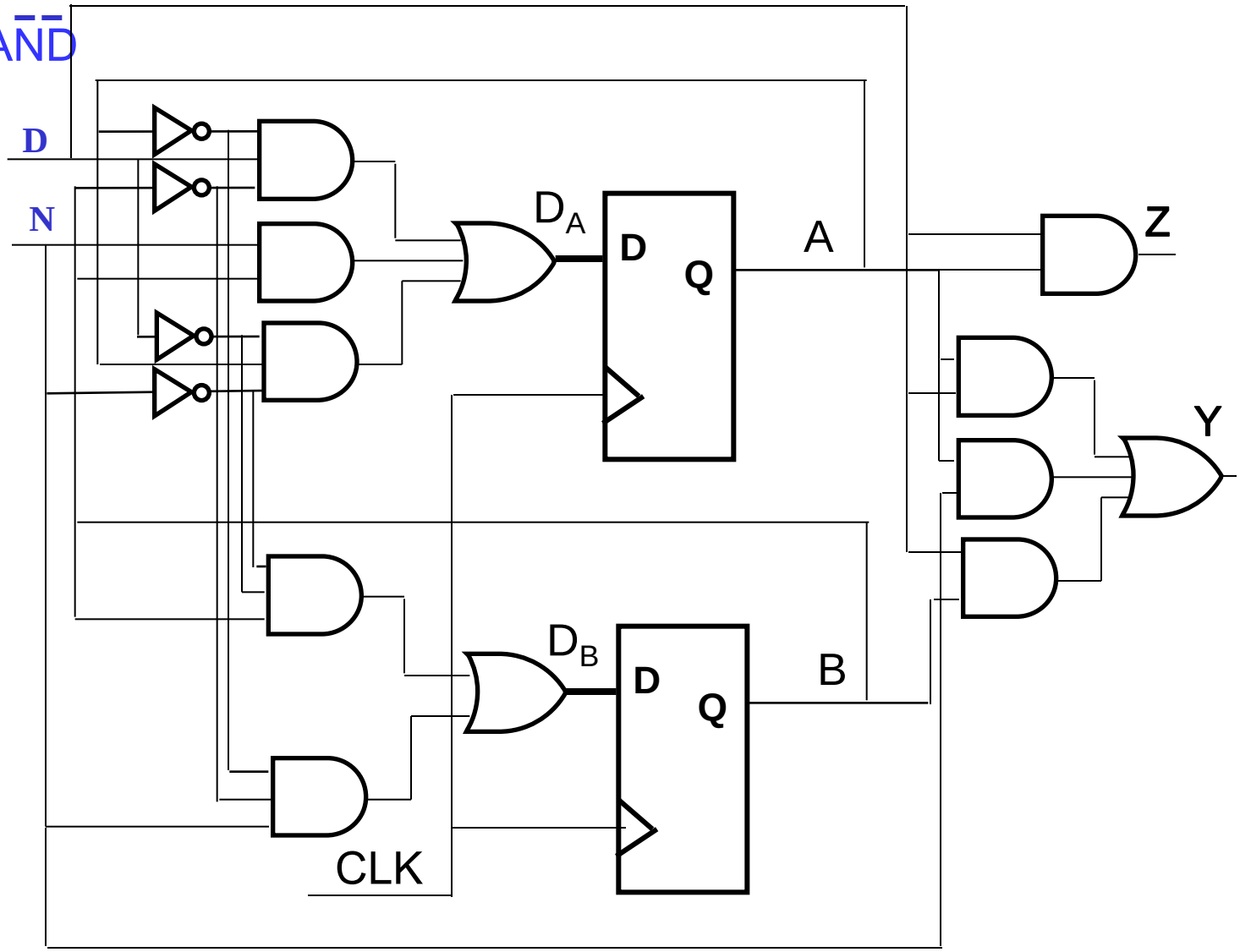
Circuit for Vending Machine

$D_A = BN + \bar{A}BD + A\bar{N}\bar{D}$

$D_B = B\bar{N}\bar{D} + \bar{A}BN$

$Y = BD + AN + AD$

$Z = AD$



Appendix A: Roofline Performance Model (1/7)

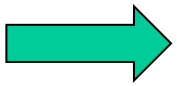
- **We hope to attain peak performance (FLOP/s), however, finite locality (reuse) and memory bandwidth limit performance.**
- **The Roofline model is a performance model seeking to give the limitations of a specific hardware in terms of algorithm implementation.**

Williams, Samuel, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 2009

Roofline Performance Model (2/7)

- **Performance Objective (Flops/s)**
  - **Flops/s is a measurement standing for floating point operations per second.**
  - **This is the number of mathematical operations (i.e. +, *, …) that a given algorithm does per second.**
- **E.g., the number of Flops for the daxpy loop (y = ax + y)**

```
void daxpy(double *x, double *y, double a, int n)
{
        int  i;
        for (i = 0; i < n; i++)
            y[i] = a*x[i] + y[i];
}
```



Flops # = 2×n=2n

Roofline Performance Model (3/7)

- **Memory Traffic (Bytes/s)**
  - **Bytes/s is the access speed of the memory at which data can be loaded or stored.**
- **E.g., the number of memory traffic (Bytes) for the daxpy loop (y = ax + y)**

```
void daxpy(double *x, double *y, double a, int n)
{
        int  i;
        for (i = 0; i < n; i++)
            y[i] = a*x[i] + y[i];
}
```

1 store

2 loads

Bytes # = ((2 loads)(8 bytes/load) + (1 store)(8 bytes/store)) × n =24n

Roofline Performance Model (4/7)

- **Arithmetic Intensity (AI)**
  - **AI is the measure of how many operations are done per bytes loaded or stored from memory.**

$$\text{AI} = \frac{\text{CPU performance}}{\text{Memory Traffic}} = \frac{\text{Flops/s}}{\text{Bytes/s}} = \frac{\text{Flops \#}}{\text{Bytes \#}}$$

  - **A high AI value indicates a lot of computation per load, while a low AI value means the algorithm is often waiting on data to be loaded before it can do anything.**
- **E.g., the AI value for the daxpy loop (y = ax + y)**

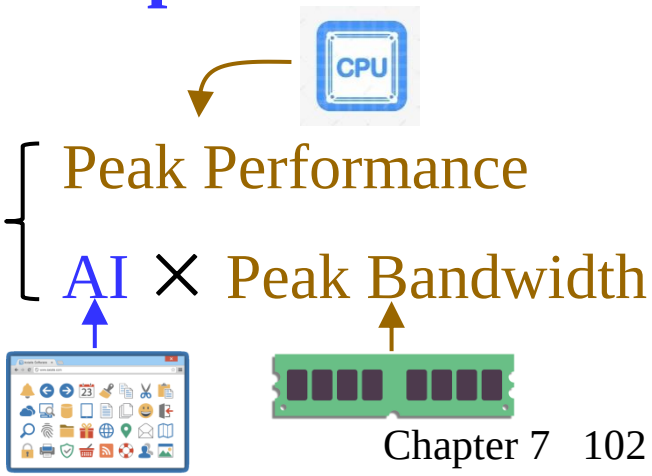
```
void daxpy(double *x, double *y, double a, int n)
{
        int  i;
        for (i = 0; i < n; i++)
            y[i] = a*x[i] + y[i];
}
```

$$\text{AI} = \frac{\text{Flops \#}}{\text{Bytes \#}} = \frac{2n}{24n} = \frac{1}{12}$$

Roofline Performance Model (5/7)

- **The Roofline model finds the upper bound on performance by using the peak performance and peak bandwidth.**
  - **Peak Performance (flops/second): The floating point max performance of the processor.**
  - **Peak Bandwidth (bytes/second): The fastest the processor can load data from memory.**
- **A formula for calculating attainable performance (Gflops/s)**

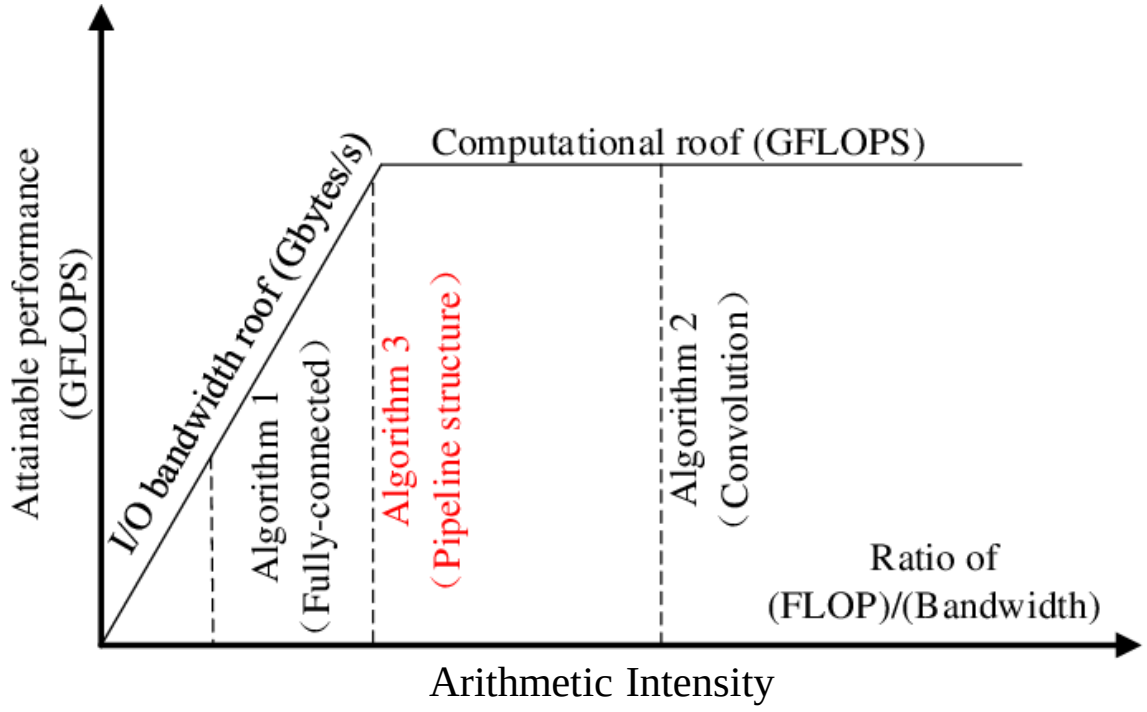
$$\text{Attainable Performance (AI)} = \min \begin{cases} \text{Peak Performance} \\ \text{AI} \times \text{Peak Bandwidth} \end{cases}$$



Roofline Performance Model (6/7)

$$\text{Attainable Performance (AI)} = \min \begin{cases} \text{Peak Performance} \\ \text{AI} \times \text{Peak Bandwidth} \end{cases}$$

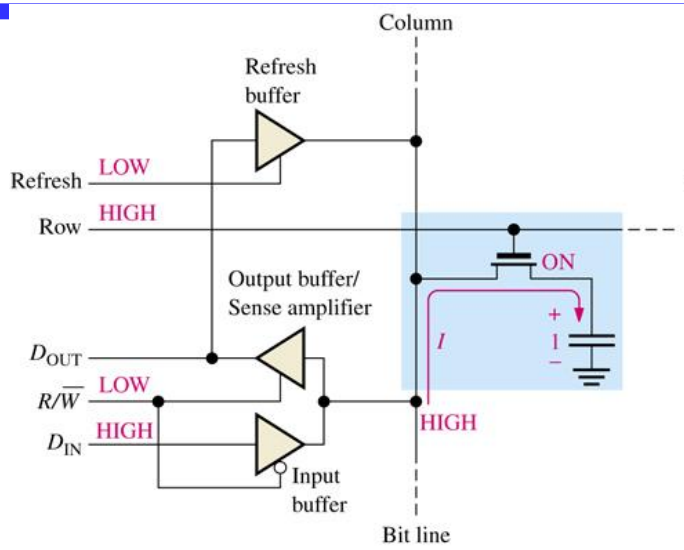
- **Roofline model**
  - **X-axis: Arithmetic Intensity (AI)**
  - **Y-axis: Attainable performance (Gflops/s)**



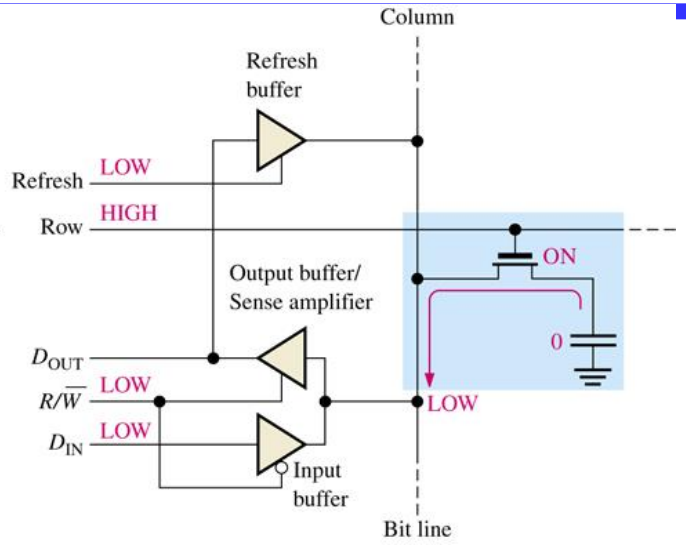
Roofline Performance Model (7/7)

- **For example, consider that a processor has**
  - **a peak performance of 64 Gflops/s and**
  - **a peak bandwidth of 16 GB/s**
- **If AI = $\frac{1}{8}$, then**
  - **Attainable Performance = min {64, $\frac{1}{8} \times 16$} = 2 Gflops/s**
  - **The attainable performance is limited by memory operations.**
- **If AI = 8, then**
  - **Attainable Performance = min {64, 8×16} = 64 Gflops/s**
  - **The attainable performance is limited by CPU's computation.**

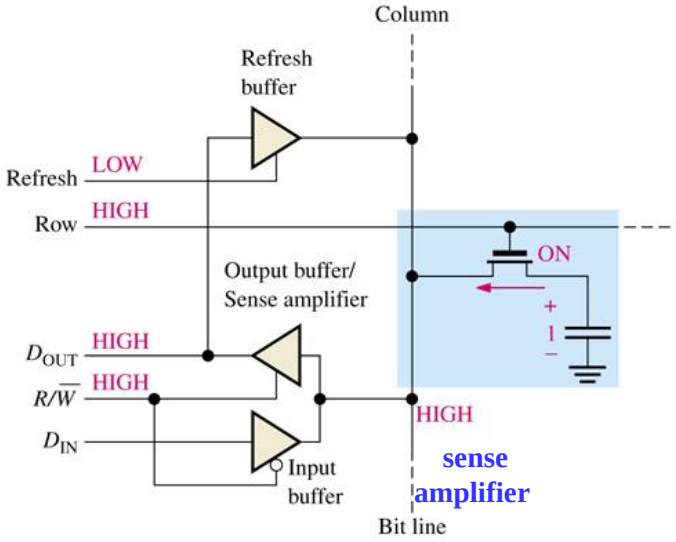
Appendix B: Dynamic RAM - Basic Operation



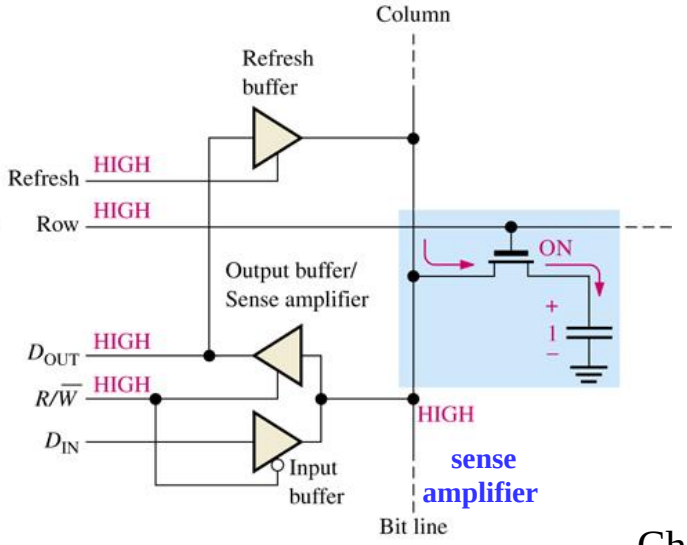
(a) Writing a 1 into the memory cell



(b) Writing a 0 into the memory cell



(c) Reading a 1 from the memory cell



(d) Refreshing a stored 1

Synchronous DRAM (continued)

- All signals are referenced to an external clock
  - Makes timing more precise with other system (CPU)
  - Synchronous registers appear on address input、data input and data output
- Burst oriented read and write accesses
  - Column address counter is used for addressing internal data to be transferred on each clock cycle
  - Beginning with the column address counts up to column address + burst length – 1
  - Burst length is programmable (e.g., 1,2,4,8)
- A user programmable mode register
  - CAS latency, burst length, burst type

Appendix C: Synchronous DRAM

- **SDRAM has a mode control word that can be loaded from the address bus.**

Mode Register

BA1	BA0	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
0	0	0	0	OP Code	0	0	CAS Latency			BT	Burst Length		

OP Code

A9	Write Mode
0	Burst Read and Burst Write
1	Burst Read and Single Write

Burst Type

A3	Burst Type
0	Sequential
1	Interleave

CAS Latency

A6	A5	A4	CAS Latency
0	0	0	Reserved
0	0	1	Reserved
0	1	0	2
0	1	1	3
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Reserved
1	1	1	Reserved

Burst Length

A2	A1	A0	Burst Length	
			A3 = 0	A3=1
0	0	0	1	1
0	0	1	2	2
0	1	0	4	4
0	1	1	8	8
1	0	0	Reserved	Reserved
1	0	1	Reserved	Reserved
1	1	0	Reserved	Reserved
1	1	1	Full Page	Reserved

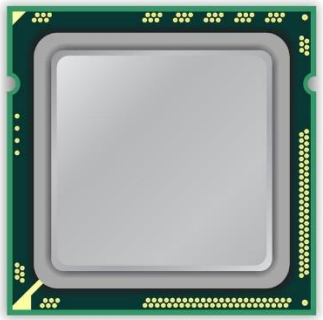
Synchronous DRAM (continued)

- **The order of the word retrieved depends on the least three significant bits of the column address and burst type (sequential or interleaved)**

Burst Length	Starting Column Address			Order of Accesses Within a Burst	
				Type = Sequential	Type = Interleaved
2			A0		
			0	0-1	0-1
			1	1-0	1-0
4		A1	A0		
		0	0	0-1-2-3	0-1-2-3
		0	1	1-2-3-0	1-0-3-2
		1	0	2-3-0-1	2-3-0-1
		1	1	3-0-1-2	3-2-1-0
8	A2	A1	A0		
	0	0	0	0-1-2-3-4-5-6-7	0-1-2-3-4-5-6-7
	0	0	1	1-2-3-4-5-6-7-0	1-0-3-2-5-4-7-6
	0	1	0	2-3-4-5-6-7-0-1	2-3-0-1-6-7-4-5
	0	1	1	3-4-5-6-7-0-1-2	3-2-1-0-7-6-5-4
	1	0	0	4-5-6-7-0-1-2-3	4-5-6-7-0-1-2-3
	1	0	1	5-6-7-0-1-2-3-4	5-4-7-6-1-0-3-2
	1	1	0	6-7-0-1-2-3-4-5	6-7-4-5-2-3-0-1
	1	1	1	7-0-1-2-3-4-5-6	7-6-5-4-3-2-1-0

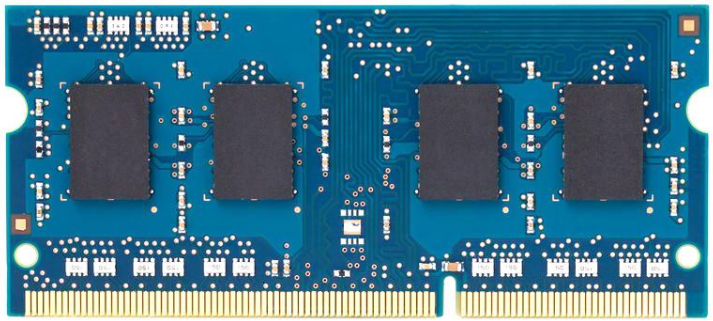
Appendix D: DRAM-row hammer

x86 CPU



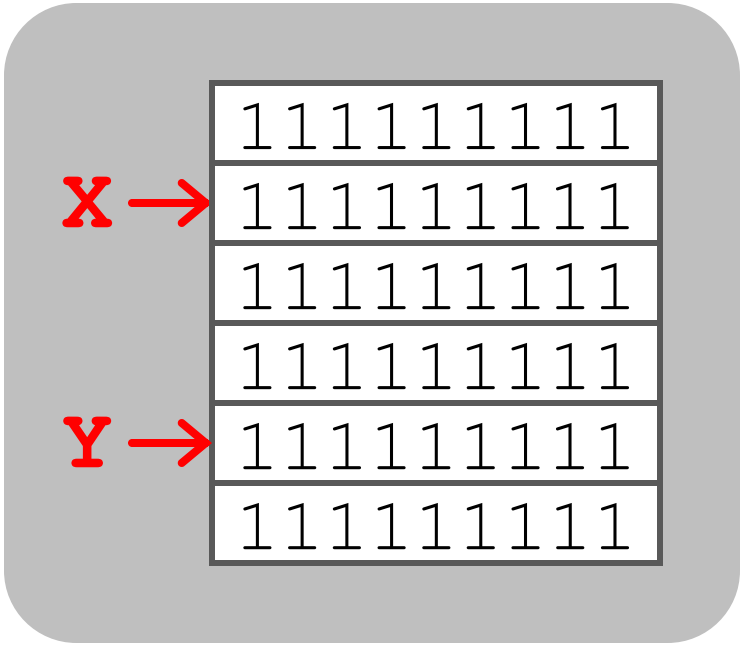
DDR3

DRAM Module



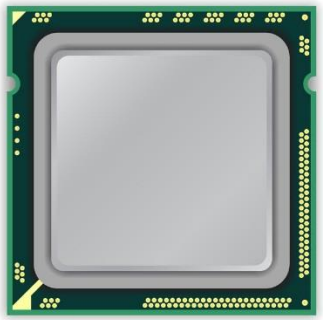
1. Avoid ***cache hits***
   – Flush **X** from cache

2. Avoid ***row hits*** to **X**
   – Read **Y** in another row



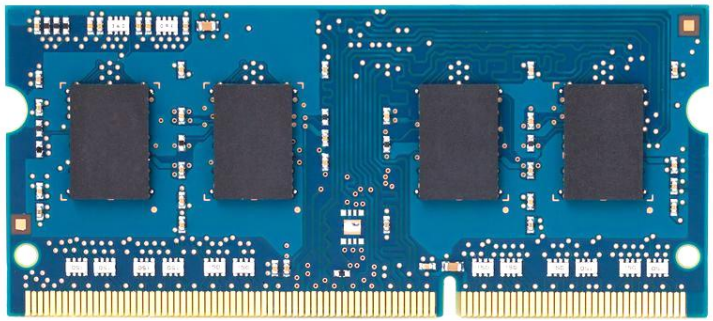
How to Induce Errors

x86 CPU

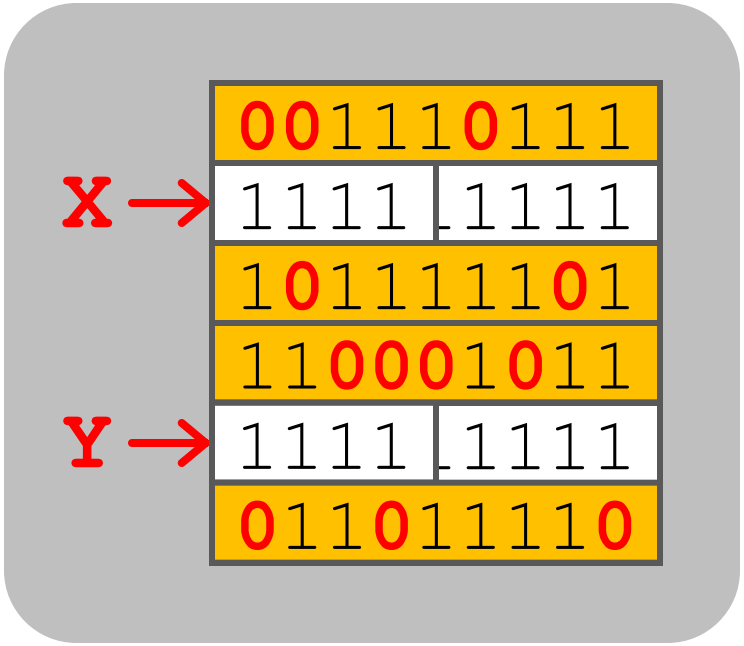


DDR3

DRAM Module

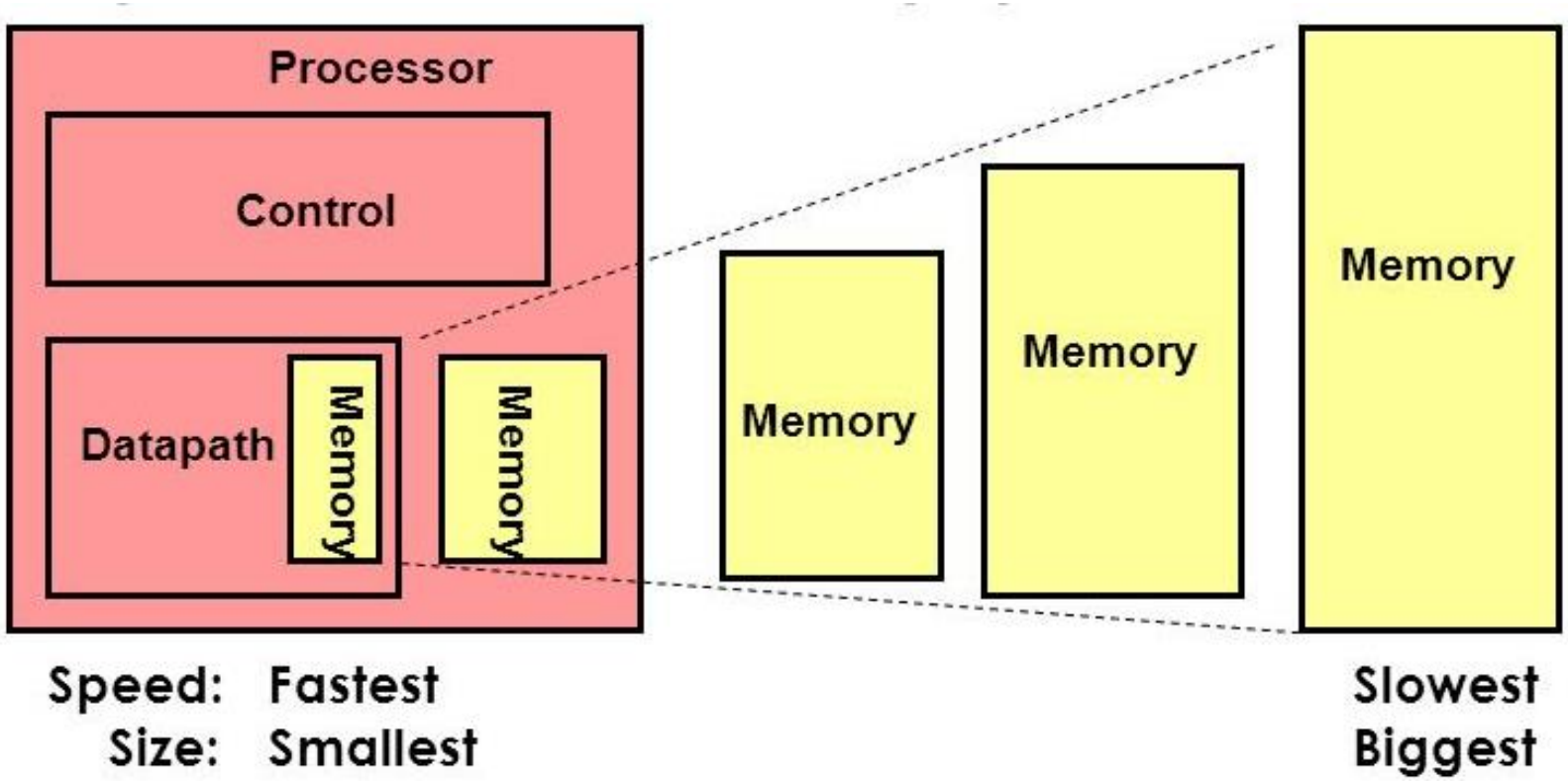


```
loop:
   mov (X), %eax
   mov (Y), %ebx
   clflush (X)
   clflush (Y)
   mfence
   jmp loop
```



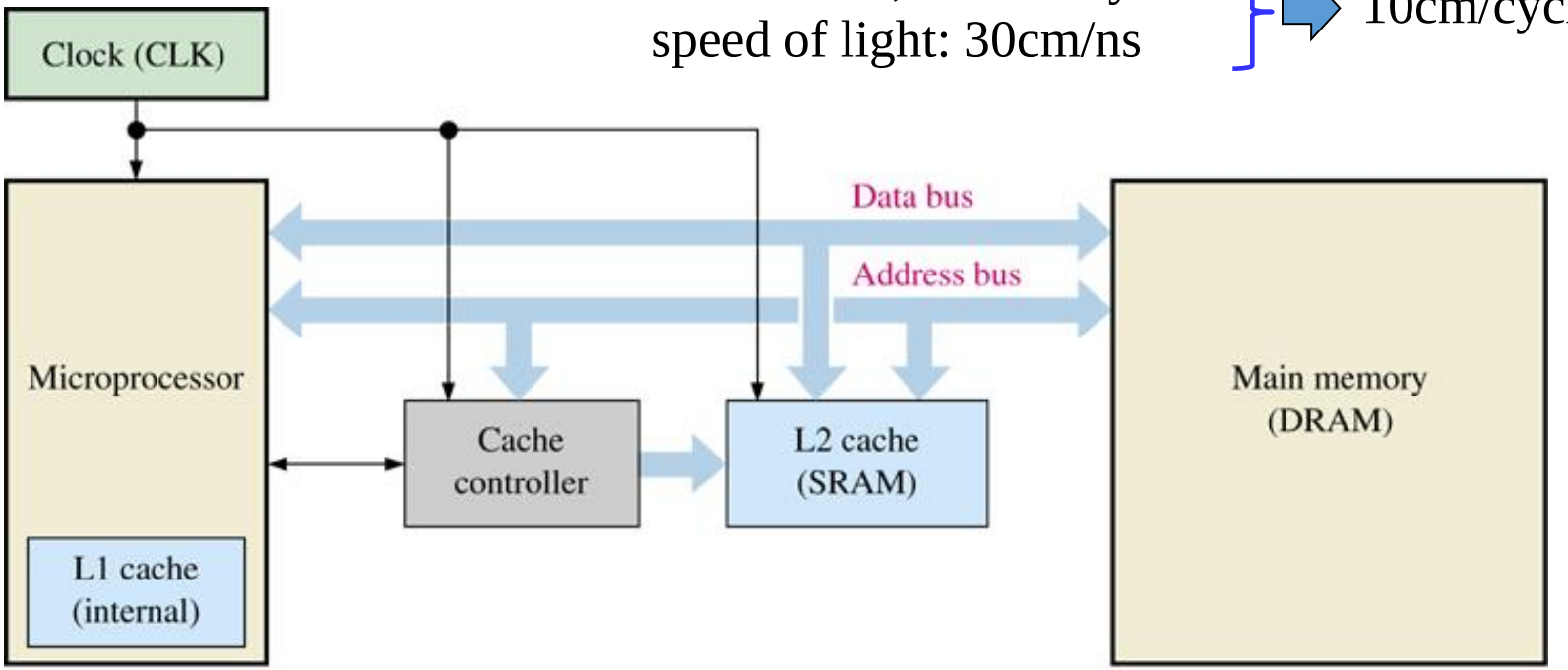
Memory Hierarchy in Computers

- **Good memory hierarchy design is important to the overall performance of a computer system.**



Memory Hierarchy in Computers (continued)

3GHz CPU, 0.33 ns/cycle
speed of light: 30cm/ns
→ 10cm/cycle



	L1 cache	L2 cache	main memory
Speed (ns) :	0.1 - 1	1 - 10	10 - 100
Size (bytes):	K	10K	M – G
Cost :	highest	→	lowest